

UNIVERSITATEA POLITEHNICA TIMIȘOARA
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

APLICAȚII CU AUTOMATE PROGRAMABILE

2018

Cuprins

1.Introducere	4
2.Structura hardware	5
2.1. Structura hardware a unui automat programabil.....	5
2.2. Platforma PROSIM	9
3.Mediul de programare TIAPortal (Step7)	14
3.1 Interfața mediului de dezvoltare TIA Portal.....	14
3.2 Crearea unui proiect	17
3.3 Crearea configurației hardware	18
3.4 Compilarea și încărcarea aplicației	22
3.5 Depanarea aplicației	26
4. Limbajul de programare LADDER.....	29
4.1 Introducere	29
4.2 Intrări, ieșiri și zone de memorie	30
4.3 Operarea la nivel de bit. Contacte, bobine și bistabile	33
4.3.1 Contacte și bobine	34
4.3.2 Contacte și bobine cu funcții speciale. Blocuri de detecție a fronturilor de semnal. Utilizarea zonelor de memorie.....	39
4.3.3 Bistabile.....	43
4.4 Timere	46
4.4.1 Timere de tip TON.....	46
4.4.2 Timer de tip TOF.....	47
4.4.3 Timer de tip TP	48
4.4.4 Timer de tip TONR.....	50
4.5 Countere	51
4.5.1 Counter de tip CTU.....	52
4.5.2 Counter de tip CTD.....	53
4.5.3 Counter de tip CTUD	54
4.6 Transmiterea și recepționarea de date.....	55
4.7 Funcții bloc	60
5.Aplicații	68
5.1 Sistem de trecere la nivel cu calea ferată	68
5.2 Parcare de mașini.....	70

5.3 Afișaj șapte segmente	72
---------------------------------	----

1.Introducere

Acest material de curs prezintă informații referitoare la modul de lucru cu un automat secvențial. Sunt prezentate pe parcursul acestui curs o serie de informații care au ca și scop final deprinderea de cunoștințelor de bază necesare pentru programarea automatelor secvențiale utilizând limbajul de programare LADDER. Pe baza cunoștințelor dobândite după parcurgerea acestui material de curs, cititorul va deprinde noțiuni de bază despre modul de funcționare și programare a automatelor secvențiale. Aceste noțiuni de bază sunt suficiente pentru a permite dezvoltarea de aplicații pentru automatele programabile care utilizează limbajul LADDER, înțelegerea, menținerea și depanarea de aplicații care sunt scrise pentru a rula pe un automat programabil folosind limbajul de programare LADDER.

Acest curs prezintă noțiuni de bază referitoare la structura hardware a unui automat programabil (pentru exemplificare se va folosi un automat SIEMENS), noțiuni de bază referitoare la programarea folosind diagrame LADDER și concepte de programare a automatelor cu ajutorul unor exemple practice.

Cursul este structurat în 5 capitole în care sunt prezentate cunoștințe generale legate de structura hardware a unui automat programabil (Cap.2), modul de utilizare a mediului de dezvoltare TIA Portal (Cap.3), blocurile care pot să fie utilizate pentru construirea de diagrame LADDER complexe în mediul de dezvoltare TIA Portal (Cap.4). În încheierea acestui curs (Cap.5), sunt prezentate o serie de exemple de aplicații cu caracter practic care au fost implementate folosind diagrame LADDER.

2.Structura hardware

În cadrul acestui capitol se va prezenta structura hardware a unui automat programabil și modul în care automatul programabil este conectat la platforma PROSIM, platforma care permite simularea unor procese din mediul industrial și permite testarea aplicațiilor dezvoltate utilizând limbajul de programare bazat pe diagrame LADDER.

2.1. Structura hardware a unui automat programabil

În cadrul acestui capitol se vor prezenta câteva considerente referitoare la structura hardware a unui automat programabil produs de firma SIEMENS. Automatele programabile au apărut pe piață în jurul anilor 1970 datorită necesității înlocuirii logicii cablate și pentru ele se folosește destul de des denumirea de PLC (Programmable Logic Controllers). Datorită faptului că logica cablată folosită pentru conducerea unui proces era greoaie, necesita mult timp pentru implementare, iar în cazul în care se dorea fixarea unei erori apărute în cadrul proiectării era foarte dificil de realizat acest lucru s-a dorit înlocuirea logicii cablate cu logica programată. Prin folosirea logicii programate se reușea eliminarea acestor neajunsuri pe care logica cablată le avea.

Folosirea automatelor programabile pentru controlul proceselor aduce o serie de beneficii cum ar fi cele enumerate mai jos:

- facilitarea procesului de testare a codului care rulează pe automat deoarece mediul de dezvoltare care este folosit pentru a scrie programe care rulează pe automat dispune de un simulator pe care poate fi testat codul scris înainte ca acesta să fie testat pe sistemul pentru care a fost creat
- rezistența automatelor la medii de lucru extreme, respectiv la perturbațiile puternice care pot să apară în mediul de lucru unde automatul își desfășoară activitatea
- ușurință în implementarea procesului de conducere pentru o gamă variată de procese din mediul tehnic

- particularizarea hardware-ului folosit pentru controlul procesului prin adăugarea de module suplimentare în funcție de procesul care se dorește a fi controlat
- cost redus, deoarece în ultimi ani prețul automatelor programabile a început să devină din ce în ce mai accesibil.
-

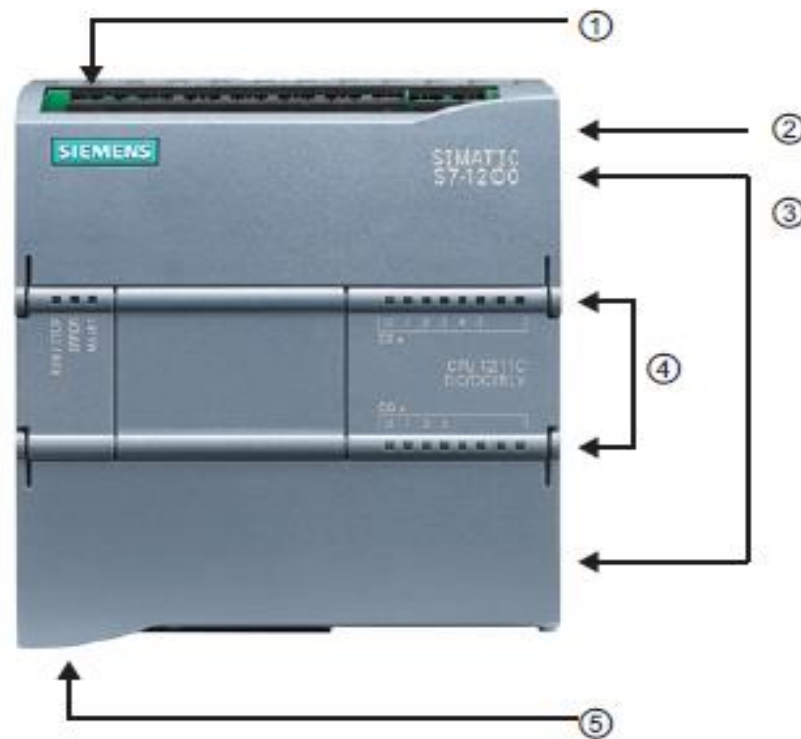


Fig 2.1.1 – Automat SIEMENS seria 1200

Din punct de vedere hardware, cel mai simplu automat care poate fi folosit pentru controlul unui proces este reprezentat de unitatea centrală de prelucrare. Unitatea centrală de prelucrare pentru un automat din seria SIEMENS 1200 conține un microprocesor puternic, o sursă de alimentare integrată, circuite pentru intrare/ieșire grupate sub forma de regiștrii și o interfață de comunicare PROFINET care folosește protocolul de comunicare TCP/IP. Unitatea centrală de prelucrare pentru un automat din seria SIEMENS 1200 este prezentat în cadrul figurii 2.1.1. În cadrul acestei figuri se pot observa marcate cu numere de la 1 la 5 principalele elemente (porturi de alimentare, leduri de stare, e.t.c) ale unității centrale de prelucrare a informației. Acestea sunt:

- alimentare cu tensiune (1)
- slot pentru cardul de memorie (2)
- capac protecție cablaj (3)
- leduri de stare pentru porturile de intrare/ieșire (4)
- conector RJ45 pentru comunicarea în rețea (5)

Automatul studiat este SIEMENS 1214C. Acest automat dispune de 14 intrări logice, 10 ieșiri logice, respectiv de 2 intrări analogice. Configurația de bază pentru acest automat poate să fie extinsă prin adăugarea de până la 8 module suplimentare de intrări/ieșiri, prin adăugarea de până la 3 module suplimentare folosite pentru comunicare cu alte automate sau prin adăugarea unui modul de semnale. Modulul de semnale poate fi folosit pentru conectarea unui panou de tip touch-screen, conectarea unui termocuplu sau conectarea de intrări/ieșiri suplimentare.

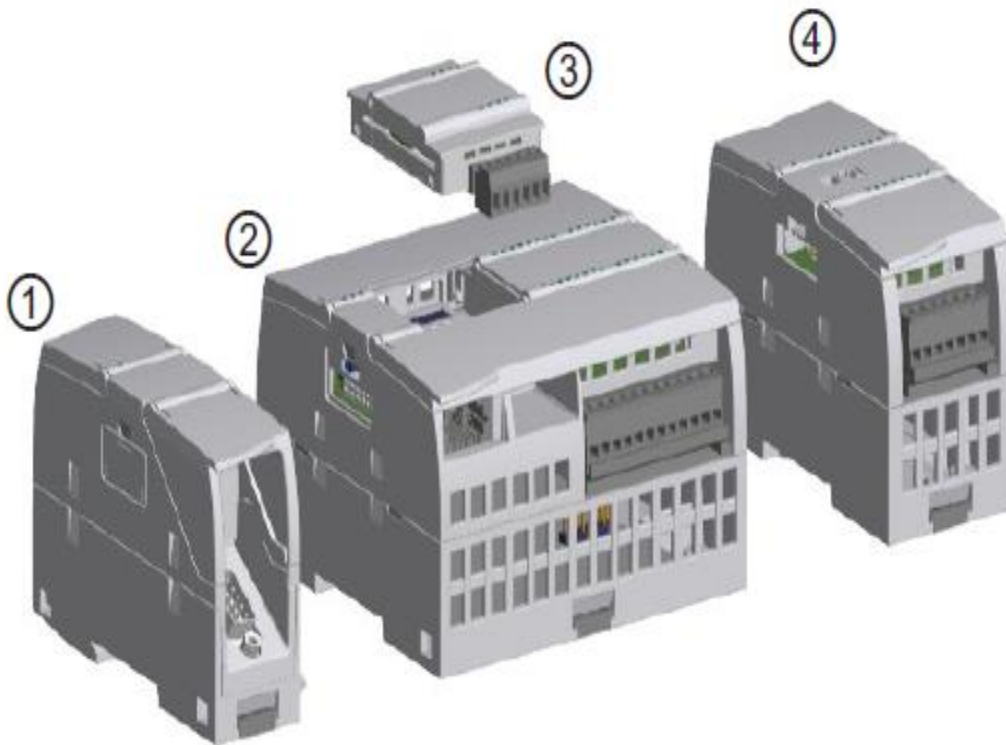


Fig 2.1.2 – Automat SIEMENS seria 1200, configurație extinsă

Configurația extinsă a unui automat programabil din seria SIEMENS 1200 este prezentată mai sus în cadrul figurii 2.1.2. Se observă în cadrul acestei figuri faptul că modulele de comunicare se pot conecta la unitatea centrală de prelucrare (2) în partea stângă a acesteia (1), modulul de semnale se conectează la unitatea centrală de prelucrare în partea superioară a acesteia (3), iar modulele suplimentare de intrări/ieșiri digitale se conectează în partea dreaptă a unității centrale de prelucrare (4). De asemenea configurația automatului poate să fie extinsă prin adăugarea de surse de alimentare suplimentare.

Utilizând o configurație extinsă, se poate adapta foarte ușor structura hardware a unui automat programabil la necesitățile pe care le impune conducerea unui proces din mediu industrial. Această flexibilitate a modului de configurare a structurii hardware este de asemenea benefică din punct de vedere economic.

Unitatea centrală de prelucrare conține o gamă variată de resurse hardware care pot să fie folosite de către programator pentru scrierea codului care rulează pe automatul programabil. Dintre aceste resurse vom menționa existența timerelor (temporizator), respectiv a counterelor (numărător), resurse care sunt folosite foarte des în programarea automatelor.

Din punct de vedere al modulelor externe, se menționează utilitatea display-urilor care folosesc tehnologia touch-screen. Acestea se pot folosi pentru a implementa interfețe utilizator care să ilustreze într-un mod grafic starea procesului condus respectiv anumite valori obținute de la senzorii existenți în cadrul procesului condus folosind automatul.

Din punct de vedere al modului în care automatul rulează aplicația care a fost scrisă pentru a rula pe acesta, se menționează faptul că orice cod de aplicație care rulează pe un automat este executat în mod secvențial. Automatele din seria SIEMENS 1200 folosesc ca și limbaj de programare, diagrame de tip LADDER. Aplicația care rulează pe automat este constituită dintr-un set de diagrame(rețele) care vor fi evaluate de către automat într-o manieră secvențială (una după alta). Timpul în care fiecare rețea este evaluată de către unitatea centrală de prelucrare

depinde de complexitatea rețelei, cele mai simple rețele sunt evaluate de obicei în câteva microsecunde.

2.2. Platforma PROSIM

Platforma PROSIM este un simulator avansat de procese ce poate să fie folosit pentru învățarea principiilor de programare ale echipamentelor de tip PLC (Programmable Logic Controller). Pentru această platformă există un număr de 33 de machete care simulează funcționarea unor procese din mediul înconjurător.

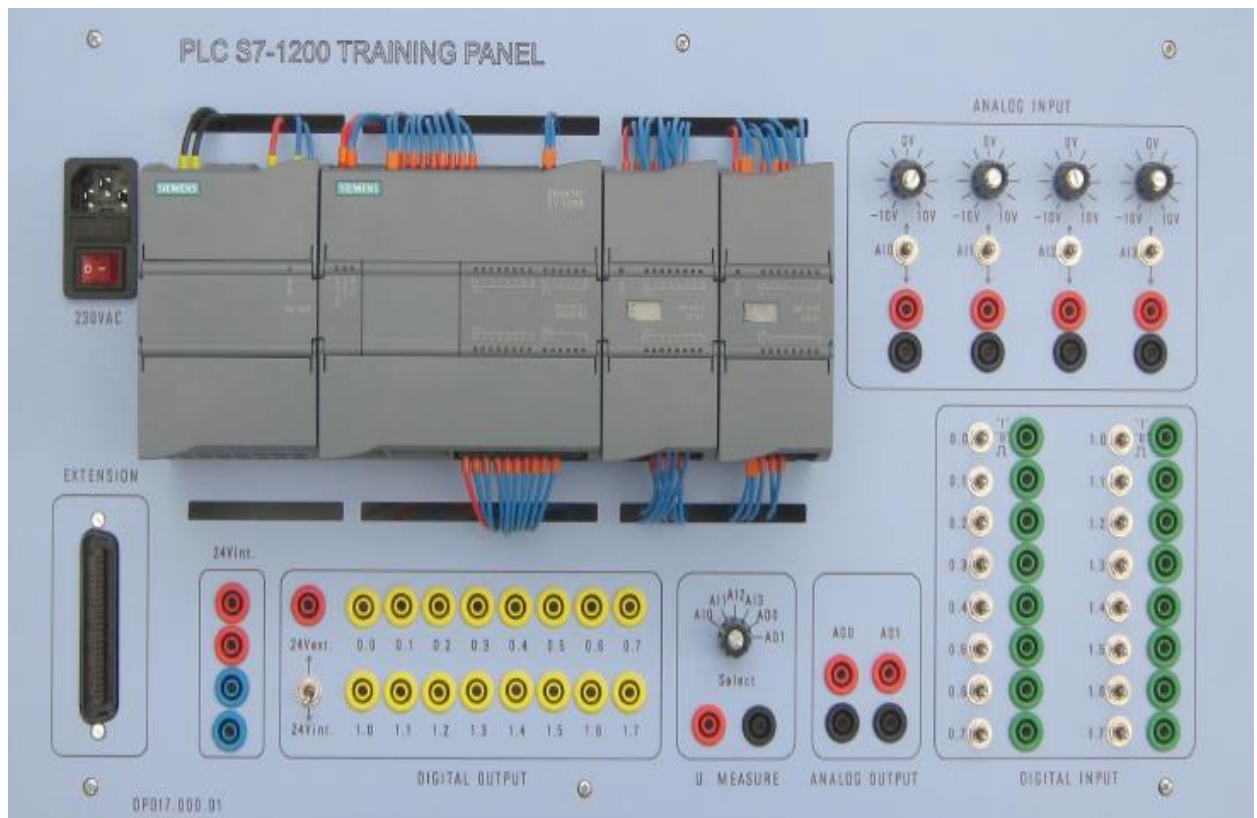


Fig 2.2.1 – Automat SIEMENS montat pe platforma PROSIM

Din punct de vedere constructiv, platforma PROSIM este alcăuită din două module care comunică între ele printr-o conexiune realizată prin fire sau printr-o magistrală cu 50 de pini. Pe unul din module este montat automatul programabil împreună cu modulele suplimentare (un

modul de intrări – ieșiri digitale, un modul de intrări analogice și un modul pentru alimentare). Această configurație este ilustrată în cadrul figurii 2.2.1. Toate intrările și toate ieșirile au fost proiectate respectând standardele industriale, adică 24 de volți pentru mărimile digitale și nivelul de tensiune 0-10 volți pentru mărimile analogice, lucrând practic cu aceleași nivele de semnale care sunt întâlnite și în cadrul echipamentelor industriale.

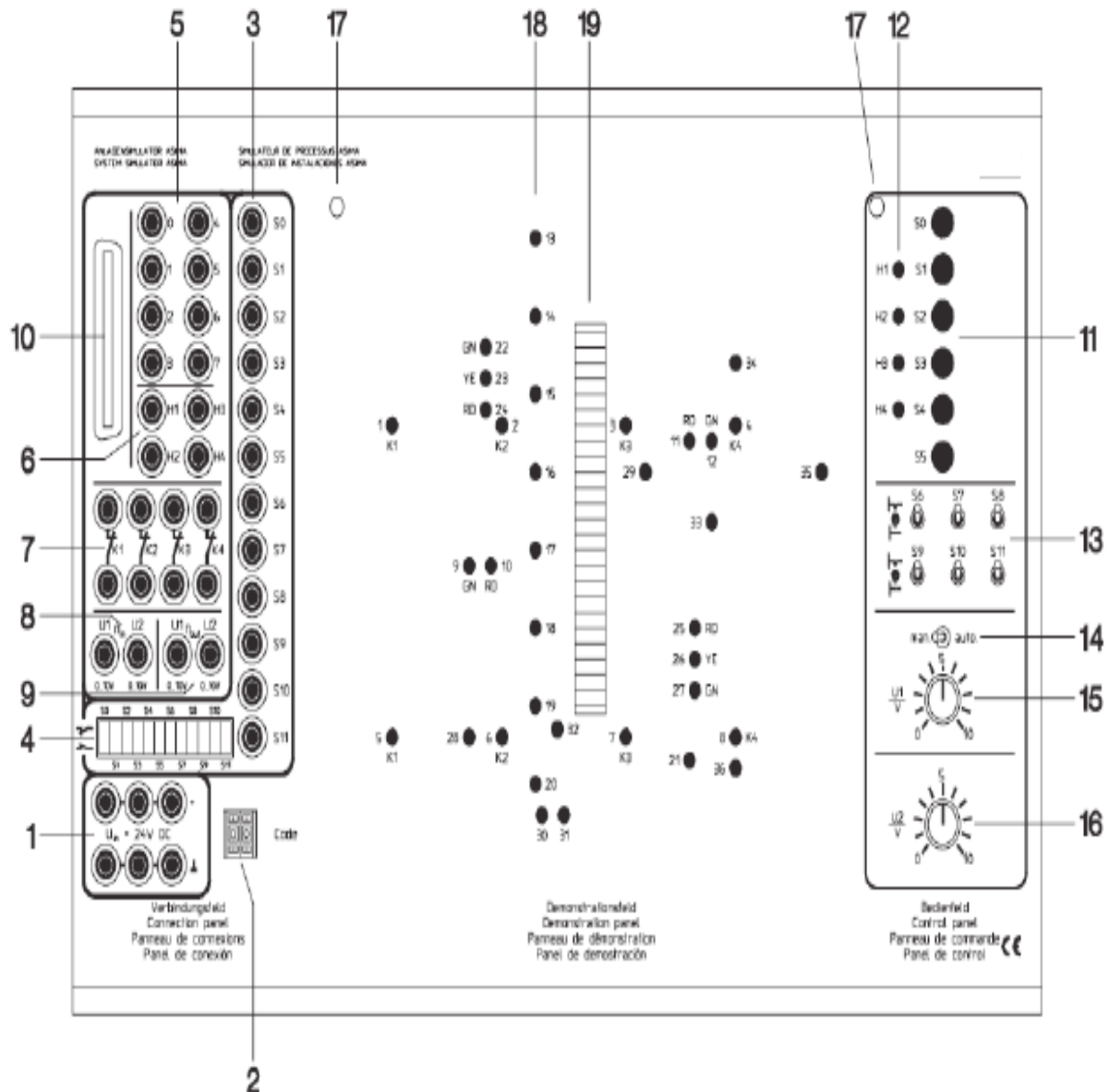


Fig 2.2.2 – Platformă PROSIM – simulare proces

Cel de al doilea modul este utilizat pentru simularea funcționării machetelor (preluarea datelor de la senzori, acționarea elementelor de execuție). Cel de al doilea modul este prezentat în cadrul figurii 2.2.2 unde se pot observa principalele părți componente ale modului:

- (1) Bornele de alimentare ale panoului. Panoul trebuie alimentat cu o tensiune stabilizată de 24V (spre exemplu din sursa automatului). În cazul utilizării cablului panglică, alimentarea pe aceste borne nu mai este necesară
- (2) Selector mască
- (3) Borne pentru conectarea senzorilor și contactelor la automatul programabil. În cazul în care se folosește magistrala pentru conectarea modulelor, aceste borne nu se folosesc.
- (4) Set de switch-uri utilizat în setarea comportamentului senzorilor și contactelor (normal închis sau normal deschis) Notă: Tipul celor 12 semnale digitale (activ pe 0 sau activ pe 1 altfel spus, normal închis sau normal deschis) care ies din panoul simulator este selectat pentru fiecare în parte cu ajutorul acestor switch-uri. Pentru aceasta sunt utilizate doar switch-urile de pe pozițiile impare (numaratoarea facandu-se de la zero, începand de la stânga); astfel al doilea switch de la stanga la dreapta (numarul 1) va controla comportamentul semnalului S0, al patrulea (numarul 3) va controla comportamentul semnalului S1 s.a.m.d. Switch-urile de pe pozițiile pare nu sunt folosite și pot avea orice stare fara a influența în vreun fel funcționarea sistemului.
- (5) Borne pentru conectarea automatului la simulator (actuatori). În cazul utilizării cablului panglică, aceste borne nu se folosesc (excepție pentru alimentarea ledurilor în cadrul planșelor care precizează acest lucru)
- (6) Borne pentru conectarea la automat a lămpilor indicatoare. În cazul utilizării cablului panglică, aceste borne nu se folosesc (excepție pentru alimentarea în paralel cu o alta ieșire). Pe măști, lămpile sunt notate cu simbolul P (lampa 1 are simbolul P1, lampa 2 are simbolul P2, s.a.m.d. Pe borne, semnalele aferente sunt notate cu simbolul H(H1,H2,samd). Cele doua notatii desemneaza aceleasi semnale iar in documentul current ne vom referi la ele prin notatia P.
- (7) Contactele normal închise a 4 relee ce pot fi folosite pentru ilustrarea caracteristicilor de siguranță.

- (8) Intrări analogice în simulator. Tensiunea pe aceste borne respectă standardul industrial 0-10V. În cazul utilizării cablului panglică, aceste borne nu se folosesc.
- (9) Ieșiri analogice din simulator, de asemenea pe standardul industrial 0-10V. În cazul utilizării cablului panglică, aceste borne nu se folosesc
- (10) Conector cablu panglică
- (11) Butoane cu revenire. Comportamentul lor (normal deschis sau normal închis) este setat de swith-urile din grupul (4)
- (12) Lămpi de semnalizare
- (13) Butoane cu funcție dublă (cu sau fara revenire). Comportamentul lor (normal deschis sau normal inchis) este setat de swith-urile din grupul (4)
- (14) Selector "Manual/Automat"
- (15) Potențiomtru - dacă selectorul (14) este pe pozitia "man", valoarea setată de acest potențiomtru (tensiune în intervalul 0-10V) este aplicată direct simulatorului de proces și în funcție de masca aplicată, se setează un parametru (spre exeplu o rată de încărcare a unui bazin). Pe poziția "auto" a selectorului (14) valoarea setată pe acest potențiomtru este ignorata, în locul ei fiind folosita intrarea U1.
- (16) Potențiomtru - valoarea setată de acest potențiomtru (tensiune in intervalul 0-10V) modifică un parametru de proces (spre exemplu rata de descarcare a unui bazin)
- (17) Ghidaje pentru fixarea măștii
- (18) Leduri ce indica starea actuatorilor procesului simulat
- (19) Bara de leduri care indica miscarea sau procesele de umplere/golire ale unor recipiente.

În momentul în care se aplică o machete pe modulul PROSIM, acesta va arăta similar cu imaginea din figura 2.2.3. în cadrul acestei figure se pot observa următoarele notații:

- (20) Masca aplicată
- (21) Numărul măștii curente (selectorul 2 trebuie setat pe acest număr)
- (22) Tabelul de semnale de intrare (actuatori și lămpi) aferent măștii curente
- (23) Etichetele semnalelor de ieșire din simulator (senzor și contacte)

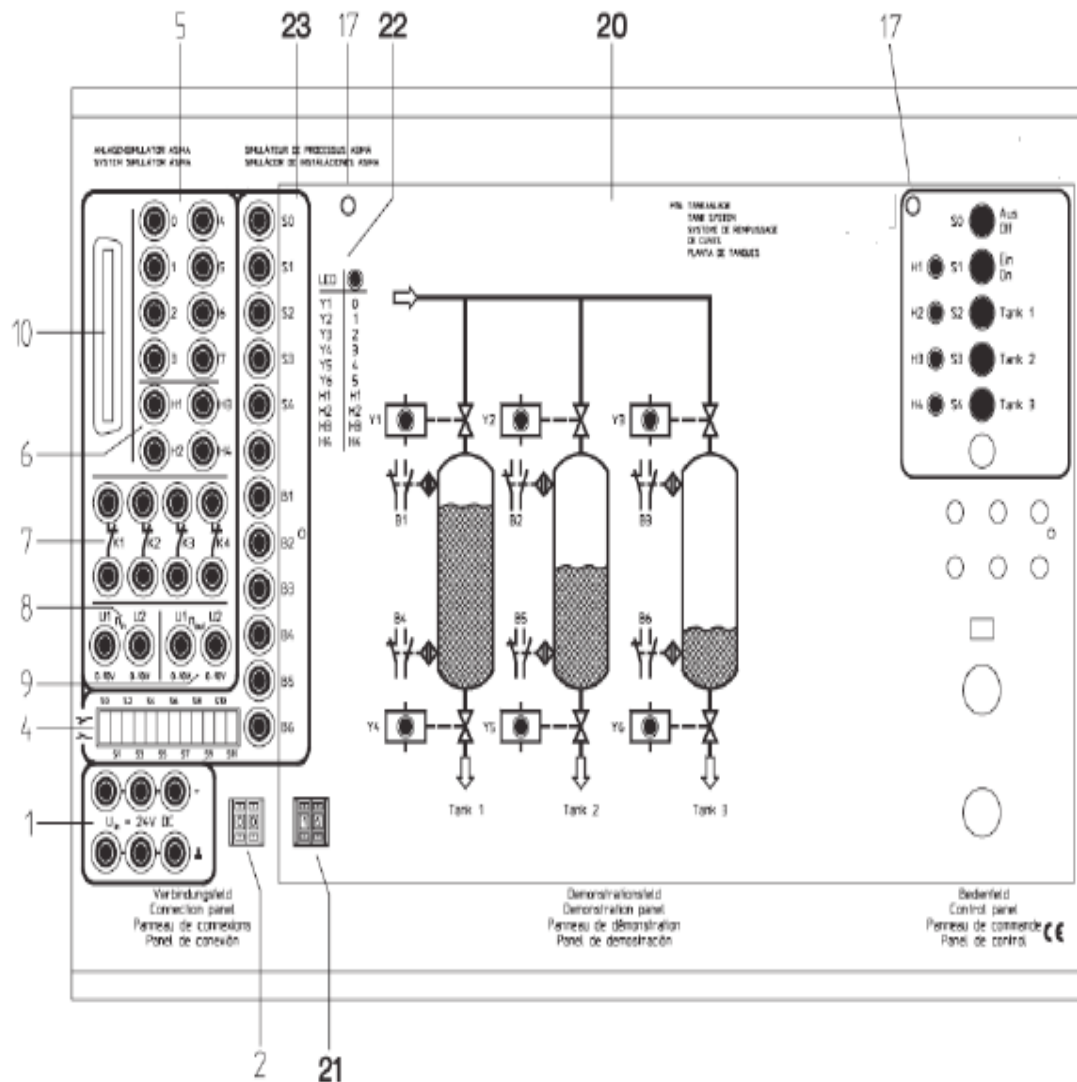


Fig 2.2.2 – Platformă PROSIM – simulare process

În momentul în care se adaugă o mască nouă pe platforma PROSIM, trebuie să se seteze valoarea înscrisă în partea din stânga jos (numărul măștii) utilizând selectorul (2). Acest lucru va asigura maparea intrărilor și ieșirilor în mod corect pentru masca dorită. Dacă acest process nu se realizează, în momentul în care se va încărca un program pe automat se va observa faptul că intrările/ieșirile care sunt folosite pentru a implementa logica programului nu corespund cu masca folosită.

3. Mediul de programare TIA Portal (Step7)

Mediul de dezvoltare TIA Portal (Step 7) este dezvoltat de compania SIEMENS. Acest mediu de dezvoltare permite crearea de aplicații care rulează pe automatele produse de compania SIEMENS folosind limbajul de programare pentru automatele programabile care se bazează pe diagrame LADDER. Pe lângă aplicațiile bazate pe diagrame LADDER se pot dezvolta aplicații bazate pe diagrame de funcții bloc (FBD - limbaj de programare care se bazează simbolurile grafice folosite în cadrul algebrei booleene) și aplicații bazate pe limbaje de programare de nivel înalt (SCL – limbaj structurat). Această lucrare tratează doar dezvoltarea de aplicații care folosesc limbajul de programare bazat pe diagrame LADDER.

În cadrul acestui paragraf se vor prezenta pași care sunt necesari pentru a crea un proiect utilizând mediul de dezvoltare TIA Portal, se vor prezenta etapele necesare pentru setarea corectă a configurației hardware și se vor prezenta cataloagele de componente pe care programatorul le poate folosi pentru crearea de aplicații care să ruleze pe automatele de tip SIEMENS. Automatul pentru care vor fi dezvoltate aplicațiile din cazul acestei lucrări este automatul SIEMENS 1214C.

3.1 Interfața mediului de dezvoltare TIA Portal

În cadrul acestui paragraf se va prezenta interfața mediului de dezvoltare TIA Portal. Se vor prezenta principalele zone ale interfeței utilizator și funcțiile pe care acestea le îndeplinesc. Aplicația TIA Portal permite utilizatorului selectarea modului în care aceasta să funcționeze. Există două moduri de lucru, un mod în care utilizator poate să vizualizeze întregul proiect și un mod care permite vizualizarea doar a informațiilor de bază (configurația hardware a proiectului, modul în care este organizat codul în proiect, vizualizarea proiectelor existente, s.a.m.d). În continuare se vor prezenta principalele zone de pe interfața utilizator pentru a se crea o vedere de ansamblu asupra funcționalităților pe care mediul de dezvoltare TIA Portal le pune la dispoziția dezvoltatorului de aplicații.

Interfața utilizator corespunzătoare modului de lucru care permite vizualizarea informațiilor de bază despre un proiect este prezentat mai jos în cadrul figurii 3.1.1. Se poate observa în cadrul acestei figuri faptul că există patru zone de interes și anume:

- Zona (1) - permite selectarea categoriei de informație ce se dorește a fi vizualizată
- Zona (2) - permite selectarea unei acțiuni din categoria desemnată de zona (1)
- Zona (3) – permite vizualizarea proprietăților corespunzătoare acțiunii selectate din zona (2)
- Zona (4) – permite comutarea modului de vizualizare a proiectului

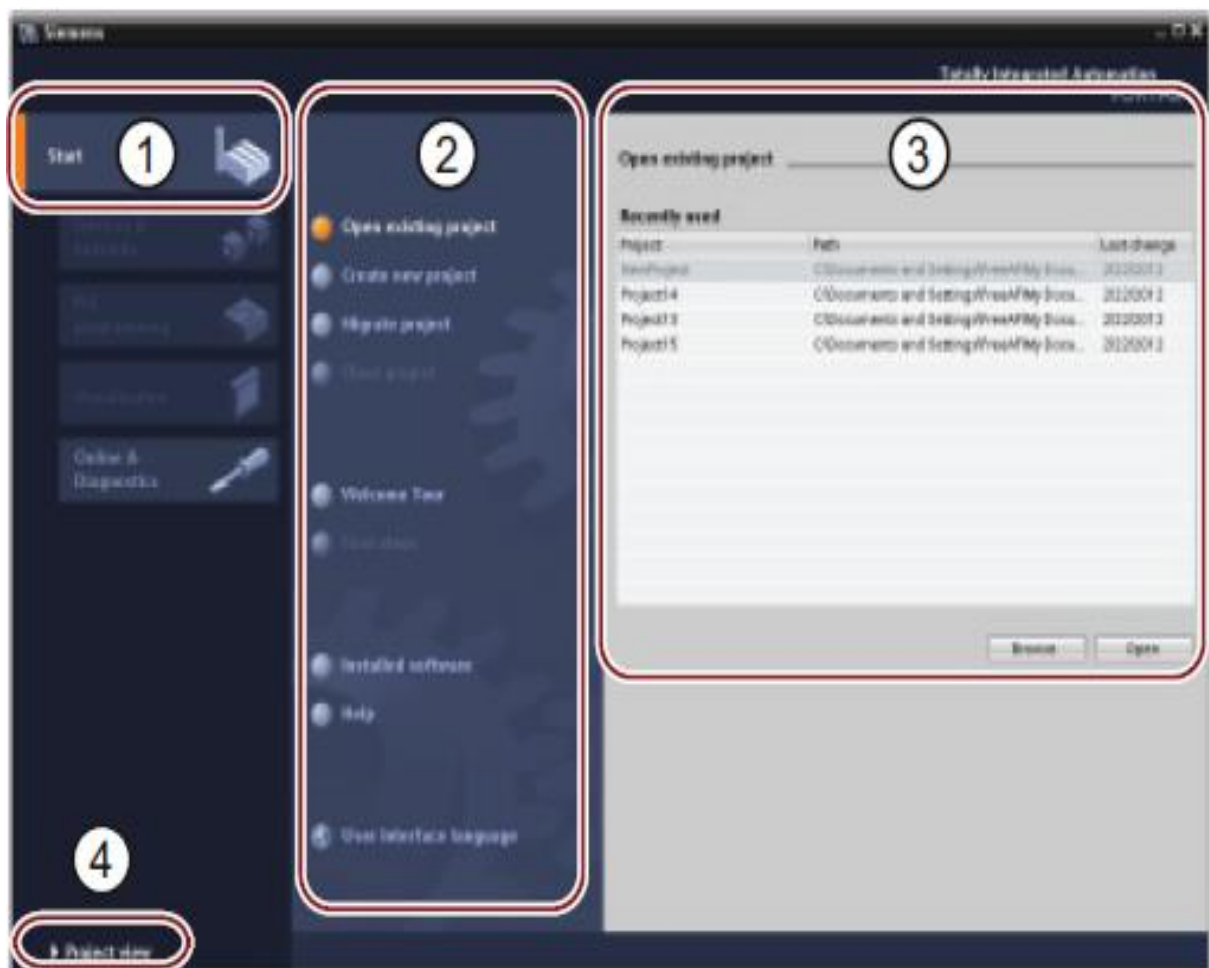


Fig 3.1.1 – TIA Portal – Vizualizare informații de bază

Interfața utilizator corespunzătoare modului de lucru care permite vizualizarea informațiilor extinse despre întregul proiect este prezentat mai jos în cadrul figurii 3.1.2. Se poate observa în cadrul acestei figuri faptul că există șapte zone de interes și anume:

- Zona (1) – este reprezentată de bara de meniuri și de bara de unelte.
- Zona (2) – permite vizualizarea structurii proiectului (configurația hardware, conectarea cu alte dispozitive, codul aplicației, organizarea codului în blocuri, s.a.m.d)
- Zona (3) – zona de lucru care permite scrierea de cod, configurarea hardware-ului, organizarea blocurilor de cod, s.a.m.d
- Zona (4) – catalogul de blocuri folosite pentru realizarea de diagrame LADDER
- Zona (5) – permite vizualizarea proprietăților, vizualizarea conținutului unei variabile, setarea proprietăților configurației hardware, s.a.m.d
- Zona (6) - permite comutarea modului de vizualizare a proiectului
- Zona (7) – permite comutarea între mai multe zone de lucru și afișează toate zonele de lucru active la un moment dat.

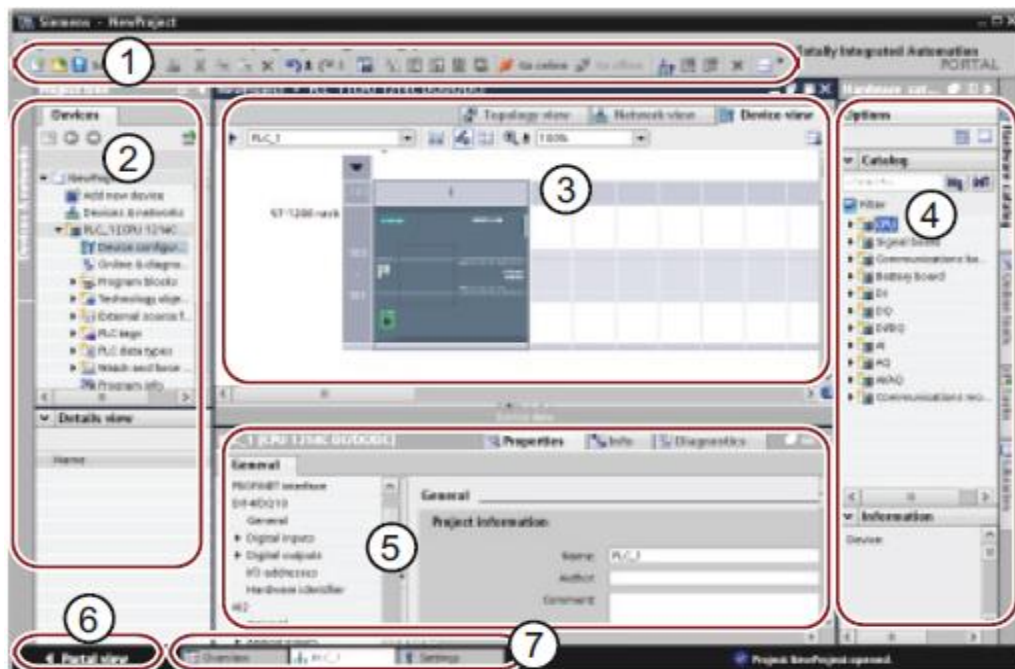


Fig 3.1.2 – TIA Portal – Vizualizare proiect

3.2 Crearea unui proiect

Crearea unui proiect nou în cadrul aplicației TIA Portal necesită executarea a trei pași care urmează a fi descriși în continuare. Pe lângă crearea de noi proiecte se pot vizualiza și proiectele existente sau se pot efectua migrări de proiecte scrise în versiuni mai vechi ale aplicației TIA Portal.

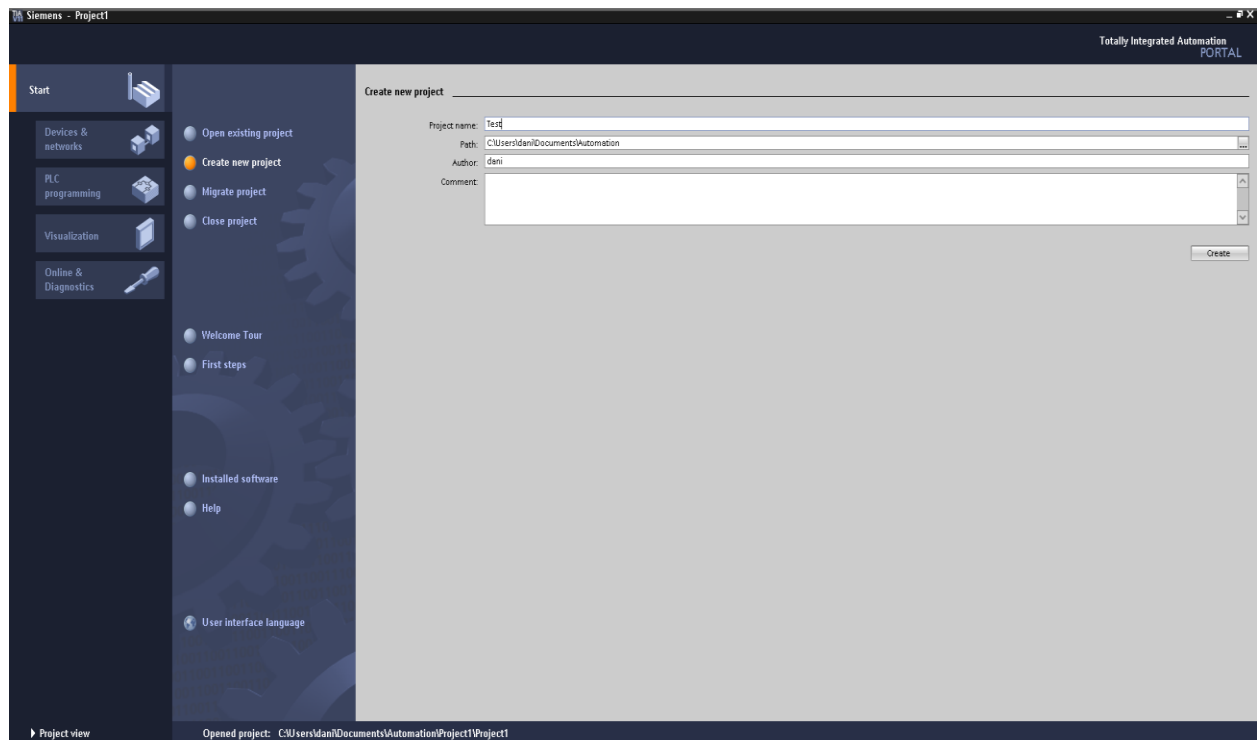


Fig 3.2.1 – Creare proiect nou

Pentru crearea unui proiect nou sunt necesari următorii pași:

- din zona de lucru (1) se selectează categoria “Start”, moment în care în zona corespunzătoare acțiunilor (2) apar disponibile un set de acțiuni referitoare la crearea de noi proiecte și vizualizarea proiectelor existente care pot să fie vizualizate în cadrul figurii 3.2.1.
- se selectează opțiunea “Create new project”, moment în care în zona care permite vizualizarea proprietăților (3) pentru acțiunea curentă apar disponibile câmpurile cu informație ce trebuiesc completate pentru crearea unui proiect nou.

- după completarea câmpurilor cu informația necesară, se apasă butonul “Create” moment în care se va crea un nou proiect.

3.3 Crearea configurației hardware

Selectarea configurație hardware dorită (tipul de unitate centrală de prelucrare a informațiilor, modulele suplimentare de intrare-ieșire sau modulele suplimentare de comunicare) se face de obicei după crearea unui proiect nou. Configurarea hardware se poate modifica ulterior după ce s-a scris codul aplicație, înainte ca aplicația să fie încărcată pe automat. Se menționează faptul că prin modificarea configurație hardware după scrierea codului se înțelege modificarea tipului unității centrale de prelucrare (CPU) cu o altă unitate centrală de prelucrare din aceeași categorie (S7-300, S7-400, S7-1200). De asemenea se poate opta pentru specificarea unei unități centrale de prelucrare a informației generale, detectarea acesteia făcându-se în momentul în care se încarcă aplicația pe automat.

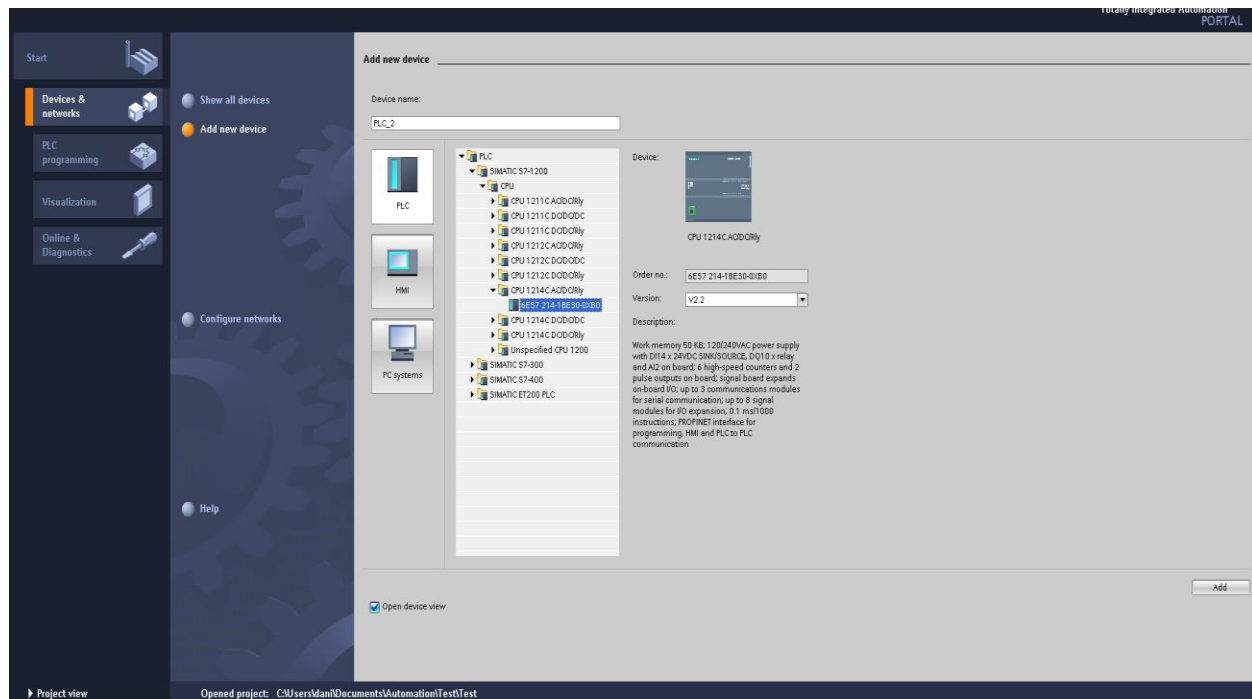


Fig 3.3.1 – Selectare CPU

Adăugarea unui CPU se poate face prin selectarea opțiunii “Add new device”. În momentul în care această opțiune este selectată, în partea dreaptă va apărea o zonă care permite selectarea tipului de CPU care se dorește a fi utilizat (fig 3.3.1). Există patru familii de CPU-uri care pot să fie selectate și anume: S7-300, S7-400, S7-1200 și ET200. În cadrul acestui paragraf se va folosi un CPU din familia S7-1200 și anume S7-1214C. Pentru celelalte tipuri de unități centrale de prelucrare a informație se procedează la fel. După selectarea tipului de CPU dorit, prin apăsarea butonului “Add” se va adăuga CPU-ul selectat în proiect. Structura hardware minimă necesară pentru o aplicație trebuie să conțină un singur CPU, fără alte blocuri. Această structură minimală este prezentată mai jos în cadrul figurii 3.3.2.

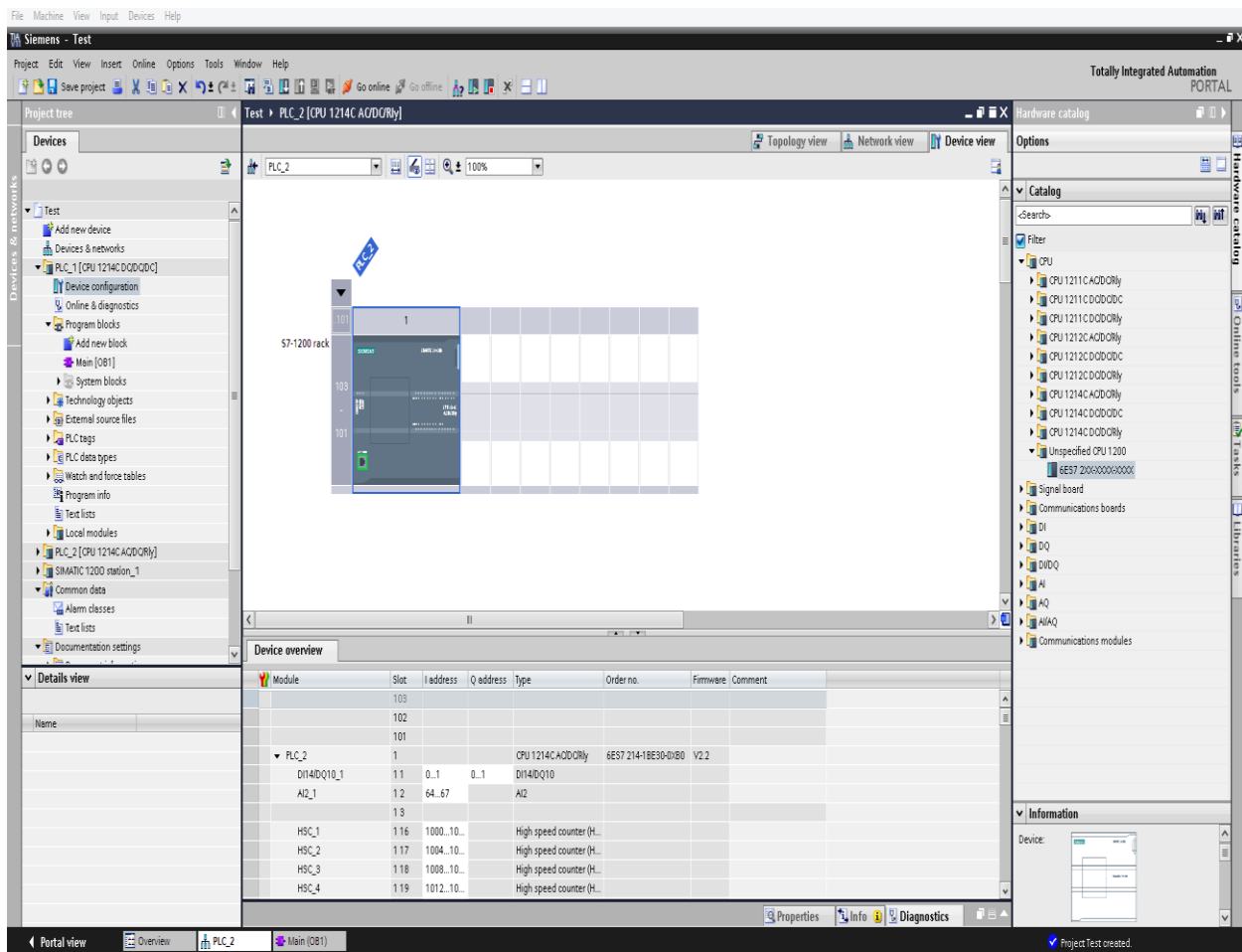


Fig 3.3.2 – Structura hardware minimală a unui proiect

Structura minimală a unui proiect prezentată în cadrul figurii 3.3.2 poate să fie extinsă prin adăugarea de module suplimentare. Aceste module pot să fie folosite pentru utilizarea de

intrări sau ieșiri suplimentare, module folosite pentru comunicarea cu alte dispozitive, respective plăci de semnale cu diferite întrebuințări. Adăugarea de astfel de module suplimentare se face prin identificarea exactă a tipului de modul ce se dorește a fi adugat în cadrul catalogului hardware (catalogul hardware se găsește în partea dreaptă a ecranului și este prezentat mai jos în figura 3.3.3) urmată de o operațiune de tipul **drag and drop** unde sursa este reprezentată de componenta hardware ce se dorește a fi adăugată, iar destinația este reprezentată de unul din sloturile libere de lângă CPU (în figura 3.3.2 se pot observa existența unui număr de 8 astfel de sloturi situate în partea dreaptă a CPU-ului).

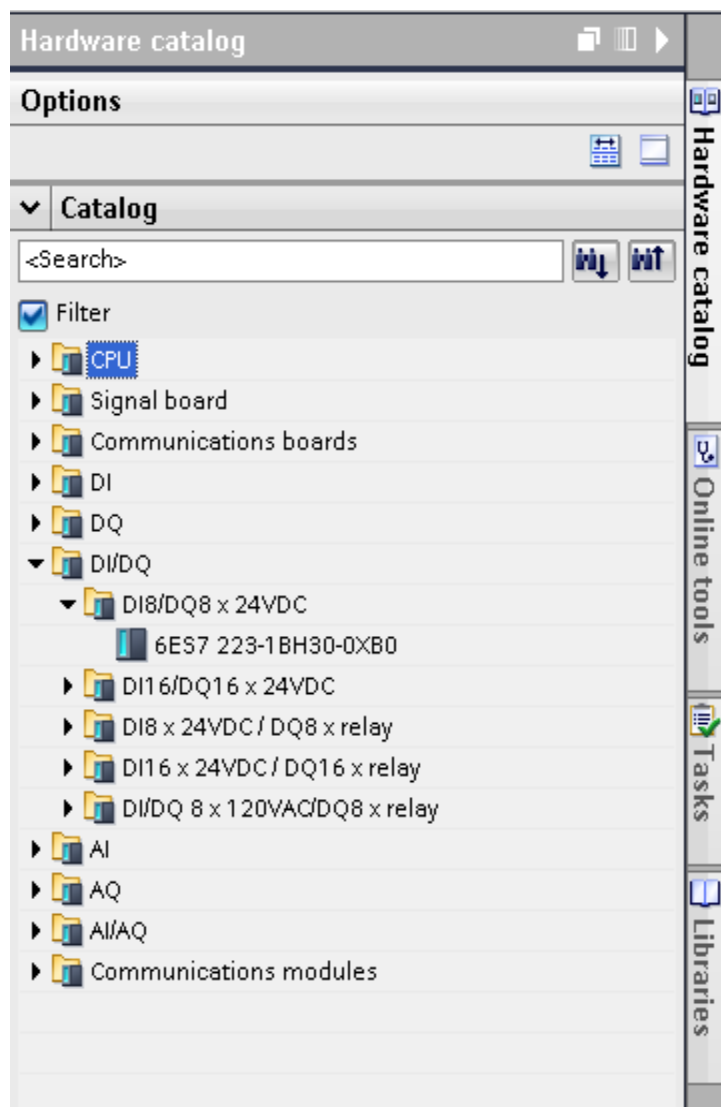


Fig 3.3.3 – Catalog component hardware

Un exemplu de astfel de configurație hardware extinsă alcătuită dintr-un CPU și un modul suplimentar de intrări-ieșiri de tip numerice este prezentat în cadrul figurii 3.3.4. În cadrul acestei figuri se poate observa de asemenea faptul că pentru modulul suplimentar de intrări – ieșiri digitale se pot efectua o serie de configurări. Cea mai importantă dintre acestea este reprezentată de adresa modulului. Adresa modulului este folosită pentru acesarea intrărilor și ieșirilor suplimentare. Această adresă poate să fie schimbată după necesitățile fiecărui proiect în funcție de numărul de astfel de module utilizate. În cazul de față adresa alocată în mod automat este 8 adresă care poate să fie schimbată. După selectarea adresei pentru dispozitivul suplimentar de intrări / ieșiri, adresa setată va fi folosită în cadrul codului aplicației pentru a accesa aceste intrări / ieșiri.

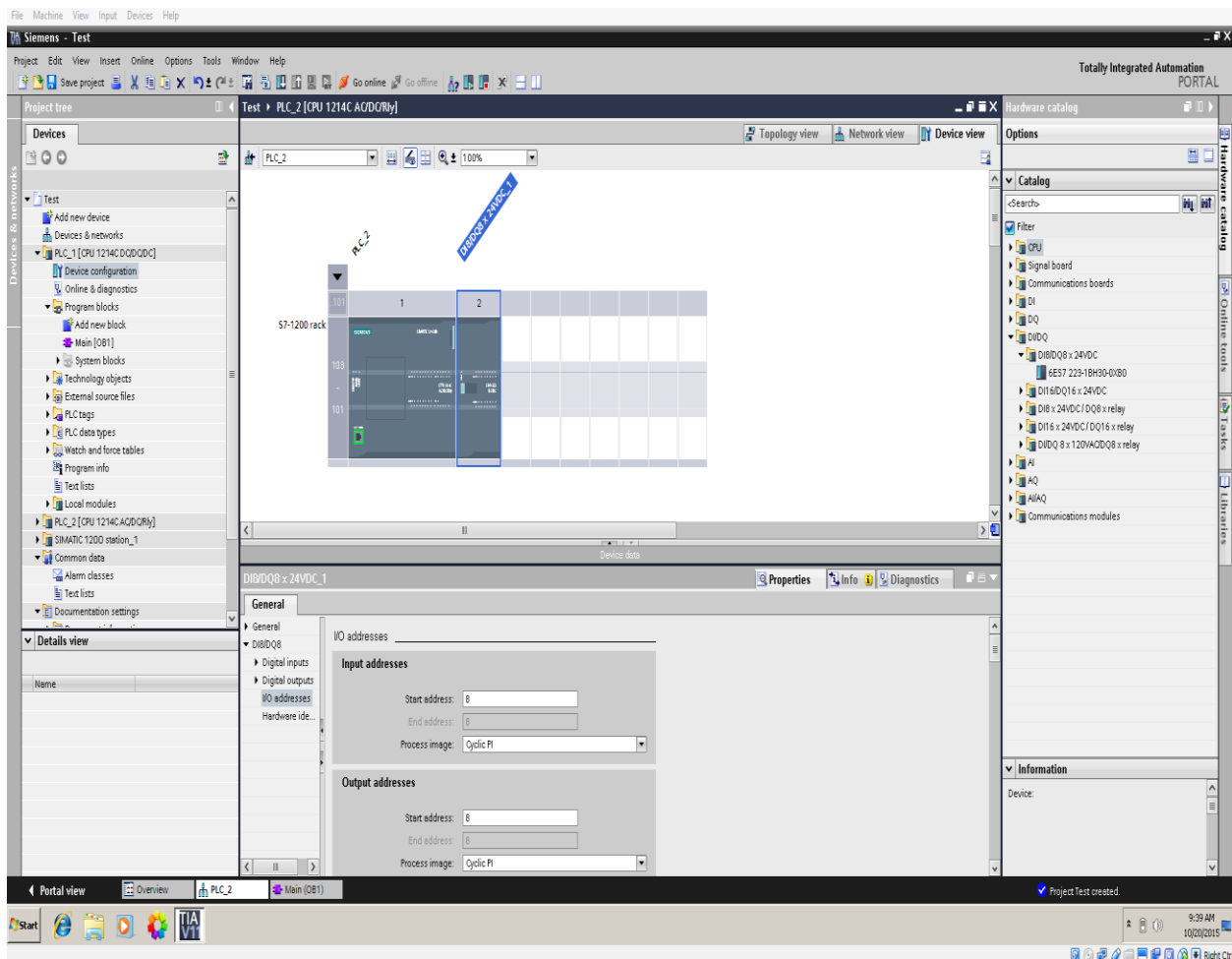


Fig 3.3.4 – Structura hardware extinsă a unui proiect

3.4 Compilarea și încărcarea aplicației

Pentru a rula aplicația scrisă pe un automat este necesară efectuarea a două operațiuni și anume: compilarea codului și încărcarea codului pe automatul propriu zis.

Compilarea codului scris se face prin efectuarea unui click dreapta pe folderul proiectului, urmată de selectarea opțiunii “Compile” (figura 3.4.1) moment în care se va deschide un submeniu cu patru opțiuni. Pentru prima compilare este necesar să se selecteze opțiunea “All” asigurându-ne astfel de faptul că atât software-ul cât și configurația hardware vor fi compilate.

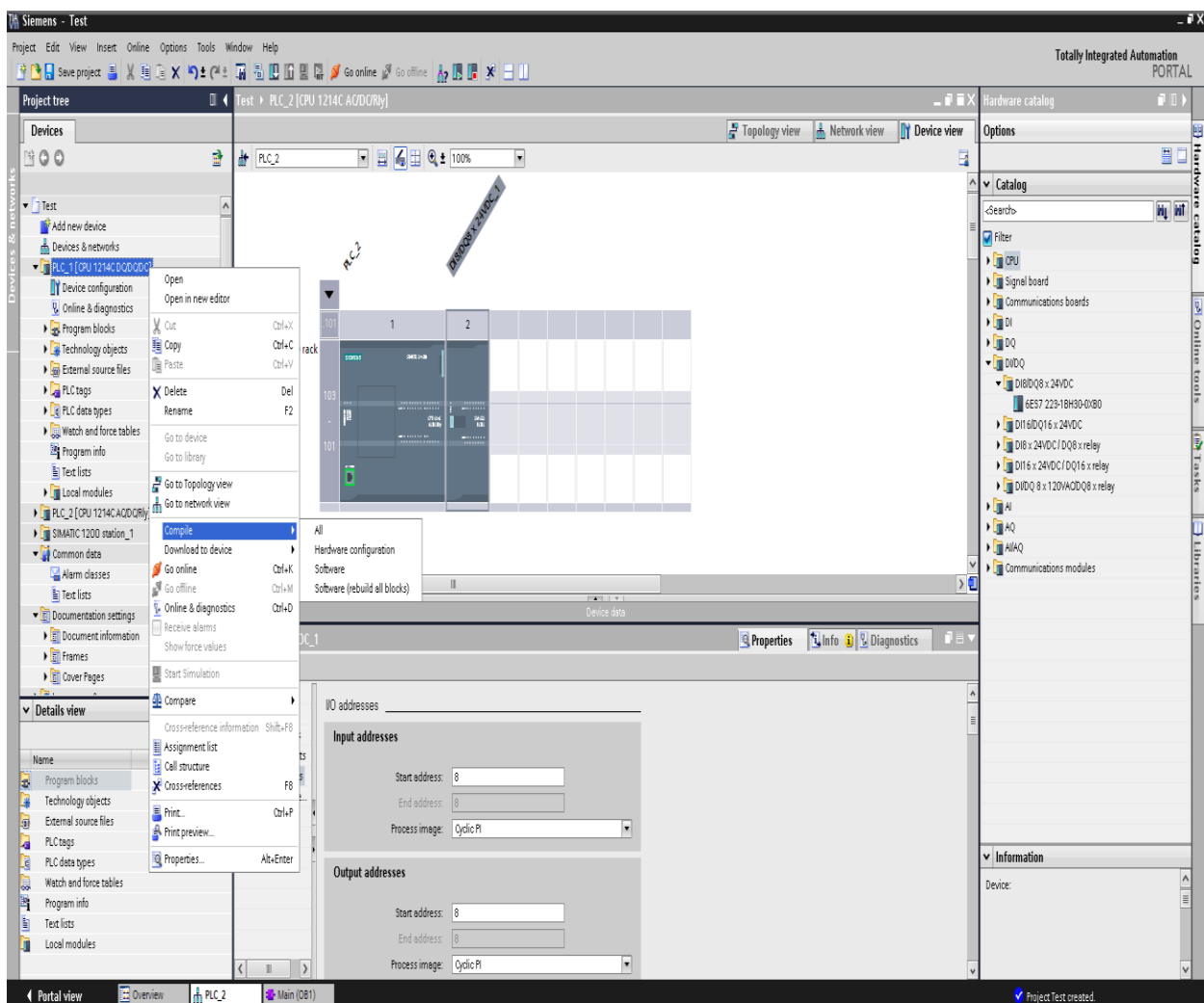


Fig 3.4.1 – Procesul de compilare

Procesul de compilare poate să decurgă fără erori dacă codul este scris corect și dacă configurația hardware a fost corect setată, sau pot să apară erori în cazul în care configurarea hardware nu a fost corect setată sau software-ul scris are erori. Erorile de compilare sunt afișate în cadrul ferestrei “Compile”. Această fereastră prezintă detalii despre fiecare eroare prezentă, ajutând programatorul cu informațiile necesare pentru corectarea erorii. În momentul în care configurația hardware și software-ul compilează fără probleme, în această fereastră nu va fi afișată nicio eroare. Fereastra care prezintă informații despre erorile de compilare este prezentată mai jos în cadrul figurii 3.4.2.

The screenshot displays the Siemens SIMATIC Manager software interface. The left sidebar shows the 'Project tree' with the 'Test' folder expanded, highlighting 'PLC_1 [CPU 1214C DC/DO/DC]'. The main workspace shows a rack diagram with a SIMATIC 1214C module. The bottom panel is divided into 'Device data' and 'Compile' tabs. The 'Compile' tab shows the status 'Compiling completed (errors: 2; warnings: 0)' and a table of errors.

Path	Description	Errors	Warnings	Time
PLC_1		2	0	10:07:54 AM
Hardware configuration		0	0	10:07:54 AM
Program blocks		2	0	10:07:58 AM
IEC_Timer_0_DB (DB1)	Block was successfully compiled.	0	0	10:07:58 AM
Main (OB1)		2	0	10:07:58 AM
Network 1	The operand required at the input or output is missing or has ...	1	0	10:07:58 AM
Network 1	The operand required at the input or output is missing or has ...	1	0	10:07:58 AM
Compiling completed (errors: 2; warnings: 0)		1	0	10:07:59 AM

Fig 3.4.2 – Erori de compilare

Pentru încărcarea codului compilat pe automat trebuie parcurse două etape. O primă etapă constă în verificarea adresei de IP către care se vor trimite configurația hardware respectiv soft-ul compilat. Verificarea adresei de IP se poate face prin vizualizarea proprietăților CPU-ului selectând opțiunea “PROFINET Interface” urmată de opțiunea “Ethernet addresses” (figura 3.4.3) . Odată selectată această opțiune se va putea schimba adresa automatului către care se vor trimite fișierele compilate. Acest lucru este foarte util în momentul în care în rețeaua curentă există conectate mai multe automate, efectuând astfel o diferențiere a automatelor pe baza adresei de IP. După verificarea adresei IP se poate trece la cea de a doua etapă și anume încărcarea fișierelor compilate pe automat.

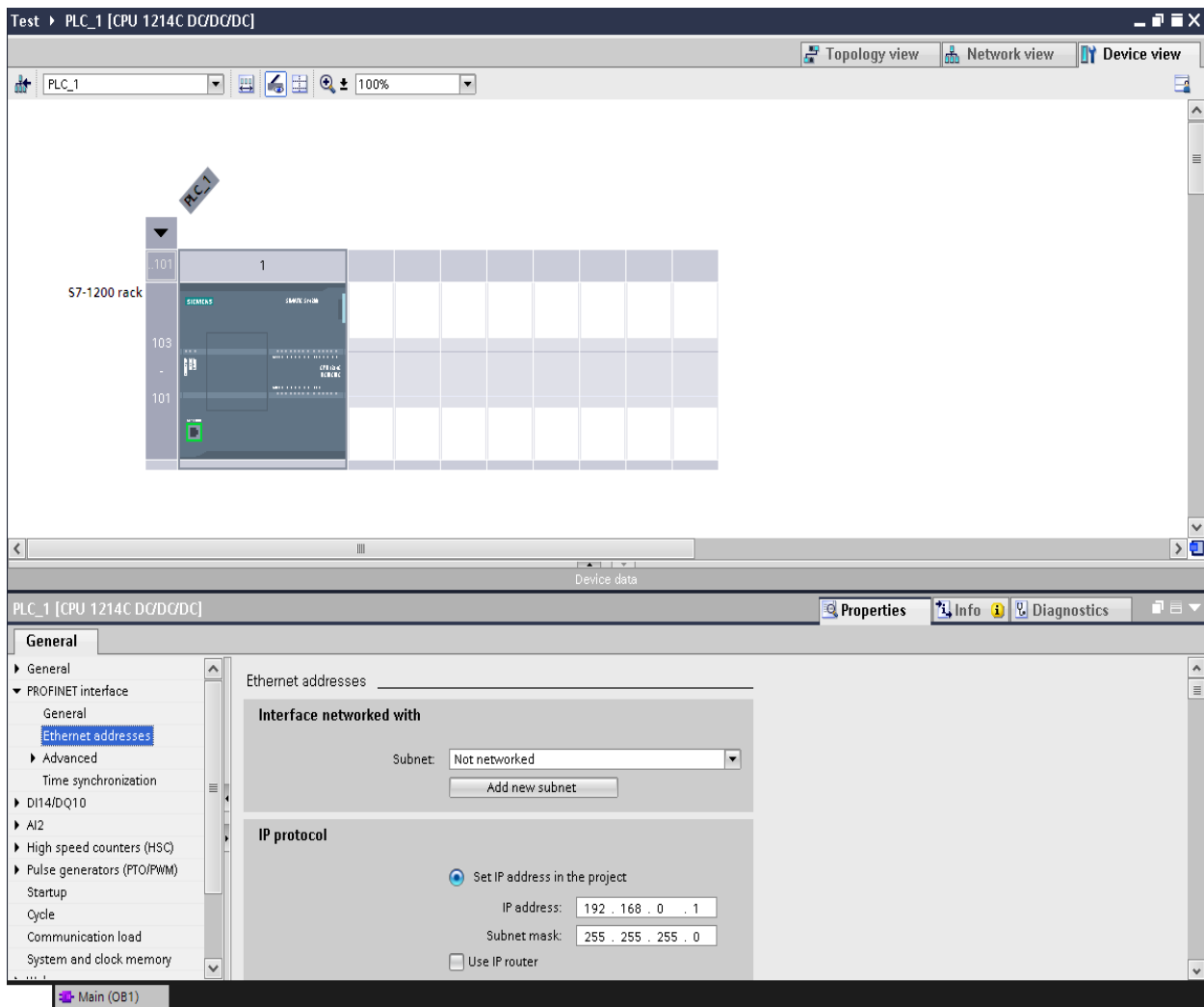


Fig 3.4.3 – Selectare adresă IP

Încărcarea fișierelor compilate pe automat se face prin efectuarea unui click dreapta pe folderul proiectului, urmată de selectarea opțiunii “Download to device” (figura 3.4.4) moment în care se va deschide un submeniu cu patru opțiuni. Pentru încărcarea de fișiere compilate este necesar să se selecteze opțiunea “All” asigurând astfel faptul că atât software-ul cât și configurația hardware vor fi încărcate pe automat. După selectarea opțiunii de încărcare a fișierelor compilate pe automat, vor apărea o serie de mesaje cu caracter informativ referitoare la statusul curent al încărcării codului pe automat.

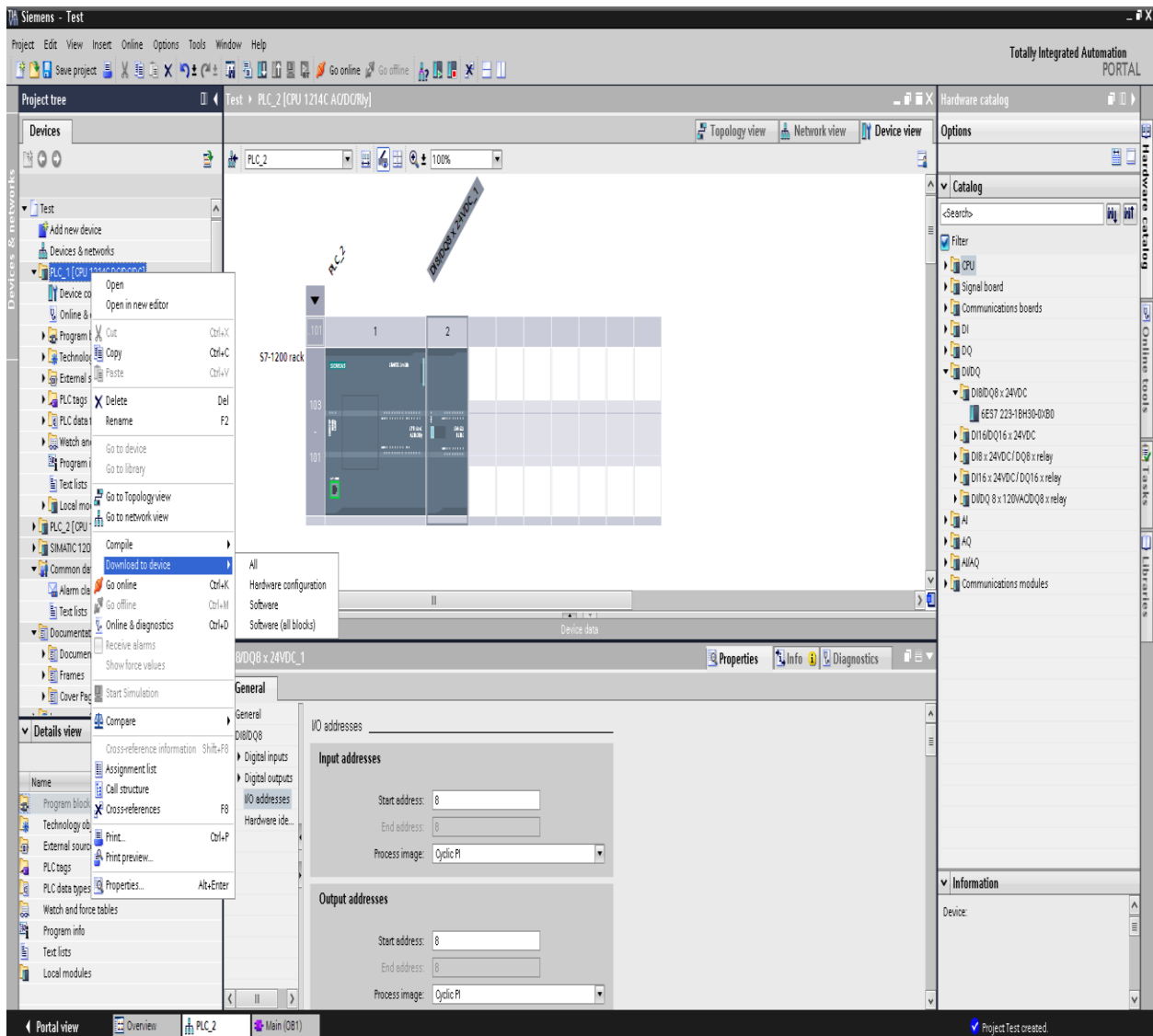


Fig 3.4.4 – Încărcare fișiere compilate pe automat

3.5 Depanarea aplicației

Mediul de dezvoltare TIA Portal oferă facilitatea de depanare a programelor scrise folosind diagramele LADDER. Depanarea aplicațiilor se face prin conectarea mediului de dezvoltare TIA Portal la automatul pe care rulează codul aplicației. Acest mod de depanare este foarte util deoarece se pot vizualiza conținutul zonelor de memorie, valoarea variabilelor de intrare sau valoarea variabilelor de ieșire reușind în acest fel să se găsească cauza pentru care aplicația nu funcționează corespunzător.

Pentru depanarea aplicației și conectarea la automat, trebuie să se apese butonul “Monitoring on/off” din bara de unelte a aplicației. Acest buton este încadrat într-un chenar roșu și poate fi observat în cadrul figurii 3.5.1. În momentul în care se apasă acest buton, mediul de dezvoltare se va conecta la automatul a cărui adresa de IP a fost specificată în momentul în care s-au încărcat fișierele compilate pe automat (specificarea adresei de IP a automatului la care aplicația TIA Portal se conectează în momentul în care se face depanarea se face în fereastra prezentă în figura 3.4.3).

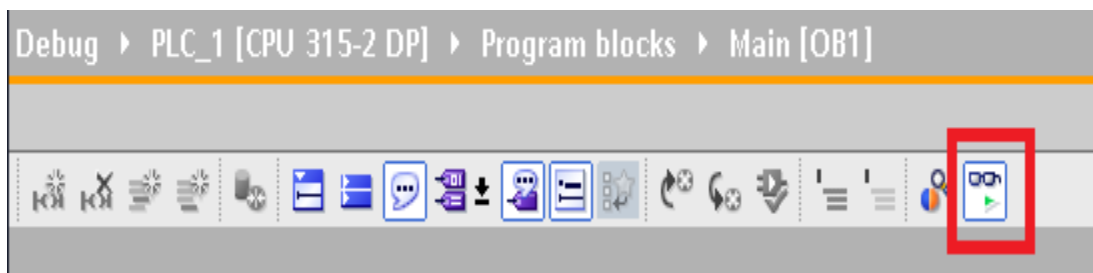


Fig 3.5.1 – Buton pentru utilizarea facilităților de depanare

Pentru a se depana programul, este obligatoriu ca să existe aceeași variantă de cod atât pe automat cât și în mediul de dezvoltare TIA Portal. Cele mai frecvente probleme apar deoarece codul încărcat pe automat este o versiune mai veche și diferă de codul existent în mediul de dezvoltare TIA Portal. Pentru rezolvarea acestei probleme, se recompilează codul existent în mediul de dezvoltare TIA Portal și se reîncarcă fișierele compilate pe automat. În momentul în care cele două versiuni ale codului sunt aceleași, conectarea mediului de dezvoltare la automat se poate face cu succes, fapt indicat prin culoarea portocalie a barei de titlu corespunzătoare

spațiului de lucru și prin prezeța în partea dreaptă a ecranului a trei butoane care sunt utilizate pentru specificarea modului de operare al unității centrale de procesare a informației (fig. 3.5.2).

Modul de lucru al unității centrale de prelucrare a informației poate să fie impus în mometul în care se depanează programul prin apăsarea unuia din cele trei butoane din partea dreaptă a ecranului, butoane care sunt ilustrate în cadrul figurii 3.5.2.

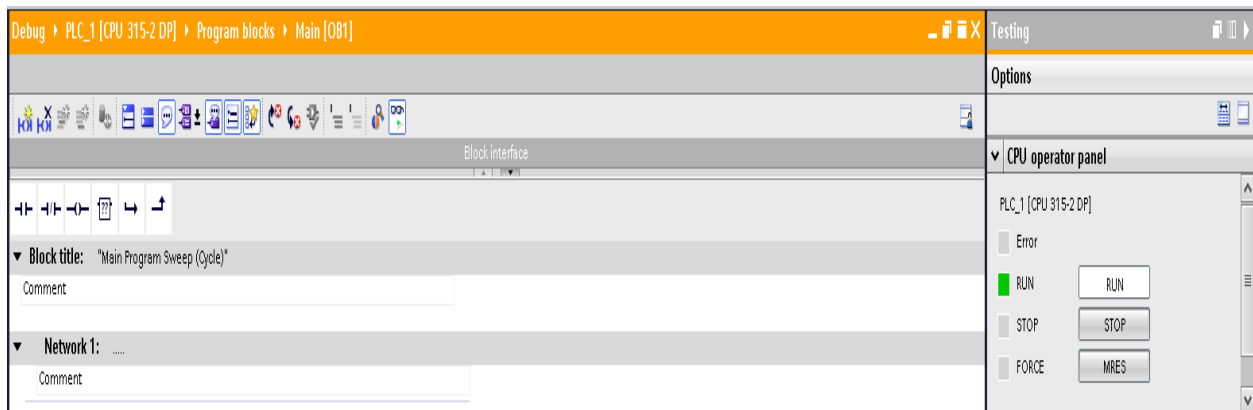


Fig 3.5.2 – Mod de lucru online

Depanarea programului se face inspectând pe rând toate rețelele aplicației. Pentru fiecare rețea se poate observa modul în care blocurile componente se comportă, modul în care semnalele (ieșirile) anumitor blocuri evoluează în timp și modul în care anumite condiții logice sunt evaluate. Spre exemplu se consideră o rețea simplă cu două elemente bloc simple. Primul element bloc (un element de tip contact) se consideră a fi condiția care se dorește a fi îndeplinită, iar cel de al doilea element bloc (un element de tip bobina) se consideră a fi rezultatul evaluării condiției precedente. În figurile 3.5.3 și 3.5.4 este prezentă în partea stângă condiția evaluată, iar în partea dreaptă este prezent rezultatul evaluării condiției.

În prima figură (3.5.3) se observă existența unei linii albastre punctate, lucru ce semnifică faptul că fluxul informațional nu ajunge în capătul drept al rețelei, condiția reprezentată de contactul din partea stângă este evaluată iar fluxul de informație este oprit.



Fig 3.5.3 – Depanarea programului

În a doua figură (3.5.4) se observă dispariția liniei albastre punctate, lucru ce semnifică faptul că fluxul informațional ajunge în capătul drept al rețelei, condiția reprezentată de contactul din partea stângă este evaluată iar fluxul de informație este lăsat să treacă.

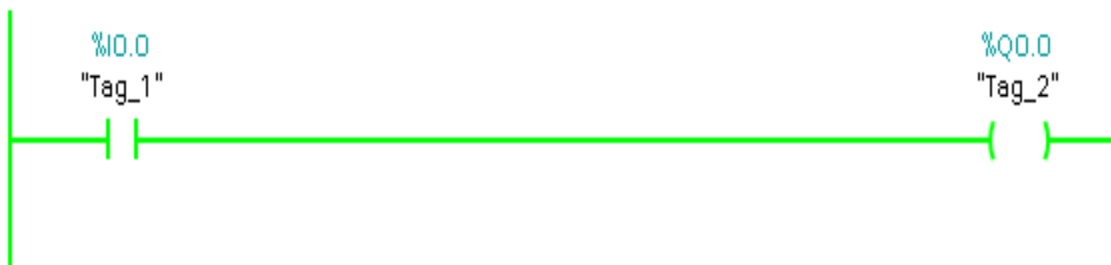


Fig 3.5.4 – Depanarea programului

Utilizând logica prezentată în exemplul de mai sus, se poate verifica modul în care fiecare din rețelele existente sunt evaluate identificând locul în care apare o problemă ce cauzează funcționarea defectuasă a aplicației.

4. Limbajul de programare LADDER

În cadrul acestui capitol se vor prezenta noțiuni introductive referitoare la programarea automatelor secvențiale utilizând diagrame LADDER. Se vor prezenta anumite concepte împreună cu exemple practice care ajută la înțelegerea lor. De asemenea se vor prezenta și o serie de elemente bloc folosite în mediul de dezvoltare TIA Portal pentru a crea aplicații care rulează pe automate programabile și care au la bază limbajul de programare LADDER.

4.1 Introducere

În cadrul acestui capitol se va face o prezentare a limbajului de programare LADDER, limbaj folosit pentru programarea automatelor secvențiale. Se vor prezenta particularitățile acestui limbaj de programare și se vor prezenta exemple practice care ajută la înțelegerea acestui limbaj de programare. Se vor prezenta modul în care este structurată memoria unui automat, tipurile de zone de memorie existente și modul în care acestea se utilizează.

Limbajele de programare care sunt folosite pentru programarea automatelor sunt diverse. Există limbaje care permit programarea automatelor la nivel de bit, limbaje care permit programarea automatelor folosind instrucțiuni, limbaje care permit programarea automatelor folosind cod asemănător codului scris în limbajul de programare C (sau Visual Basic) , respectiv limbaje care permit programarea automatelor utilizând diagrame (fiecare dintre diagrame este alcătuită dintr-o succesiune de blocuri care rezolvă o anumită problemă). În cadrul acestei curs se va trata problematica programării automatelor utilizând diagrame LADDER. Acest limbaj de programare care folosește diagrame a fost introdus deoarece se dorea realizarea de aplicații pentru automatele programabile fără a fi nevoie de cunoștințe complexe de programare. Diagramele LADDER folosesc anumite blocuri (simboluri) pentru anumite elemente fizice (intrări, ieșiri, timere, countere). Folosind aceste simboluri, se pot crea anumite legături, se pot evalua anumite condiții și se pot lua anumite decizii bazate pe informațiile furnizate de blocurile componente. Fiecărui simbol din cadrul unei diagrame LADDER trebuie să i se asocieze o semnificație (adică un bloc de date, un registru de date sau un bit).

Un program scris în limbajul de programare LADDER se prezintă sub forma unor succesiuni de rețele. Fiecare rețea conține anumite blocuri care prelucreează informația în vederea generării unei comenzi. O rețea simplă, care nu conține niciun bloc de prelucrare a informației este prezentată mai jos în cadrul figurii 4.1.1. Plecând de la o astfel de rețea, se pot adăuga diferite blocuri care implementează anumite funcționalități ale programului. Un program care rulează pe automat este alcătuit dintr-o serie de rețele care rulează una după alta într-o buclă continuă. Un ciclu de rulare a programului se realizează începând cu prima rețea și se încheie cu ultima rețea, moment în care execuția programului se va relua.

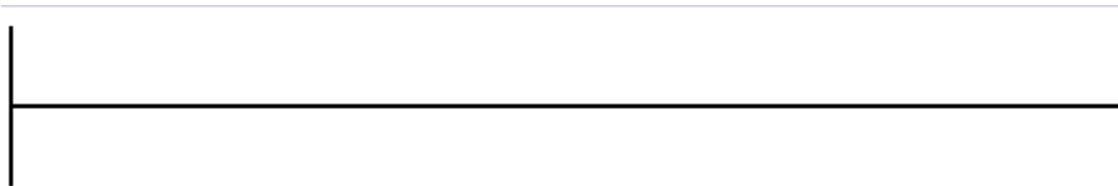


Fig 4.1.1 – Rețea LADDER

4.2 Intrări, ieșiri și zone de memorie

Fiecărui bloc dintr-o aplicație scrisă folosind limbajul LADDER i se poate asocia un anumit tip de variabilă. Aceste variabile pot să fie de trei tipuri și anume:

- variabile de intrare (accesate în program folosind I)
- variabile de ieșire (accesate în program folosind Q)
- variabile de memorie (accesate în program folosind M)

Variabilele de intrare sunt folosite pentru citirea de date de la senzorii conectați la automat. Variabilele de intrare reprezintă din punct de vedere fizic valoarea unui semnal pe care automatul îl primește de la un senzor care este conectat la acesta. Variabilele de intrare pot să fie de tip numeric (adică un semnal logic cu valoare 1 sau 0 logic) sau pot să fie de tip analogic, sub forma unui semnal care poate lua valori într-un anumit interval.

Variabilele de memorie sunt folosite pentru efectuarea de operații intermediare. De asemenea variabilele de memorie sunt folosite și în asociere cu elementele bloc care se folosesc

în cadrul limbajului de programare LADDER (timere, countere, etc). Automatul dispune de două tipuri de zone de memorie și anume: zone de memorie care își păstrează valoarea atunci când automatul este scos de sub tensiune, respectiv zone de memorie care nu își păstrează valoarea atunci când automatul este scos de sub tensiune. În funcție de cerințele de implementare ale aplicației care se dorește implementată, se poate folosi un tip din cele două zone de memorie sau chiar amândouă tipurile combinate.

Variabilele de ieșire sunt folosite pentru trimiterea de comenzi către **proces**. Prin comandă se înțelege un semnal numeric (un semnal logic care poate avea valoarea 1 sau 0) sau un semnal analogic. O variabilă de ieșire conectează practic automatul cu un element de execuție (o pompă, un motor, etc) care poate fi pornit sau oprit prin intermediul semnalului generat de către variabila de ieșire.

Toate cele trei tipuri de variabile și anume variabile de intrare, variabile de ieșire și variabile de memorie sunt reprezentate fizic prin intermediul unor regiștrii pe 8 biți. Intrările, ieșirile și zonele de memorie ale unui automat pot fi imaginate sub forma unei table în cadrul căreia fiecare rând este reprezentat de un registru de 8 biți exact cum este prezentat în cadrul figurii 4.2.1. Această tabelă este împărțită în trei regiuni și anume: o regiune dedicată variabilelor de intrare, o secțiune dedicate variabilelor de ieșire și o secțiune dedicate variabilelor de memorie.

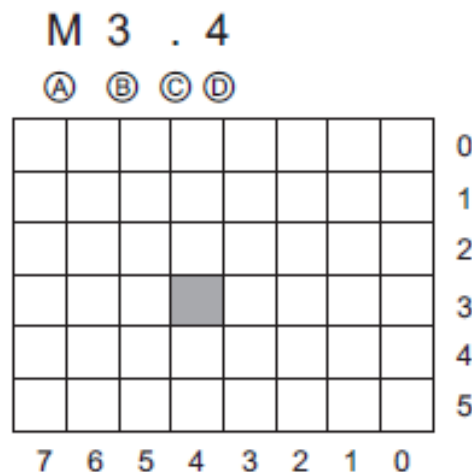


Fig 4.2.1 – Modul de structurare a memoriei

Accesarea variabilelor de memorie, variabilelor de intrare sau de ieșire se poate face în mai multe moduri. O primă variantă de accesare a unei variabile de intrare, ieșire sau de memorie este accesarea la nivel de bit (**TX.Y**) care se face prin specificarea tipului variabilei T (poate să fie I,M sau Q) urmată de numărul registrului (X) respectiv numărul bitului (Y) care se dorește a fi accesat.

Pe lângă accesarea la nivel de bit, variabilele de intrare, ieșire sau memorie pot să fie accesate la nivel de registru (byte sau 8 biți), la nivel de word (16 biți) sau la nivel de double word (32 de biți). Pentru a înțelege mai bine diferența dintre accesarea la nivel de bit, byte word sau double word ilustrează legătura dintre acestea în cadrul figurii 4.2.2. În cadrul aceste figuri se poate observa faptul că un registru (byte) este alcătuit din 8 biți, un word sau registru pe 16 biți este alcătuit din 2 byte și un double word sau registru pe 32 de biți este alcătuit din 2 word-uri sau 4 byte.

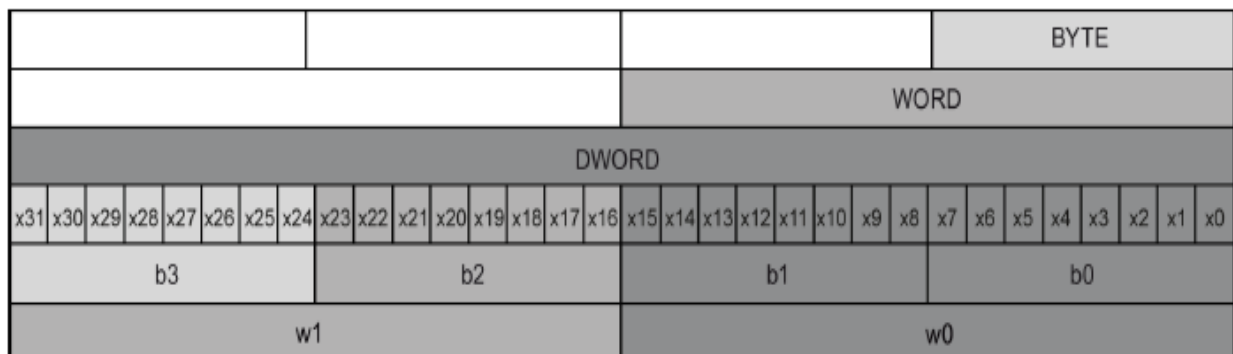


Fig 4.2.2– Legătura dintre bit, byte, word și double word

Accesarea la nivel de byte sau registru al unei variabile de intrare, ieșire sau memorie se face după următorul format (**TBNr**) unde T este tipul registrului care se dorește a fi accesat (intrare, ieșire sau memorie), B semnifică faptul că este vorba despre o accesare la nivel de byte (sau registru) iar Nr reprezintă numărul registrului care se dorește a fi accesat.

Accesarea la nivel de word (accesarea la nivel de word poate fi văzută ca și accesarea unui registru pe 16 biți) al unei variabile de intrare, ieșire sau memorie se face după următorul format (**TWNr**) unde T este tipul registrului care se dorește a fi accesat (intrare, ieșire sau memorie), W

semnifică faptul că este vorba despre o accesare la nivel de word (sau registru pe 16 biți) iar Nr reprezintă numărul registrului pe 16 biți care se dorește a fi accesat.

Accesarea la nivel de double word (accesarea la nivel de word poate fi văzută ca și accesarea unui registru pe 32 biți) al unei variabile de intrare, ieșire sau memorie se face după următorul format (**TDNr**) unde T este tipul registrului care se dorește a fi accesat (intrare, ieșire sau memorie), D semnifică faptul că este vorba despre o accesare la nivel de double word (sau registru pe 32 biți) iar Nr reprezintă numărul registrului pe 32 biți care se dorește a fi accesat.

În funcție de dimensiunea registrului folosit în cadrul aplicației (8, 16 sau 32 de biți) se poate stoca în cadrul acestuia un anumit tip de dată. Tipurile de dată care se pot stoca în registrii de 8, 16 sau 32 de biți sunt prezentați mai jos în cadrul figurii 4.2.3.

Data type	Bit size	Number type	Number range	Constant examples	Address examples
Bool	1	Boolean	FALSE or TRUE	TRUE, 1,	I1.0 Q0.1 M50.7 DB1.DBX2.3 Tag_name
		Binary	0 or 1	0, 2#0	
		Octal	8#0 or 8#1	8#1	
		Hexadecimal	16#0 or 16#1	16#1	
Byte	8	Binary	2#0 to 2#11111111	2#00001111	IB2 MB10 DB1.DBB4 Tag_name
		Unsigned integer	0 to 255	15	
		Octal	8#0 to 8#377	8#17	
		Hexadecimal	B#16#0 to B#16#FF	B#16#F, 16#F	
Word	16	Binary	2#0 to 2#1111111111111111	2#1111000011110000	MW10 DB1.DBW2 Tag_name
		Unsigned integer	0 to 65535	61680	
		Octal	8#0 to 8#177777	8#170360	
		Hexadecimal	W#16#0 to W#16#FFFF, 16#0 to 16#FFFF	W#16#F0F0, 16#F0F0	
DWord	32	Binary	2#0 to 2#11111111111111111111111111111111	2#11110000111111110001111	MD10 DB1.DBD8 Tag_name
		Unsigned integer	0 to 4294967295	15793935	
		Octal	8#0 to 8#3777777777	8#74177417	
		Hexadecimal	DW#16#0000_0000 to DW#16#FFFF_FFFF, 16#0000_0000 to 16#FFFF_FFFF	DW#16#F0FF0F, 16#F0FF0F	

Fig 4.2.3– Tipuri de date care pot fi stocate în regiștrii automatului

4.3 Operarea la nivel de bit. Contacte, bobine și bistabile

Cele mai simple operații care pot să fie implementate folosind diagramele LADDER sunt operațiile logice la nivel de bit. Pentru implementarea acestora se folosesc în principal ca și blocuri contactele și bobinele în asociere cu un bit al unui registru de intrare, ieșire sau memorie.

Aceste două tipuri de blocuri sunt cele mai simple blocuri care se pot folosi atunci când se scrie un program utilizând limbajul de programare LADDER și există mai multe variante ale acestor blocuri care îndeplinesc funcționalități diferite.

4.3.1 Contacte și bobine

Blocurile de tip contact și bobină sunt prezentate mai jos în cadrul figurilor 4.3.1.1a și 4.3.1.1b. Contactele sunt simbolizate sub forma a două linii paralele și pot să fie de două feluri: contact normal închis, respectiv contact normal deschis. Din punct de vedere al semnificație fizice, un contact normal deschis se poate privi ca și un întrerupător care lasă curentul să treacă prin el atunci când este apăsat, iar când acesta nu este apăsat curentul nu trece prin întrerupător sau ca și un element care permite sau nu permite trecerea unui semnal logic în funcție de o anumită condiție (Fig 4.3.1.1a). Un contact normal închis se comportă într-o manieră opusă unui contact normal deschis și anume acesta lasă curentul să treacă prin el când nu este apăsat, respectiv oprește trecerea curentului când este apăsat (Fig 4.3.1.1b). Rețeaua din Fig 4.3.1.1a poate să fie asociată din punct de vedere fizic cu aprinderea unui led atunci când se apasă un buton.

Blocurile de tip contact se folosesc pentru a crea o asociere cu o variabilă de intrare sau cu o variabilă de memorie. Asocierea dintre un bloc contact și o variabilă de intrare sau de memorie se realizează la nivel de bit și nu la nivel de registru. Acesta asociere se poate crea prin specificarea bitului cu care se dorește a fi asociat contactul deasupra contactului în cauză.

Blocurile de tip bobină sunt folosite pentru a crea o asociere cu o variabilă de ieșire, adică cu o comandă către procesul condus. Blocurile de tip bobină pot să fie și ele de două feluri normal închise și normal deschise la fel ca și contactele. Din punct de vedere al asocierii care se poate face între o bobină și un registru, se menționează faptul că bobinele se asociază în general cu variabile de memorie și variabile de ieșire.

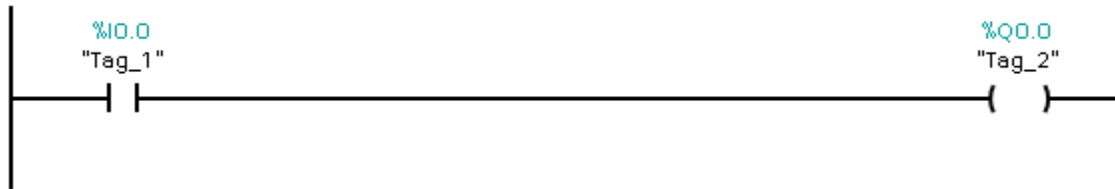


Fig 4.3.1.1a – Contact normal deschis și bobină

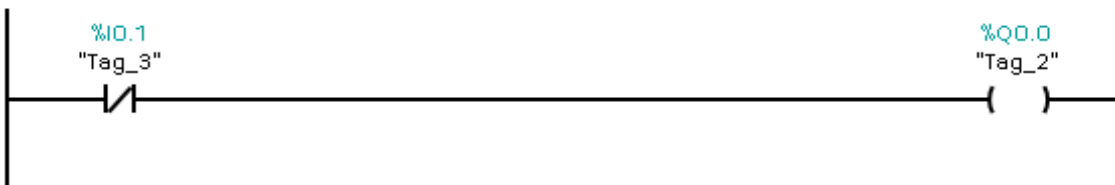


Fig 4.3.1.1b – Contact normal închis și bobină

Blocul de tip negație prezentat în cadrul figurii Fig 4.3.1.2 realizează negarea semnalului. Un bloc de tip contact normal deschis urmat de un bloc de negație este echivalent cu un contact normal închis, adică rețeaua din Fig 4.2.2 este echivalentă cu rețeaua din Fig 4.2.1b.

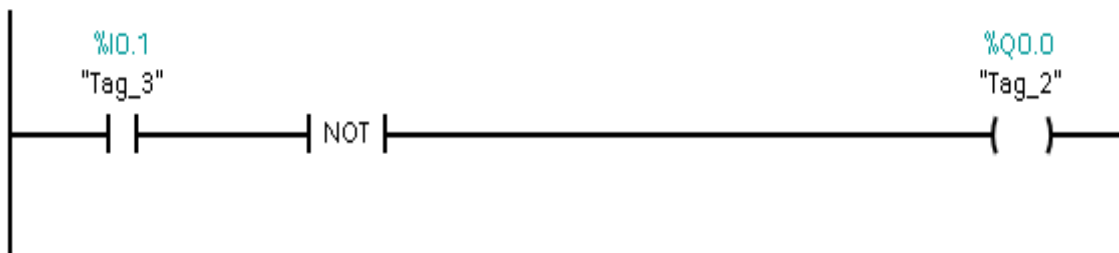


Fig 4.3.1.2 – Contact normal închis

Folosind blocuri de tip contact normal închis, contact normal deschis și bobine se pot implementa și funcții logice complexe. Pentru a exemplifica acest lucru se consideră următoarea funcție logică: $Q0.0 = I0.0 * \overline{I0.1} + I0.2$. Pentru implementarea acestei funcții se pot folosi două variante. Prima variantă este reprezentată de utilizarea a două contacte normal deschise, a unui contact normal închis și a unei bobine, iar cea de a doua variantă de implementare se poate realiza prin utilizarea a trei contacte normale deschise, a unui bloc de negare și a unei bobine. Prima variantă de implementare a funcției logice considerate mai sus este prezentată în figura Fig 4.3.1.3. Pe baza acestui exemplu se deduce faptul că orice funcție logică (pornind de la funcții

logice simple cum ar fi SAU, SI, XOR și countinuând cu funcții logice complexe) care are o anumită expresie se poate implemeta folosind contacte și bobine.

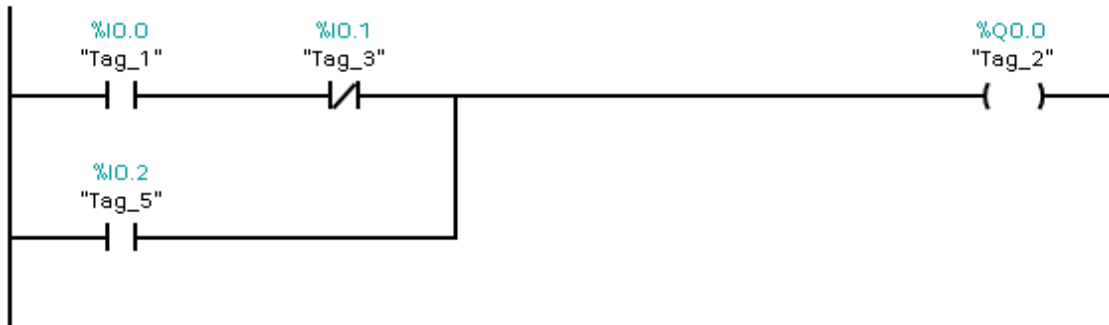


Fig 4.3.1.3 – Funcție logică

Pentru o rețea alcătuită din elemente bloc, se pot asocia pentru fiecare din elementele bloc componente câte un tag. Un tag (o etichetă) reprezintă un nume sugestiv pentru un anumit bloc, nume care are un rol în identificarea funcționalității pe care elementul respectiv o îndeplinește. În cadrul figurii 4.3.1.4 este prezentat un astfel de exemplu unde există o implementare a funcției SAU logic. Intrările I0.0 i s-a asociat tagul "IntrareA", intrării I0.1 i s-a asociat tagul "IntrareB", iar rezultatului funcției logice asociat cu ieșirea Q0.0 i s-a asociat tagul "Rezultat_A_SAU_B". Asocierea de taguri este foarte utilă pentru aplicațiile de mari dimensiuni deoarece procesul de depanare, mentenanța codului sau dezvoltarea ulterioară a aplicației devine mult mai ușor.

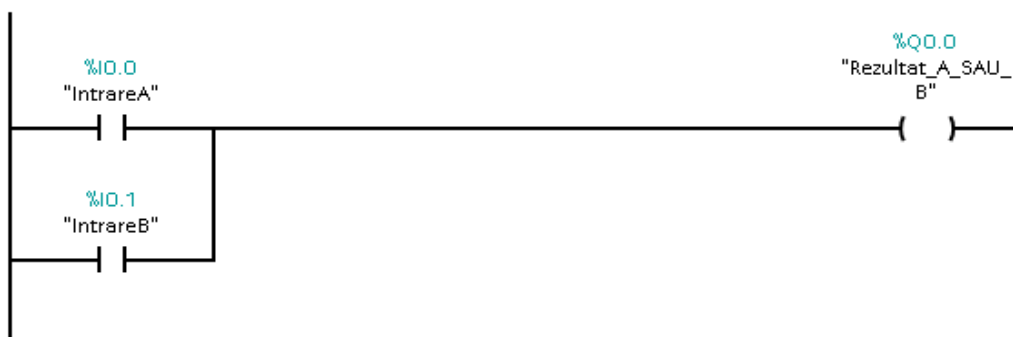


Fig 4.3.1.4 – Utilizarea de taguri

Din punct de vedere al modului în care fluxul de informație circulă într-o rețea, se menționează faptul că acesta este unidirecțional. Nu se pot crea rețele care să permită circulația fluxului de informație într-o manieră bidirecțională. Un astfel de rețea este prezentată mai jos în cadrul figurii 4.3.1.5. Circulația fluxului de date este permisă într-o singură direcție deoarece în această manieră se evită posibilitățile prin care o anumită condiție logică ar putea să nu fie executată. Unele medii de dezvoltare tratează această problemă.

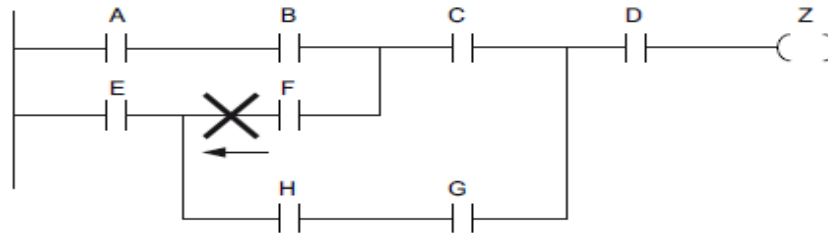


Fig 4.3.1.5 – Rețea care permite trecerea fluxului de date în sens invers

O altă restricție care se impune în momentul în care se crează o rețea este aceea că o rețea nu poate să conțină un scurtcircuit. Un scurt circuit este practic o conexiune în paralel a unei rețele care permite ca anumite condiții logice să nu fie verificate. Un exemplu de scurt circuit într-o rețea LADDER este prezentat mai jos în cadrul figurii 4.3.1.6. În cadrul acestei figuri se poate observa faptul că evaluarea condiției logice asociate cu cel de al doilea contact (contactul B) poate să nu fie efectuată deoarece fluxul de informație va trece prin conexiunea paralelă cu contactul B. În funcție de mediul de programare folosit, această problemă poate să fie tratată în mod automat.

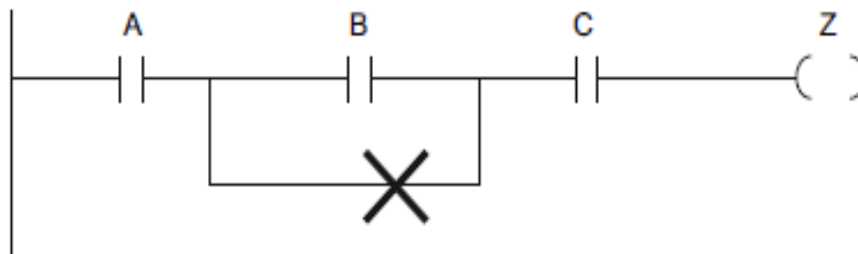


Fig 4.3.1.6 – Exemplu de scurtcircuit

Din punct de vedere al modului în care se utilizează elementele de tip bobină, se recomandă ca un element de tip bobină să fie folosit în asociere cu o singură zonă de memorie sau cu o

singură ieșire în cadrul unei aplicații. Această recomandare este foarte importantă deoarece în cadrul unei aplicații de mari dimensiuni, un bit de ieșire sau un bit de memorie poate să fie folosit în asociere cu mai multe elemente de tip bobină. Problema care poate să apară este datorată faptului că mai multe rețele încearcă pe rând să schimbe conținutul unei zone de memorie sau a unei ieșiri, moment în care apare o stare conflictuală, aplicația nefuncționând după cum ar trebui. Un astfel de exemplu este prezentat mai jos în cadrul figurii 4.3.1.7. În cadrul acestei figuri se poate observa faptul că există două ramuri ale unei rețele, prima ramură implementând funcția SI logic, iar cea de a doua ramură implementând funcția SAU logic. Amândouă ramurile rețelei sunt conectate la ieșirea Q0.0 a automatului. Problema care apare în cazul de față este următoarea: dacă rezultatul funcției SI logic este 1 logic, iar rezultatul funcției SAU logic este 0 logic, prima ramură a rețelei va schimba valoarea bitului de ieșire Q0.0 în 1 logic, dar cea de a doua ramură a rețelei va schimba și ea valoarea bitului de ieșire Q0.0. Ca și urmare a acestei situații de utilizare multiplă în asociere cu mai multe bobine a unei zone de memorie, valoarea bitului de ieșire Q0.0 s-ar putea să nu reflecte cu exactitate ceea ce s-a dorit să se implementeze.

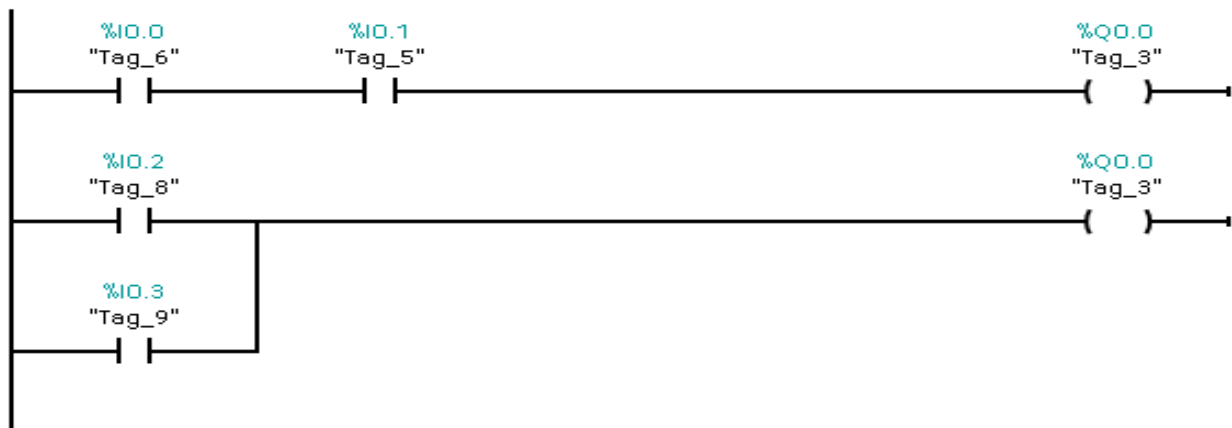


Fig 4.3.1.7 – Utilizarea incorectă a unei ieșiri

Din punct de vedere al modului în care fiecare ramură a unei rețele se încheie, se precizează faptul că fiecare ramură a unei rețele trebuie să se încheie cu un element de tip bobină sau cu un alt element bloc terminal (timer, counter, bloc de comunicare, etc). O ramură a unei rețele nu se poate încheia cu un element de tip contact. Această situație este prezentată în cadrul figurii 4.3.1.8. Dacă această condiție nu este respectată, vor exista erori de compilare.

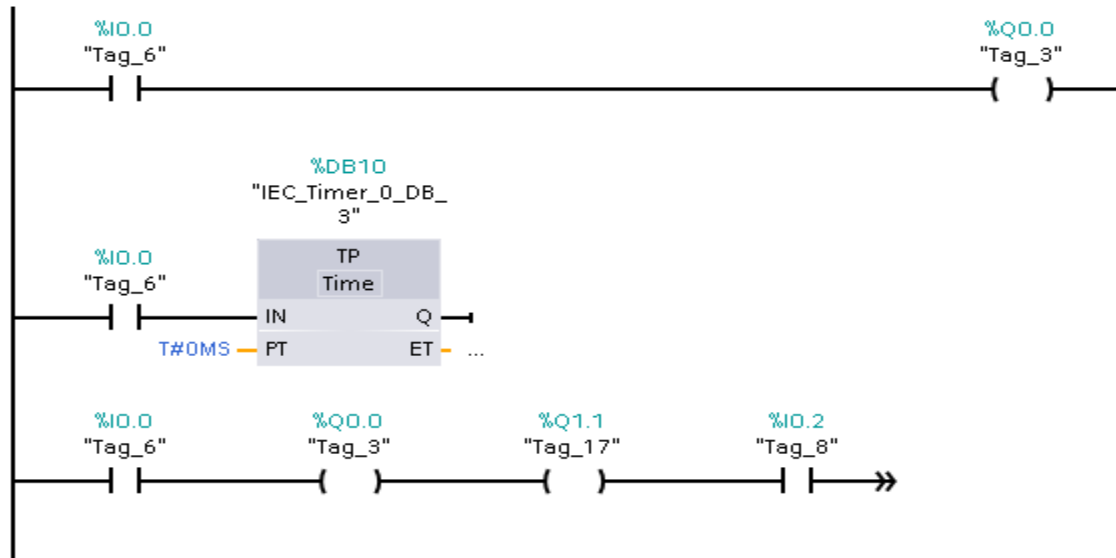


Fig 4.3.1.7 – Modul de încheiere a ramurilor unei rețele

4.3.2 Contacte și bobine cu funcții speciale. Blocuri de detecție a fronturilor de semnal. Utilizarea zonelor de memorie.

Pe lângă funcționalitățile care au fost prezentate în cadrul subcapitolului anterior, există contacte și bobine cu funcții speciale care în asociere cu o zonă de memorie pot să îndeplinească și alte funcții speciale cum ar fi:

- setarea unei zone de memorie
- resetarea unei zone de memorie
- detectarea unui front de semnal crescător
- detectarea unui front de semnal coborâtor

În cadrul acestui paragraf se va prezenta de asemenea modul în care se pot utiliza variabile de memorie. De obicei variabilele de memorie sunt folosite pentru a stoca anumite informații intermediare, pentru a crea o legătură între două rețele, pentru a împărți o rețea mai complexă care implementează o anumită funcționalitate în mai multe rețele simple sau pentru a recepționa date de la un dispozitiv cu care automatul comunică.

Accesarea unei variabile de memorie pentru a seta/reseta conținutul acesteia se poate face la nivel de bit sau la nivel de registru după cum s-a specificat și în paragrafele anterioare. Conținutul unei variabile de memorie se poate seta/reseta folosind blocuri de tip bobina care setează/resetează o zonă de memorie sau blocuri de tip bistabil. Accesarea conținutului unei zone de memorie la nivel de bit se face în marea majoritate a cazurilor utilizând blocuri de tip contact normal închis sau contact normal deschis. În continuare se vor prezenta câteva exemple care prezintă modul în care sunt folosite variabilele de memorie.

Pentru început considerăm cazul în care se dorește împărțirea unei rețele complexe în mai multe rețele simple. Pentru aceasta considerăm funcția logică $Q0.0 = I0.0 * \overline{I0.1} + I0.2$. Această funcție logică este prezentată într-o primă variantă de implementare în cadrul figurii 4.3.1.3. Dacă folosim o variabilă de memorie putem să împărțim rețeaua din figura 4.3.1.3 în două rețele, situație prezentată în figura 4.3.2.1. Se observă în cadrul acestei figuri faptul că prima rețea evaluează expresia $I0.0 * \overline{I0.1}$ și pune rezultatul acestei evaluări în zona de memorie M10.0. Cea de a doua rețea, folosește valoarea de la zona de memorie M10.0 și evaluează expresia $Q0.0 = M10.0 + I0.2$ rezultând valoarea finală a funcției. Pentru cazul din figura 4.3.2.1 zona de memorie M10.0 își va schimba valoarea de fiecare dată când variabilele I0.0 sau I0.1 își schimbă valoare, în acest fel valoarea funcției se va actualiza și ea.

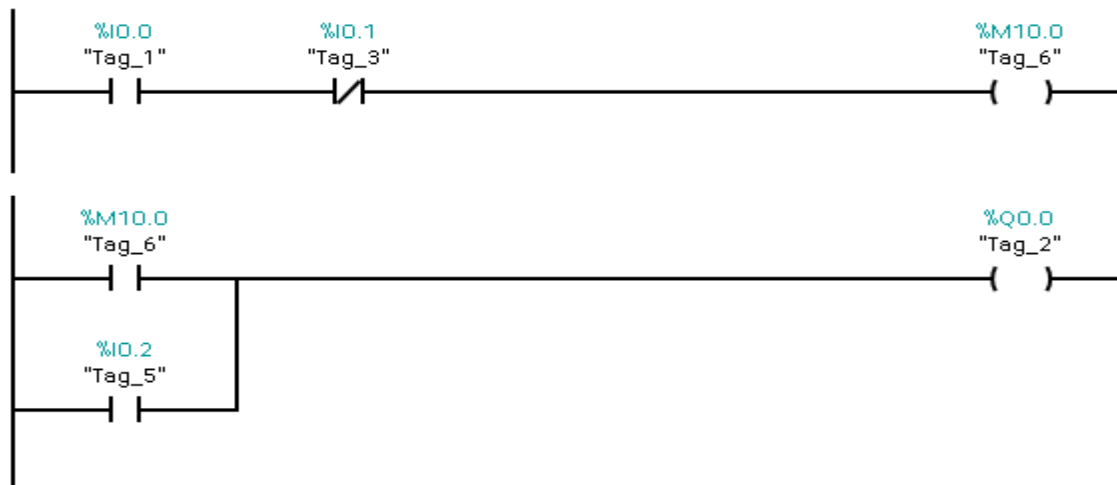


Fig 4.3.2.1 – Utilizarea zonelor de memorie

Memorarea unui eveniment care a avut loc la un moment de timp într-o zonă de memorie se poate face prin utilizarea bobinelor cu funcție specială care setează/resetează o zonă de memorie sau a bistabilelor de tip SR sau RS.

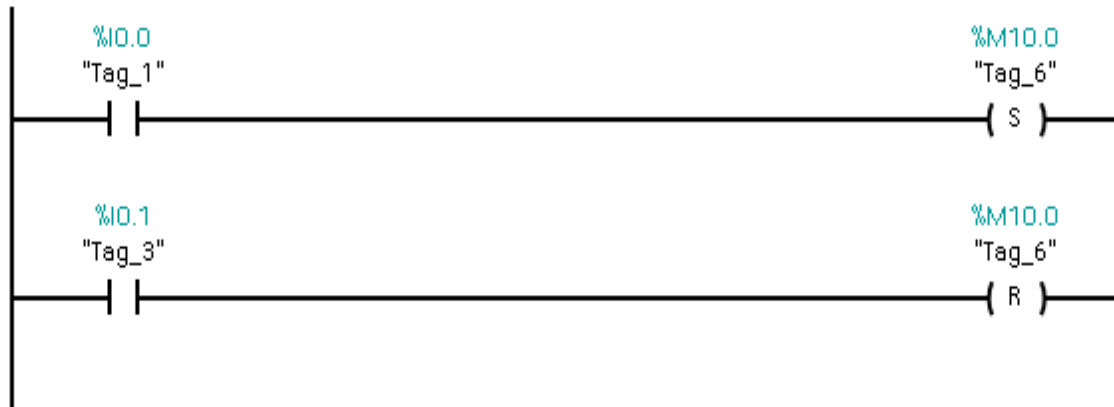


Fig 4.3.2.2 – Bobine cu funcție de set / reset

Bobinele cu funcție specială, sunt blocuri care au simbolul asemănător cu cel al unei bobine, dar care pot să seteze sau să reseteze o anumită zonă de memorie. Aceste tipuri de bobine se pot folosi de asemenea pentru setarea sau restarea semnalelor de intrare și ieșire. Aceste bobine cu funcție specială sunt prezentate în cadrul figurii 4.3.2.2. Prima rețea setează bitul M10.0, iar cea de a doua rețea resetează bitul M10.0

Pentru detectarea fronturilor de semnal există două posibilități. O primă posibilitate este reprezentată de utilizarea contactelor și bobinelor cu funcție specială. Acestea folosesc o zonă de memorie în care se stochează valoarea semnalului monitorizat la pasul anterior de rulare. Elementul de tip contact sau bobină compară valoarea curentă a semnalului cu varianta anterioară a semnalului detectând astfel o tranziție din 0 în 1 logic sau din 1 în 0 logic. Elementele de tip contact și bobină cu funcție de detectare a fronturilor de semnal sunt prezentate mai jos în cadrul figurii 4.3.2.3.

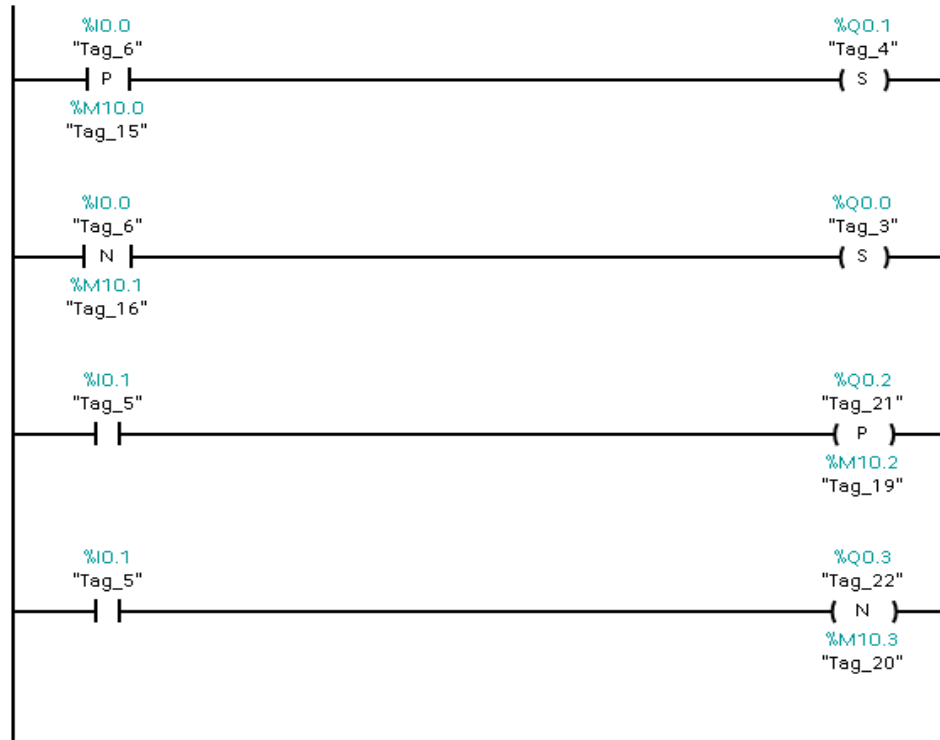


Fig 4.3.2.3 – Bobine și contacte cu funcție de detectare a fronturilor de semnal

În cadrul figurii 4.3.2.3 se observă utilizarea zonelor de memorie M10.0, M10.1, M10.2, M10.3 de către cele două contacte respectiv de către cele două bobine. Aceste zone de memorie stochează valoarea de la pasul anterior de rulare. Blocurile de tip contact vor compara valoarea curentă a semnalului cu valoarea anterioară în fiecare pas de rulare, iar dacă acestea valori vor diferi, contactele se vor închide setând ieșirea Q0.0 respectiv Q0.1 după care noua stare a semnalului este memorată în zona de memorie asociată fiecărui contact, pregătind acest element pentru un următor ciclu de rulare al rețelei. Elementele de tip bobină care detectează frontul de semnal funcționează exact ca și un contact, monitorizează valoarea unui semnal, iar dacă se va detecta o schimbare se va seta o ieșire sau o zonă de memorie.

Pentru detectarea fronturilor de semnal se mai poate utiliza de asemenea blocurile N_TRIG și P_TRIG. Aceste blocuri dispun de o intrare la care se conectează semnalul care se dorește a fi monitorizat, o ieșire care va trece din 0 logic în 1 logic atunci când se va detecta un front de semnal și o zonă de memorie în care se stochează valoarea de la pasul anterior de rulare

pentru semnalul monitorizat. Blocurile N_TRIG și P_TRIG sunt prezentate mai jos în cadrul figurii 4.3.2.4.

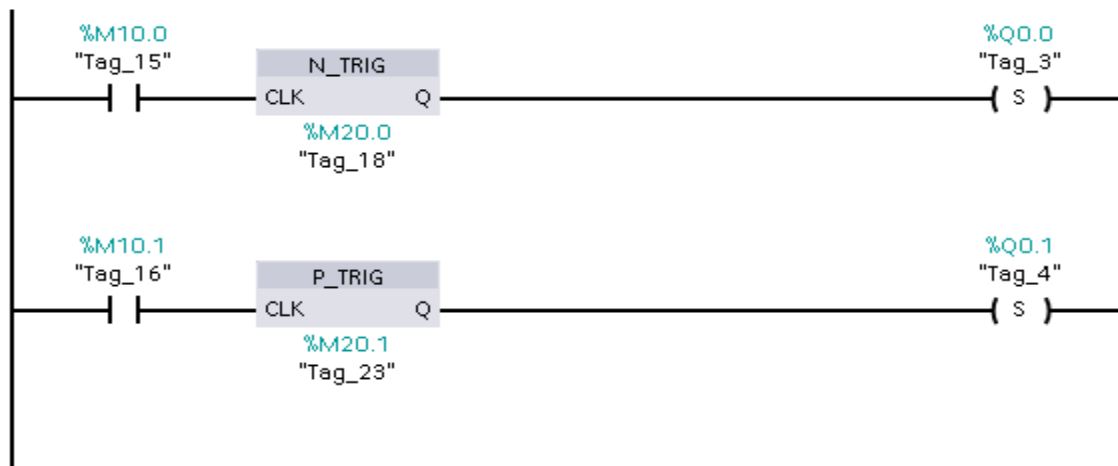


Fig 4.3.2.4 – Blocurile N_TRIG și P_TRIG

4.3.3 Bistabile

Un bistabil este un bloc care setează sau resetează o zonă de memorie în funcție de starea curentă a zonei de memorie în cauză și de valoarea a două mărimi de intrare denumite intrare de set, respectiv intrare de reset. Un bistabil poate să fie văzut ca și un ansamblu de bobine cu funcție specială care setează și resetează aceeași zonă de memorie, dar care rezolvă problema comutării conținutului zonei de memorie prin stabilirea unei reguli de care se va ține cont atunci când se ajunge în starea de conflict (atunci când ambele intrări sunt active pe valoarea 1 logic).

Există două tipuri de bistabile și anume: bistabile de tip SR și bistabile de tip RS. Pentru bistabilul de tip SR, prioritar este intrarea de reset atunci când ambele intrări sunt active, iar pentru bistabilul de tip RS prioritară este intrarea de set atunci când ambele intrări sunt active.

Pentru a înțelege mai bine modul în care funcționează cele două tipuri de bistabile se consideră figura 4.3.3.1 unde este prezentat tabelul de funcționare al celor două tipuri de bistabile. În acest tabel este prezentat modul în care evoluează ieșirea bistabilelor în funcție de valorile celor două intrări.

	S1	R	"OUT"
RS	0	0	Stare anterioară
	0	1	0
	1	0	1
	1	1	1
	S	R1	
SR	0	0	Stare anterioară
	0	1	0
	1	0	1
	1	1	0

Fig 4.3.3.1 – Tabel funcționare bistabile de tip SR și RS

Modul de funcționare al unui bistabil se poate implementa utilizând elemente de tip contact și elemente de tip bobină cu funcție de set / reset în combinație cu un bit de memorie. O variantă de implementare al unui bistabile de tip SR, respectiv tip RS este prezentată mai jos în cadrul figurii 4.3.3.2 unde este prezentată implementarea unui bistabil cu reset prioritar, iar în cadrul figurii 4.3.3.3 este prezentată implementarea unui bistabil cu set prioritar.

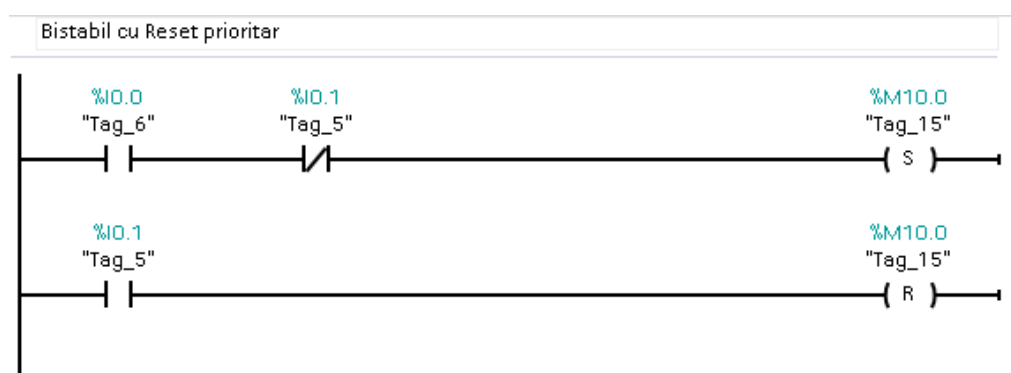


Fig 4.3.3.2 – Implementare bistabil cu intrarea de reset prioritară

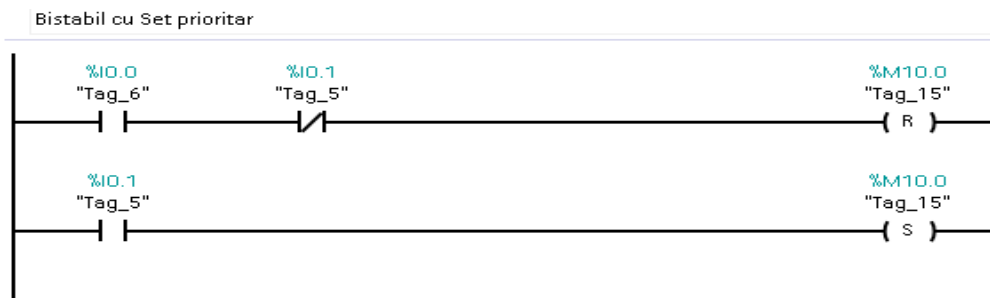


Fig4.3.3.3 – Implementare bistabil cu intrarea de set prioritară

Pe lângă varianta de implementare cu contacte și bobine, în cadrul limbajului de programare LADDER se dispune de două blocuri care implementează această funcționalitate de bistabil. Blocul de tip SR care implementează funcționarea unui bistabil cu intrarea de reset prioritară, respectiv blocul denumit RS care implementează funcționarea unui bistabil care are intrarea de set prioritară.

În figura 4.3.3.4 sunt prezentate cele două blocuri de tip bistabil (SR și RS), blocuri care se folosesc în cadrul diagramelor LADDER. Fiecare bloc de tip bistabil are două intrări, o zonă în care se setează adresa operandului care va fi setat/resetat (poate să fie un bit de intrare, un bit dintr-o zonă de memorie sau un bit de ieșire) de către bistabil și o ieșire care are valoarea egală cu conținutul de la adresa operandului pe care bistabilul o gestionează. În prima rețea este prezentat un bistabil de tip SR, iar în cea de a doua rețea este prezentat un bistabil de tip RS. În mod uzual se folosesc bistabile de tip bloc SR sau RS în cadrul aplicațiilor și nu se folosesc bistabile implementare cu elemente de tip contact și bobină.

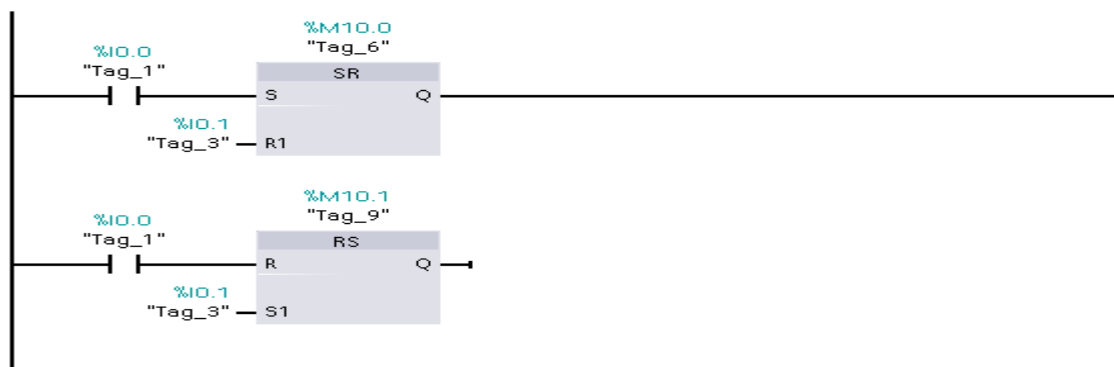


Fig 4.3.3.4 – Bistabile de tip SR și RS

4.4 Timere

Timerele sunt blocuri folosite în cadrul aplicațiilor scrise folosind limbajul de programare LADDER pentru a realiza temporizările necesare. Există patru tipuri de timere care se pot folosi pentru automatele SIEMENS 1200:

- timer on-delay (TON)
- timer off-delay (TOF)
- timer pulse (TP)
- timer accumulate (TONR)

Din punct de vedere funcțional, un timer este un bloc care folosește un numărător fizic al automatului. Timerul automatului este folosit pentru a număra o valoare egală cu o constantă de timp (PT), constantă care va fi utilizată pentru specificarea temporizării dorite. Această constantă de timp este de obicei de ordinul milisecundelor sau al secundelor. Blocul de tip timer conține o intrare (IN) folosită pentru a porni procesul de temporizare, o ieșire folosită pentru a citi valoarea curentă a numărătorului (ET) și o ieșire de tip logic (Q) care își schimbă starea și care este folosită pentru a indica faptul că s-a realizat temporizarea specificată prin constanta de timp. În continuare se vor detalia pe rând modul în care funcționează cele patru timere.

4.4.1 Timere de tip TON

Blocul specific unui timer de tip TON, împreună cu modul în care evoluează ieșirea acestuia (Q) și conținutul acestuia (PT) în funcție de valoare semnalului de intrare (IN) este prezentat în cadrul figurii 4.4.1.1. Se poate observa din această figură faptul că intrarea (IN) pornește procesul de temporizare atunci când trece din 0 logic în 1 logic. Procesul de temporizare continuă până când a fost atinsă valoarea specificată prin constanta de timp (PT). În momentul în care valoarea specificată prin constanta de timp (PT) a fost atinsă, blocul de tip timer oprește procesul de numărare și își modifică starea ieșirii (Q) din 0 logic în 1 logic. Ieșirea timerului va menține valoarea 1 logic până când intrarea (IN) își va modifica valoarea în 0 logic, moment în care timerul va fi pregătit pentru un nou ciclu de numărare. Dacă intrarea (IN) trece din 1 logic în 0 logic înainte

ca valoarea conținută de timer să atingă constanta de timp specificată (PT) conținutul timerului se va reseta și acesta va fi pregătit pentru un nou ciclu de numărare.

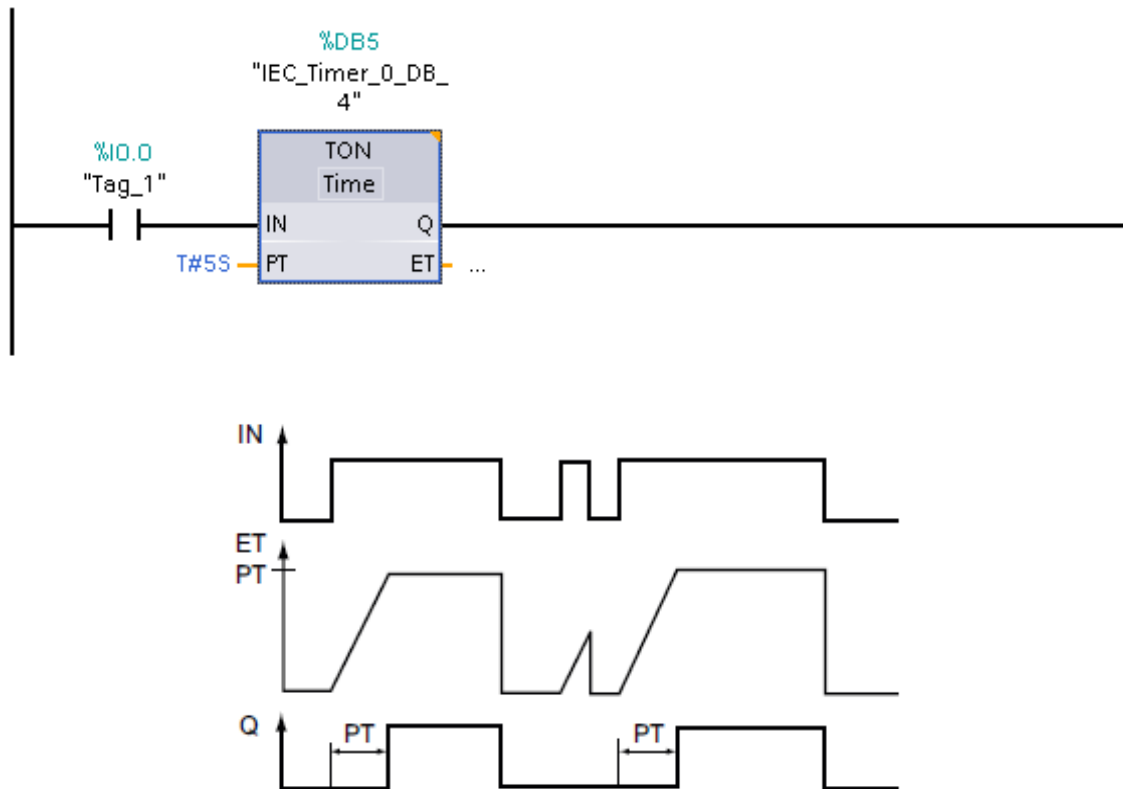


Fig 4.4.1.1 – Timer TON

4.4.2 Timer de tip TOF

Blocul specific unui timer de tip TOF, împreună cu modul în care evoluează ieșirea acestuia (`Q`) și conținutul acestuia (`PT`) în funcție de valoarea semnalului de intrare (`IN`) este prezentat în cadrul figurii 4.4.2.1. Se poate observa din această figură faptul că intrarea (`IN`) determină modificarea ieșirii (`Q`) pe nivelul 1 logic atunci când are loc o tranziție din 0 logic în 1 logic. Procesul de temporizare pornește atunci când intrarea (`IN`) trece din 1 logic în 0 logic. Procesul de temporizare continuă până când a fost atinsă valoarea specificată prin constanta de timp (`PT`). În momentul în care valoarea specificată prin constanta de timp (`PT`) a fost atinsă, blocul de tip timer oprește procesul de temporizare și își modifică starea ieșirii (`Q`) din 1 logic în 0 logic. Ieșirea

timerului va menține valoarea 0 logic până când intrarea (IN) își va modifica din nou valoarea din 0 logic în 1 logic moment în care se va iniția un nou ciclu de numărare. Dacă intrarea (IN) își va modifica valoarea înainte ca valoarea pe care timerul o conține să fie egală cu constanta de timp (PT), conținutul timerului se va reseta și va începe un nou ciclu de numărare.

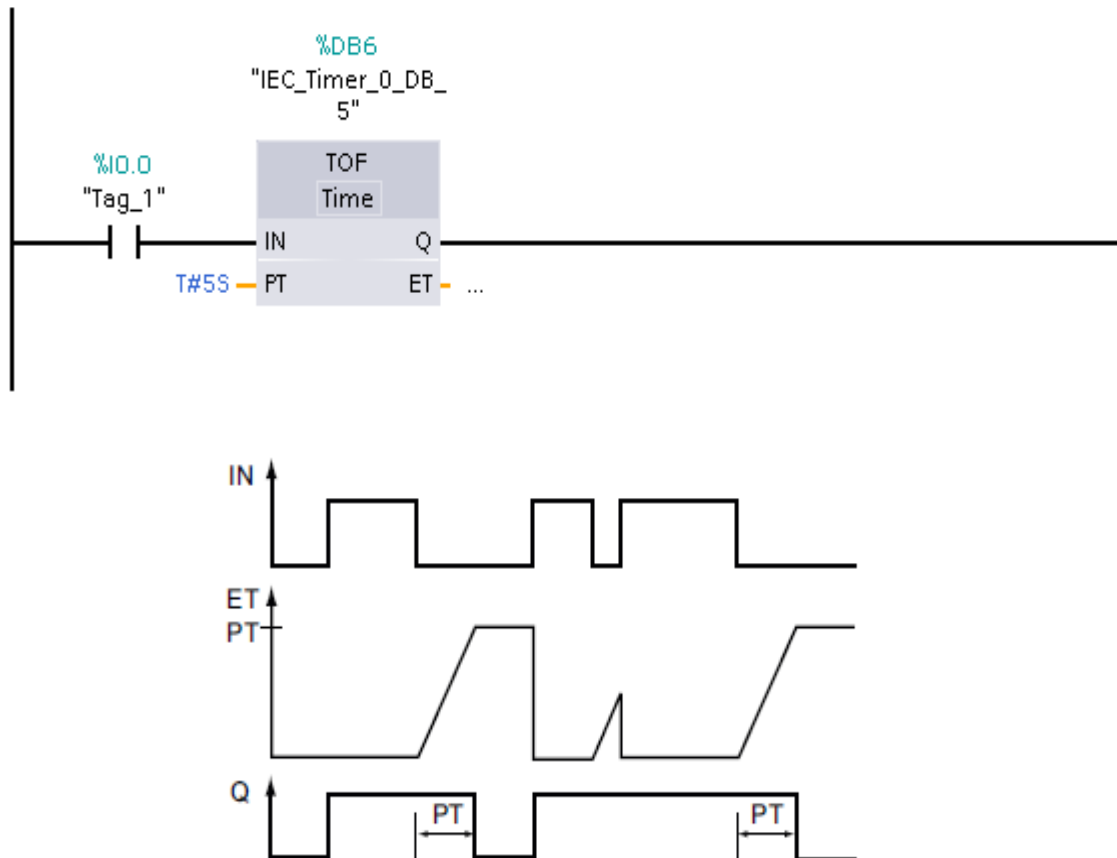


Fig 4.4.2.1 – Timer TOF

4.4.3 Timer de tip TP

Blocul specific unui timer de tip TP, împreună cu modul în care evoluează ieșirea acestuia (`Q`) și conținutul acestuia (`PT`) în funcție de valoare semnalului de intrare (`IN`) este prezentat în cadrul figurii 4.4.3.1. Se poate observa din această figură faptul că intrarea (`IN`) determină modificarea stării ieșirii (`Q`) pe nivelul 1 logic atunci când are loc o tranziție din 0 logic în 1 logic a semnalului

de intrare (IN), moment în care și procesul de temporizare pornește. Procesul de temporizare continuă până când a fost atinsă valoarea specificată prin constanta de timp (PT). În momentul în care valoarea specificată prin constanta de timp (PT) a fost atinsă, blocul de tip timer oprește procesul de temporizare și își modifică starea ieșirii (Q) din 1 logic în 0 logic. Ieșirea timerului va menține valoarea 0 logic până când intrarea (IN) își va modifica din nou valoarea din 0 logic în 1 logic moment în care se va iniția un nou ciclu de temporizare. Dacă intrarea (IN) își va modifica valoarea înainte ca valoarea pe care timerul o conține să fie egală cu constanta de timp (PT), conținutul timerului nu se va reseta, procesul de numărare încheindu-se doar atunci când valoarea înscrisă în timer atinge valoare constantei de timp (PT).

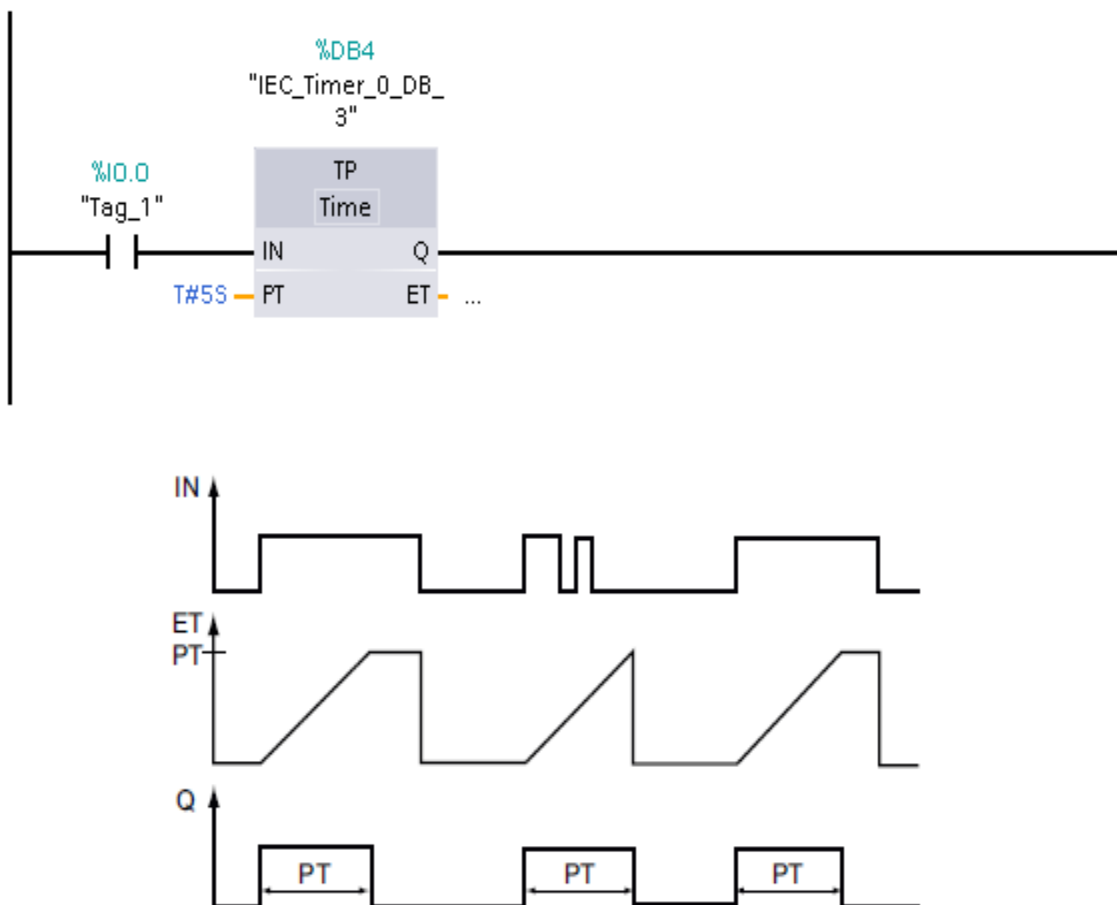


Fig 4.4.3.1 – Timer TP

4.4.4 Timer de tip TONR

Blocul specific unui timer de tip TONR, împreună cu modul în care evoluează ieșirea acestuia (Q) și conținutul acestuia (PT) în funcție de valoare semnalului de intrare (IN) și a semnalului de reset (R) este prezentat în cadrul figurii 4.4.4.1.

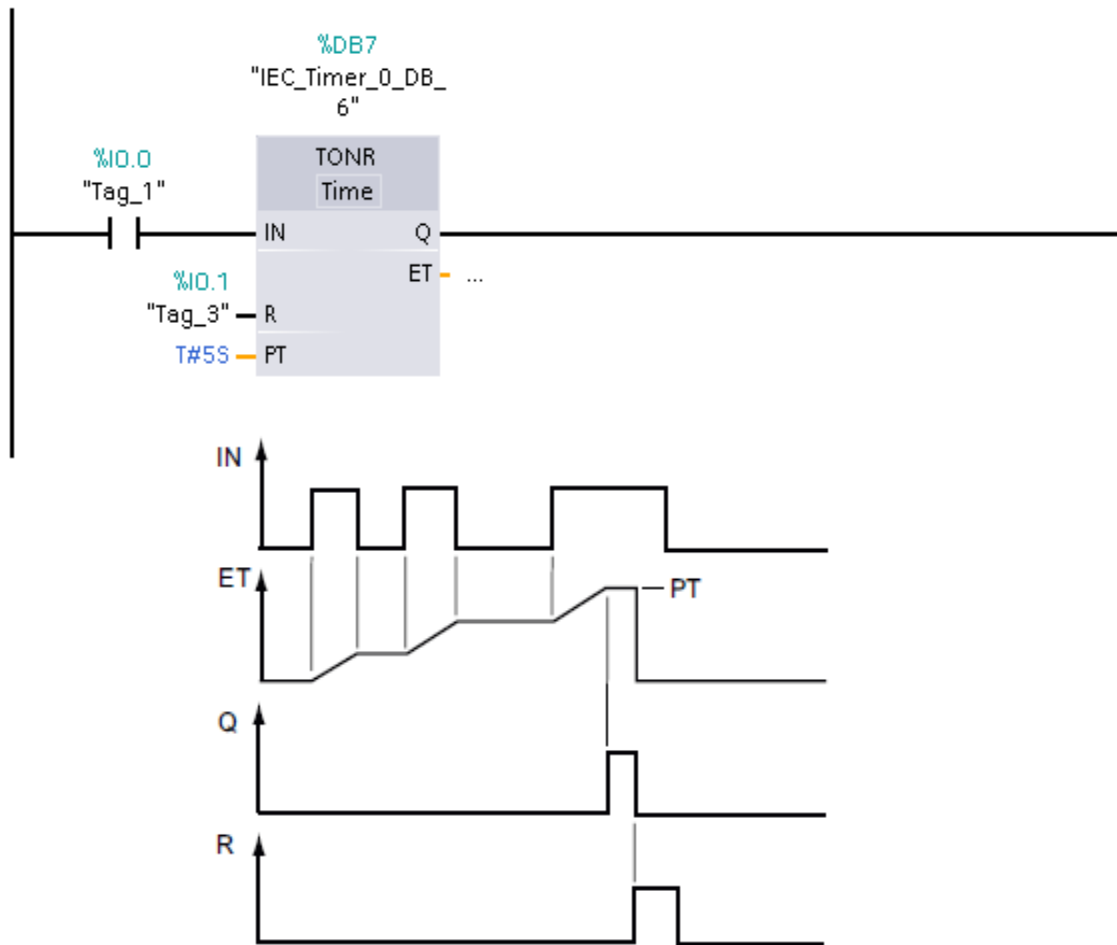


Fig 4.4.4.1 – Timer TONR

Se poate observa din această figură faptul că intrarea (IN) pornește procesul de temporizare atunci când trece din 0 logic în 1 logic. Procesul de temporizare continuă până când a fost atinsă valoarea specificată prin constanta de timp (PT) sau atât timp cât valoarea intrării (IN) rămâne 1 logic. În momentul în care valoarea specificată prin constanta de timp (PT) a fost atinsă, blocul

de tip timer oprește procesul de temporizare și îți modifică starea ieșirii (Q) din 0 logic în 1 logic. Ieșirea temporizatorului va menține valoarea 1 logic indiferent de evoluția intrării (IN). Ieșirea (Q) își va modifica valoarea în 0 logic în momentul în care apare o tranziție din 0 logic în 1 logic la intrarea de reset (R), moment în care timerul va fi pregătit pentru un nou ciclu de temporizare. Dacă intrarea (IN) trece din 1 logic în 0 logic înainte ca valoarea conținută de timer să atingă constanta de timp specificată (PT) conținutul timerului se va păstra, procesul de numărare continuând atunci când intrarea (IN) va trece din nou în 1 logic.

4.5 Countere

Counter-ele (numărătoare) sunt blocuri folosite în cadrul aplicațiilor scrise folosind limbajul de programare LADDER pentru a contoriza anumite evenimente apărute. Aceste blocuri contorizează numărul evenimentelor apărute la o intrare, denumită intrare de numărare. Prin eveniment se definește tranziția unui semnal logic din nivelul 0 logic în nivelul 1 logic, adică apariția unui front crescător al semnalului aplicat la intrarea de numărare a counter-ului. Există trei tipuri de countere care se pot folosi pentru automatele SIEMENS 1200:

- counter up (CTU)
- counter down (CTD)
- counter up-down (CTUD)

Din punct de vedere funcțional, un counter este un bloc care folosește o zonă de memorie a automatului pentru a număra evenimente. Blocul de tip timer conține la modul general o intrare care reprezintă sursa de evenimente ce se doresc a fi numărate, o intrare care încarcă în numărător o valoare prestabilită sau resetează conținutul numărătorului, o ieșire de tip logic care își schimbă valoarea în funcție de valoarea conținută în numărător și o ieșire de tip numeric care conține valoarea curentă a numărătorului. În continuare va fi prezentat modul în care cele trei tipuri de numărătoare funcționează și modul în care semnalele de control influențează funcționarea celor trei tipuri de countere amintite mai sus.

4.5.1 Counter de tip CTU

Blocul specific unui counter de tipul CTU, împreună cu modul în care evoluează mărimea de ieșire (Q) și valoare curentă a counter-ului (CV) în funcție de intrarea (IN) și valoarea presetată (PV) este prezentat în cadrul figurii 4.5.1.1. Se poate observa în această figură faptul că mărimea de ieșire (Q) a counter-ului are nivelul 0 logic, atâta timp cât valoarea curentă a counterului este mai mică decât valoarea presetată (PV). În momentul în care valoarea curentă a counter-ului devine mai mare sau egală cu valoarea presetată (PV) ieșirea (Q) va trece din nivelul 0 logic în nivelul 1 logic. Semnalul de intrare reset (R) are rolul de a înscrie valoarea de 0 în counter (CV) și de a seta ieșirea (Q) a counterului pe nivelul de 0 logic, pregătind astfel counter-ul de un nou ciclu de numărare.

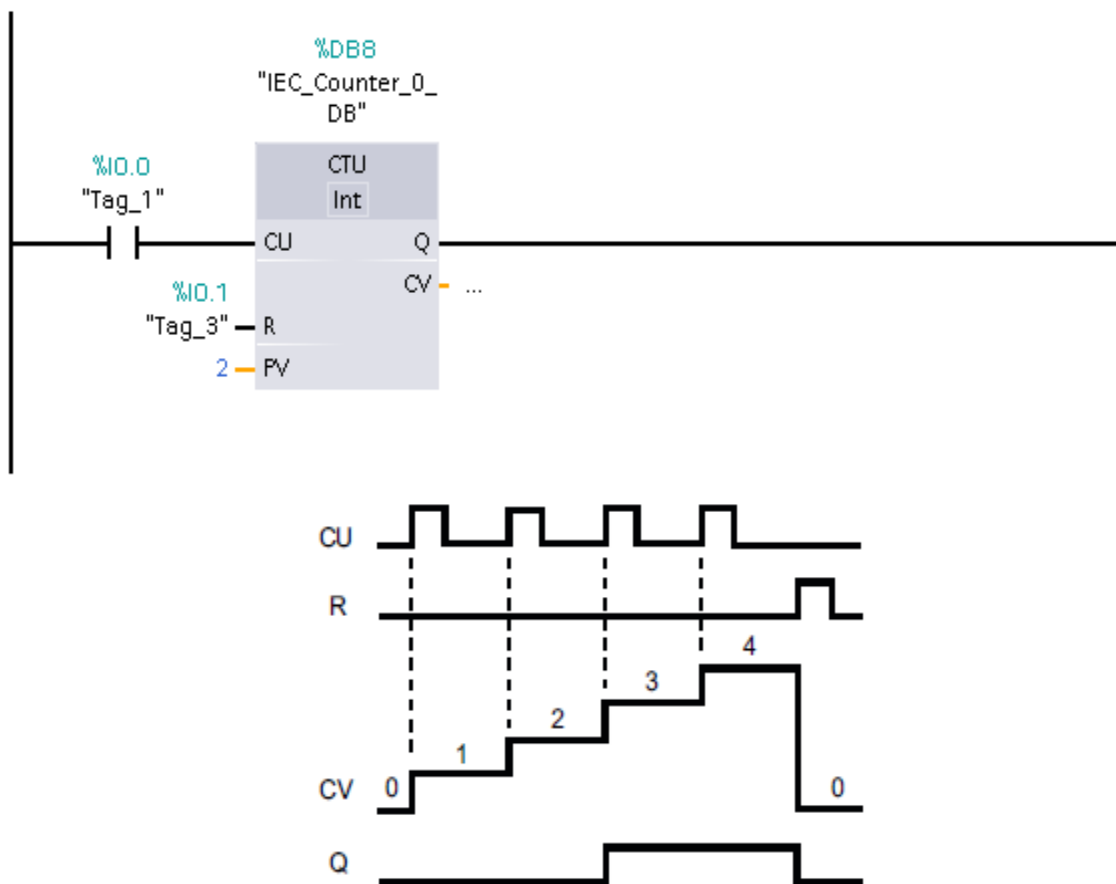


Fig 4.5.1.1 – Counter CTU

4.5.2 Counter de tip CTD

Blocul specific unui counter de tipul CTD, împreună cu modul în care evoluează mărimea de ieșire (Q) și valoare curentă a counter-ului (CV) în funcție de intrarea (IN) și valoarea presetată (PV) este prezentat în cadrul figurii 4.5.2.1. Se poate observa în această figură faptul că mărimea de ieșire (Q) a counter-ului are nivelul 0 logic, atâta timp cât valoarea curentă a counterului este mai mare decât 0. În momentul în care valoarea curentă a counter-ului devine mai mică sau egală cu 0 ieșirea (Q) va trece din nivelul 0 logic în nivelul 1 logic. Semnalul de intrare load (LOAD) are rolul de a înscrie valoarea presetată (PV) în counter și de a seta ieșirea (Q) a couterului pe nivelul de 0 logic, pregătind astfel counter-ul de un nou ciclu de numărare.

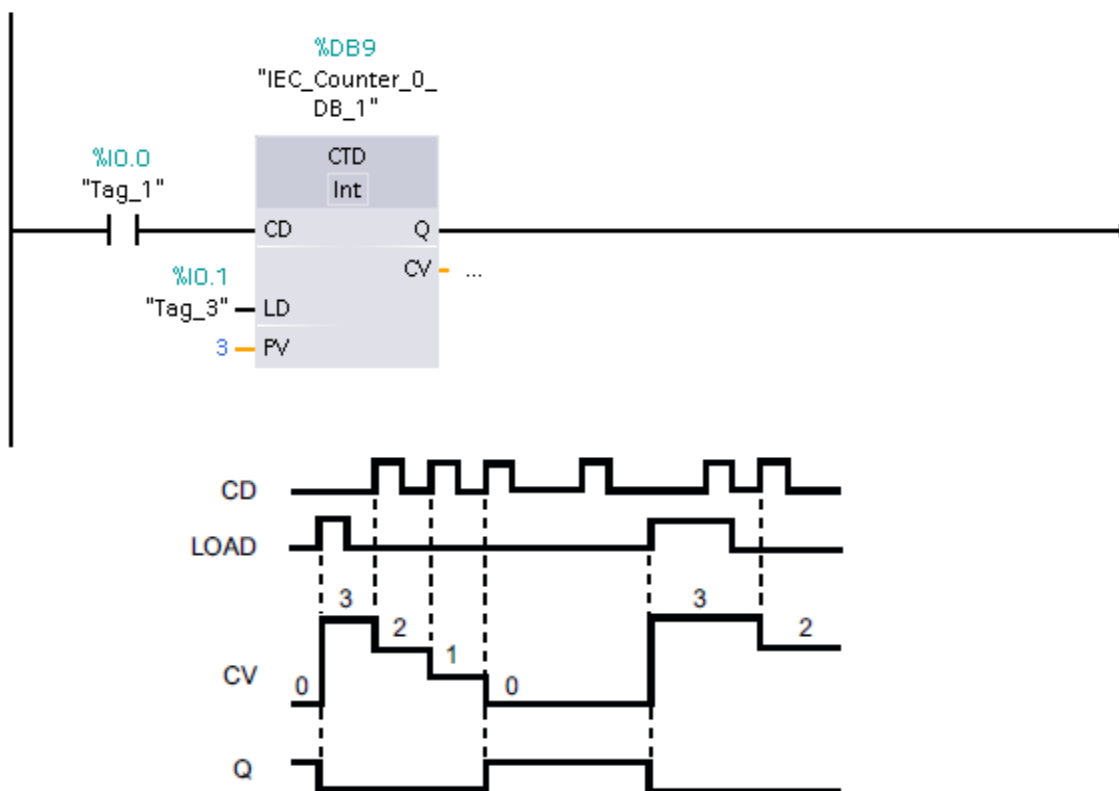


Fig 4.5.2.1 – Counter CTD

4.5.3 Counter de tip CTUD

Blocul specific unui counter de tipul CTUD, împreună cu modul în care evoluează mărimea de ieșire (QU), mărimea de ieșire (QD) și valoare curentă a counter-ului (CV) în funcție de intrarea (CU), intrarea (CD), valoarea presetată (PV), semnalul de reset (R) și semnalul de load (LOAD) este prezentat în cadrul figurii 4.5.3.1.

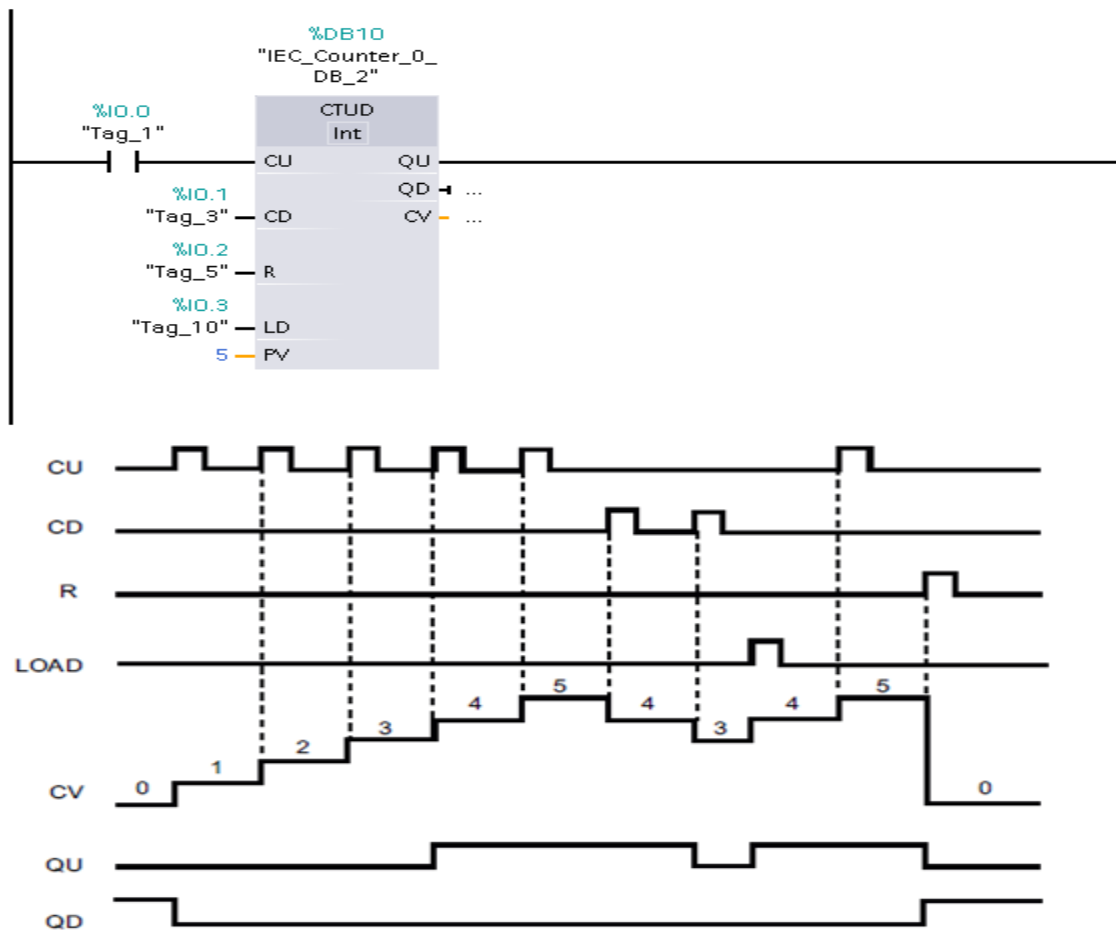


Fig 4.5.3.1 – Counter CTUD

Se poate observa din figura 4.5.3.1 faptul că un counter de tipul CTUD este de fapt un counter care combină modul de funcționare al counterului CTU și al counterului CTD. Cele două ieșiri ale counter-ului CTUD notate cu QU și QD se comportă exact ca ieșirile counterelor CTU și CTD.

Ieșirea (QU) are valoarea 1 logic atunci când conținutul counterului (CV) este mai mare sau egal cu valoarea de referință (PV), iar în cazul în care conținutul counterului (CV) este mai mic decât valoarea de referință (PV) ieșirea (QU) va avea valoarea 0 logic. Ieșirea (QD) are valoare 0 logic atunci când conținutul counterului (CV) este mai mare ca și zero, , iar în cazul în care conținutul counterului (CV) este mai mic sau egal cu 0 ieșirea (QU) va avea valoarea 1 logic.

Semnalul (LOAD) încarcă valoarea de referință (PV) în counter, (CV) devenind egal cu (PV) în momentul în care apare un front crescător al semnalului (LOAD). Semnalul (RESET) încarcă valoarea 0 în counter, (CV) devenind egal cu 0 în momentul în care apare un front crescător al semnalului (RESET).

4.6 Transmiterea și recepționarea de date

Un automat programabil este capabil să comunice în rețea cu alte dispozitive pentru a transmite date sau pentru a recepționa date sau comenzi. Transmiterea și recepționarea de date pentru un automat SIEMENS din seria 1200 se face utilizând protocolul de comunicare TCP/IP. În continuarea acestui paragraf se va prezenta modul în care se poate stabili o conexiune între un automat SIEMENS din seria 1200 și un alt dispozitiv.

Pentru transmiterea și recepționare de date se pot folosi două blocuri existente și anume TSEND_C și TRCV_C. Blocul TSEND_C este folosit pentru transmiterea de date dinspre automat către un alt dispozitiv, iar blocul TRCV_C este folosit pentru recepționare de date de la un alt dispozitiv. Transmiterea de date, respectiv recepționarea de date se face utilizând protocolul TCP IP după cum a fost amintit și mai sus. În continuare se va arăta modul în care trebuie setate cele două blocuri de TSEND_C și TRCV_C pentru a permite comunicarea și recepționarea de date de la un alt dispozitiv.

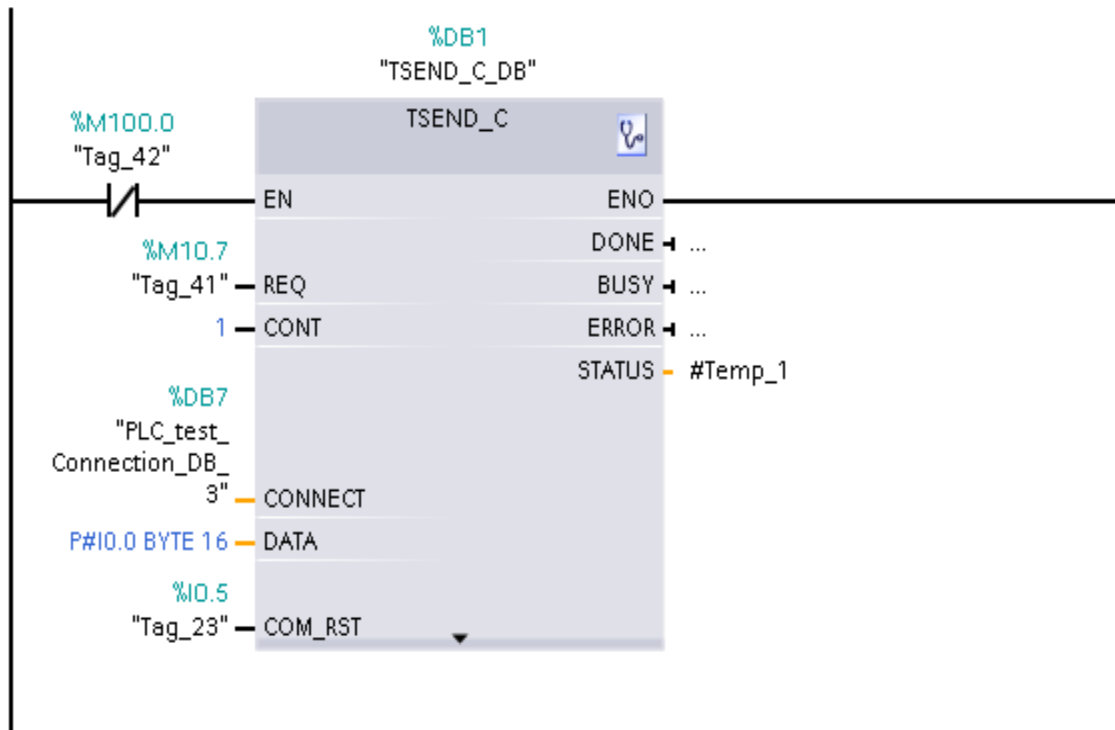


Fig 4.6.1 – Bloc pentru transmiterea de date

Blocul folosit pentru transmiterea datelor TSEND_C este prezentat în cadrul figurii 4.6.1. Ca și semnale de intrare în cadrul acestui bloc avem următoarele semnale:

- EN – semnal de enable care specifică dacă transmiterea de date trebuie să aibă loc sau nu
- REQ – semnal care dictează momentul în care are loc transmisia de date (când se detectează un front crescător al semnalului datele vor fi transmise)
- CONT – specific modul în care va fi gestionată conexiunea (0 – conexiunea va fi închisă după ce datele au fost transmise, 1- conexiunea rămâne deschisă după transmiterea datelor)
- CONNECT – bloc pentru specificarea proprietăților conexiunii (adresă IP, portul pe care are loc conexiunea, etc).
- DATA – specific adresa și dimensiunea datelor care vor fi transmise
- COM_RST – folosit pentru resetarea conexiunii

Connection parameter

General

	Local	Partner
End point:	PLC_test	Unspecified
Interface:	CPU 1214C DC/DC/DC, IE	
Subnet:	PN/IE_1	
Address:	192.168.0.111	192.168.0.190
Connection type:	TCP	
Connection ID:	123	
Connection data:	PLC_test_Connection_DI	
	☒ Establish active connection	☐ Establish active connection

Address details

	Local Port	Partner Port
Port (decimal):		2000

Fig 4.6.2 – Configurare bloc TSEND_C – IP și port

Toate semnalele de intrare prezentate mai sus trebuie configurate corespunzător pentru ca transmiterea de date să funcționeze cu succes. Singurii parametri ai blocului TSEND_C care sunt mai greu de configurat sunt CONNECT și DATA, ceilalți parametri se configurează relativ ușor (specificând o valoare logică sau o zonă de memorie la intrarea acestora). Pentru configurarea parametrilor CONNECT și DATA este nevoie să se vizualizeze proprietățile blocului de TSEND_C (fig 4.6.2 și fig 4.6.3). În figura 4.6.2 este prezentat modul în care trebuie să se seteze adresa de IP și portul dispozitivului către care se vor transmite date, iar în figura 4.6.3 se poate observa modul în care se setează blocul de date care urmează a fi trimis.

In/Outputs

Associated connection pointer (CONNECT)

Pointer to the associated connection description

CONNECT:

Send area (DATA):

Specified the data area to be sent

Start:

Length:

Send length (LEN):

Maximum number of bytes to send with the request

LEN:

Restart of the block (COM_RST):

Complete restart of the block

COM_RST:

Fig 4.6.3 – Configurare bloc TSEND_C – date trimise

Blocul folosit pentru recepționarea datelor TSEND_C este prezentat în cadrul figurii 4.6.4. Ca și semnale de intrare în cadrul acestui bloc avem următoarele semnale:

- EN – semnal de enable care specifică dacă recepția de date trebuie să aibă loc sau nu
- RN_R – semnal care dictează momentul în care are loc recepția de date (când se detectează un front crescător al semnalului datele vor fi recepționate)
- CONT – specifică modul în care va fi gestionată conexiunea (0 – conexiunea va fi închisă după ce datele au fost recepționate, 1- conexiunea rămâne deschisă după recepționarea datelor)
- CONNECT – bloc pentru specificarea proprietăților conexiunii (adresă IP, portul pe care are loc conexiunea, etc).
- DATA – specifică adresa unde vor fi puse datele recepționate
- COM_RST – folosit pentru resetarea conexiunii

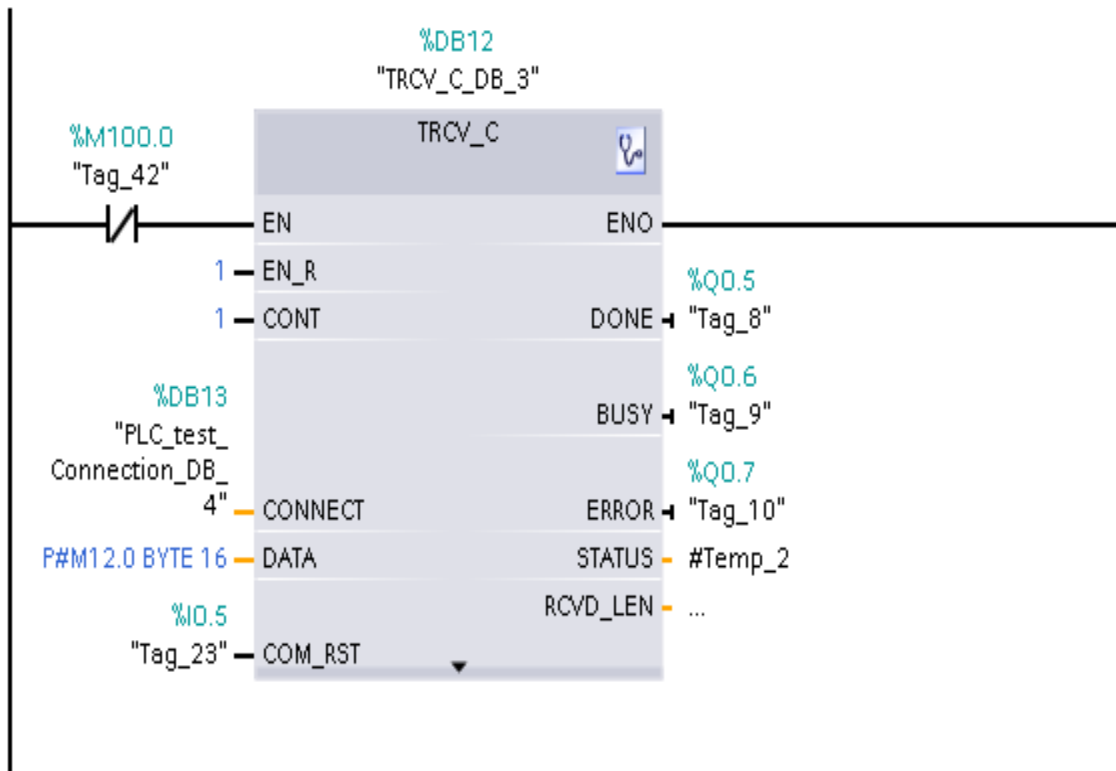


Fig 4.6.4 – Configurare bloc TRCV_C

Toate semnalele de intrare prezentate mai sus trebuie configurate corespunzător pentru ca recepționarea de date să funcționeze cu succes. Singurii parametri ai blocului TRCV_C care sunt mai greu de configurat sunt CONNECT și DATA, ceilalți parametri se configurează relativ ușor (specificând o valoare logică sau o zonă de memorie la intrarea acestora). Configurarea parametrilor CONNECT și DATA se face în mod similar cu parametrii blocului TSEND_C.

Pentru generarea unui semnal folosit pentru trimiterea, respectiv recepționarea de date utilizând un bloc de tip TSEND_C respectiv TRCV_C se poate utiliza un timer care va genera la ieșirea acestuia un semnal de tip dreptunghiular. Timerul trebuie să se repornească singur la un anumit interval de timp specificat prin constanta de timp a timerului care va dicta și frecvența cu care se vor primi, respectiv transmite datele de către cele două blocuri. În figura 4.6.5 este prezentată o astfel de schemă care generează un semnal ce poate fi folosit pentru transmiterea și recepționarea de date la intervale de 100 milisecunde. Pentru a asigura repornirea în mod ciclic

a timerului se poate observa faptul că s-a folosit un bistabil de tip SR care folosește ieșirea timerului pentru a resetare sa.

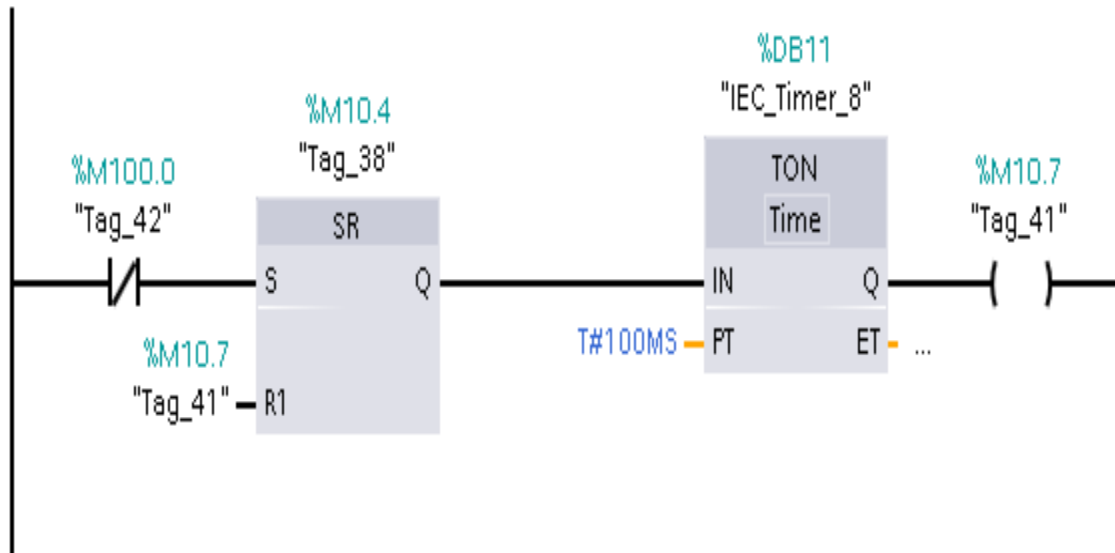


Fig 4.6.5 – Generare semnal dreptunghiular

Utilizând un bloc TSEND_C, un bloc TRCV_C și un timer se poate realiza comunicarea cu alte automate sau cu alte dispozitive pentru a asigura transmiterea de date, monitorizarea procesului condus sau primirea de comenzi.

4.7 Funcții bloc

Funcțiile bloc reprezintă o modalitate prin care în mediul de lucru TIA Portal se poate refolosi codul bazat pe diagrame LADDER. O funcție bloc crează practic un nou bloc în TIA Portal, bloc care poate să fie folosit după aceea ori de câte ori este nevoie. Refolosirea codului în TIA Portal este utilă deoarece în momentul în care există într-o aplicație două sau mai multe porțiuni de cod care îndeplinesc aceeași funcționalitate se poate crea o singură funcție bloc care să fie refolosită. Acest lucru aduce o serie de beneficii cum ar fi: testarea o singură dată a codului scris, modificarea în cadrul funcției bloc va asigura propagarea modificărilor peste tot în aplicație unde funcția bloc este utilizată, posibilitatea structurării codului aplicației.

O funcție bloc poate fi văzută ca și un bloc care permite definirea unor variabile de intrare și ieșire și care realizează o prelucrare a informației furnizate de către variabilele de intrare, returnând un rezultat care este accesibil în afara funcției bloc prin intermediul variabilelor de ieșire. Pentru o mai bună înțelegere a conceptului de funcție bloc, se consideră figura 4.7.1. În cadrul acestei figuri se poate observa diferența dintre o structură liniară unde codul este scris sub forma de rețele care rulează una după alta în mod ciclic și structura modulară care este compusă din cod sub forma de rețele combinate împreună cu funcții bloc unde rețelele sunt structurate în așa fel încât funcția bloc să realizeze o anumită funcționalitate. O altă întrebuintare a funcțiilor bloc este cazul în care se dorește implementarea unei rutine de tratare a întreruperilor. Pentru aceasta se poate implementa o funcție bloc care poate să fie executată doar când se detectează o anumită cerere sau o anumită condiție logică este validată.

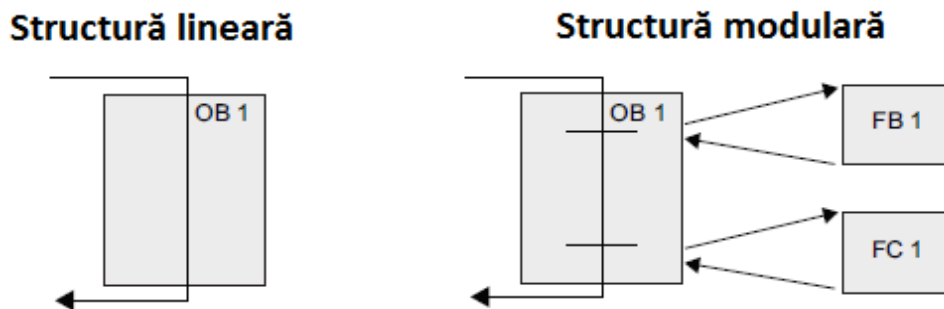


Fig 4.7.1 – Conceptul de funcție bloc

Fiecare funcție bloc este folosită în asociere cu un bloc de memorie, fiecare instanță a unei funcții bloc având asociată o zonă distinctă de memorie. Acest lucru asigură faptul că dacă avem mai multe instanțe a aceleași funcții bloc care rulează la un anumit moment de timp, fiecare funcție bloc va modifica doar blocul de date cu care aceasta este asociată. Desigur există posibilitatea utilizării de variabile statice în cadrul unei funcții bloc dacă este nevoie, caz în care dacă o singură instanță a unei funcții bloc va modifica o variabilă statică, modificarea va fi văzută de toate instanțele acelei funcții bloc existente la momentul respectiv. Acest concept este prezentat mai jos în cadrul figurii 4.7.2 unde se poate observa existența unei funcții bloc denumite FB_Motor. Această instanță este folosită în asociere cu trei blocuri de date diferite, îndeplinind trei sarcini diferite.

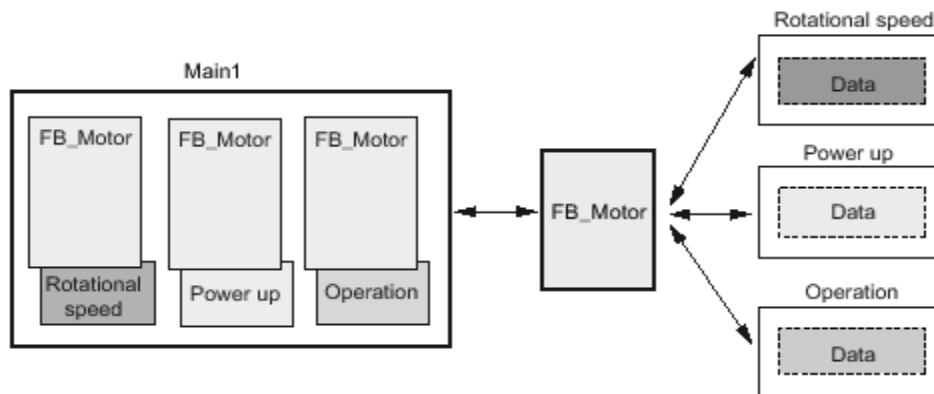


Fig 4.7.2 – Legătura dintre funcții bloc și zonele de memorie

Crearea unei funcții bloc se face din folderul “Program blocks”, selectând opțiunea “Add New Block”(fig 4.7.3) moment în care se deschide fereastra din figura 4.7.4 de unde se selectează tipul de bloc care se dorește a fi creat (Function block).

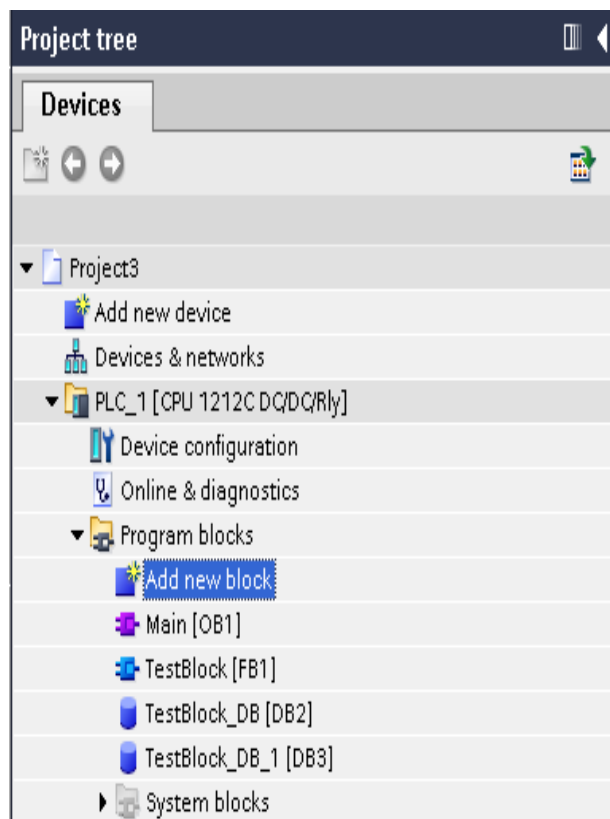


Fig 4.7.3 – Crearea funcție bloc

Tot în cadrul ferestrei prezentate în figura 4.7.4 se poate seta și numele blocului care urmează a fi creat, nume care va fi folosit în cadrul aplicației TIA Portal pentru identificarea blocului. După selectarea numelui pentru blocul care se dorește a fi creat prin apăsare butonului OK se va crea un nou block (cu numele TestBlock pentru cazul de față). Acest bloc nou creat se poate observa în cadrul figurii 4.7.3, amplasat sub blocul Main al aplicației.

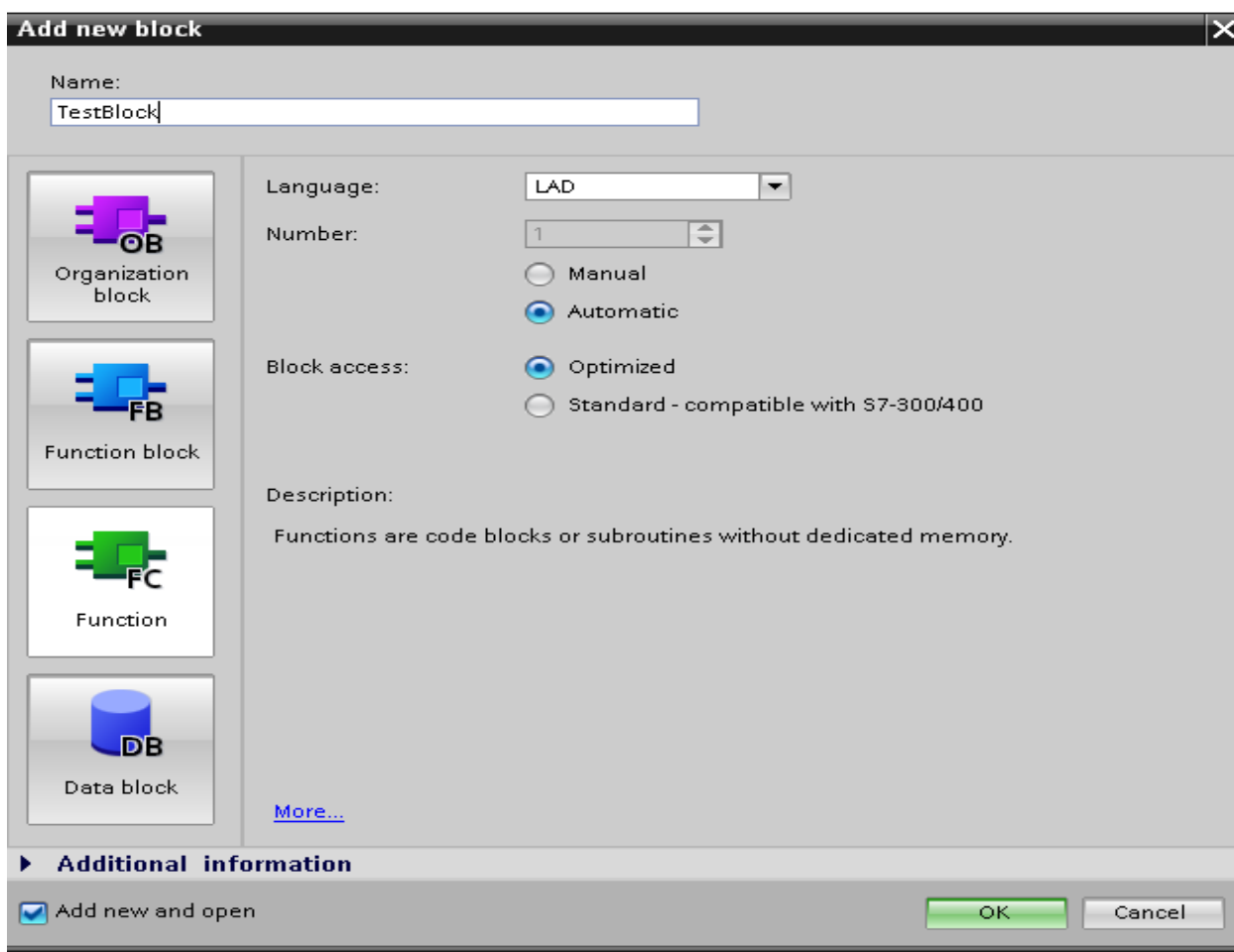


Fig 4.7.4 – Creare funcție bloc

O funcție bloc permite declararea de variabile care sunt folosite în interiorul acestui bloc pentru a stoca anumite informații, pentru a verifica anumite condiții sau pentru a returna rezultate. În funcție de scopul pentru care va fi folosită variabila de memorie, acesteia i se va asigna un anumit tip în momentul în care va fi creată. Variabilele care se pot declara în cadrul unei funcții bloc sunt de cinci tipuri și anume:

- variabile de intrare - variabilele de intrare sunt variabile care în cadrul funcției bloc pot fi folosite doar pentru verificarea valorilor acestora. Variabilele declarate ca și variabile de intrare nu pot să fie asignate, valoarea lor nu poate să fie schimbată de către funcția bloc (se folosesc în general în asociere cu contacte, comparatoare, etc).
- variabile de ieșire – variabilele de ieșire sunt variabile care în cadrul funcției bloc pot să li asigneze valori. Acest tip de variabile vor fi folosite pentru returnarea unei valori de către funcția bloc. Conținutul variabilelor de ieșire nu poate să fie verificat (se folosesc în general în asociere cu bobine).
- variabile de intrare / ieșire – unei variabile de acest tip i se pot asigna valori și în același timp se poate verifica conținutul ei (aceste tipuri de variabile pot să fie asociate atât cu contacte cât și cu bobine)
- variabile statice – variabilele de acest tip au aceeași valoare pentru toate instanțele aceluiași tip de funcție bloc (dacă în program există n instanțe ale aceleiași funcții bloc, în momentul când o instanță a funcției bloc efectuează o operație de schimbare a conținutului unei variabile statice, toate celelalte n-1 instanțe vor vedea această modificare)
- variabile temporare – sunt variabile care sunt folosite doar în interiorul funcției bloc.

Declararea de variabile se face în cadrul funcției bloc, în fereastra de declarare a variabilelor funcției bloc, fereastră prezentată în cadrul figurii 4.7.5. Această fereastră poate să fie expandată / colapsată folosind cele două săgeți amplasate la baza ei. În cadrul acestei ferestre se poate selecta tipul de variabilă care se dorește a fi folosită, se poate specifica valoarea inițială a variabilei sau se poate specifica dacă la pierderea tensiunii de alimentare conținutul variabilei să fie memorat sau nu. Se poate observa în cadrul figurii 4.7.5 faptul că au fost declarate două variabile de intrare denumite IN1 respectiv IN2 și două variabile de ieșire denumite OUT1 și OUT2.

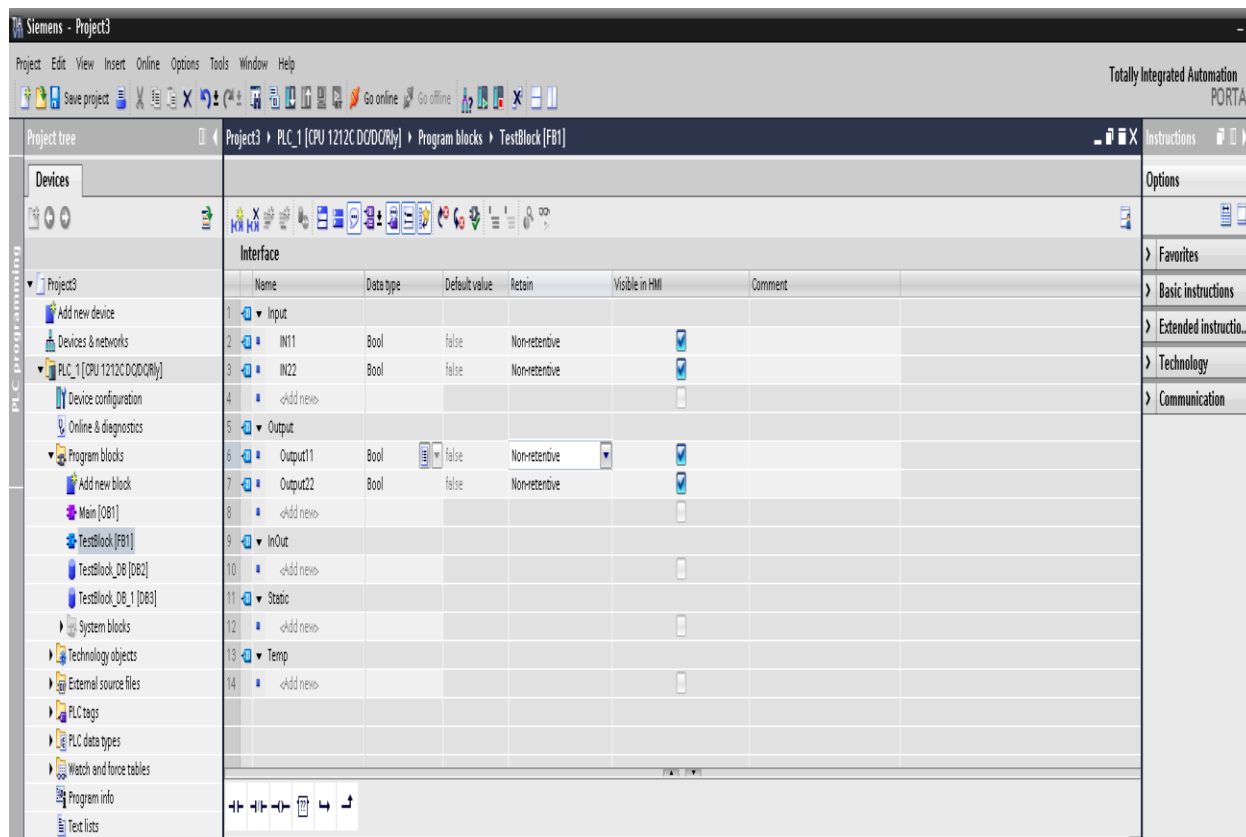


Fig 4.7.5 – Declarare variabile pentru o funcție bloc

Pentru exemplificarea conceptului de funcție bloc, se consideră următorul scenariu: se dorește realizarea unei funcții bloc care să realizeze două operații logice și anume operația SI logic și operația SAU logic. Cele două operații logice sunt prezentate în cadrul figurii 4.7.6. Aici se poate observa operația SI logic și operația SAU logic unde variabilele de intrare sunt reprezentate de intrările I0.0 și I0.1 sunt evaluate, iar rezultatul operației logice SI este trimis la ieșirea Q0.0 iar pentru operația logică SAU rezultatul este trimis la ieșirea Q0.1.

Se dorește realizarea unei funcții bloc care să implementeze operațiile logice prezentate în cadrul figurii 4.7.6. Funcția bloc va trebui să aibă două variabile de intrare care vor fi folosite pentru evaluarea operațiilor logice SI respectiv SAU și două variabile de ieșire care vor fi folosite pentru a returna rezultatul operațiilor logice. Se consideră ca și variabile de intrare variabilele IN1 și IN2, iar ca variabile de ieșire se consideră variabilele OUT1 și OUT2 (declararea acestor variabile se face folosind fereastra prezentată în figura 4.7.5).

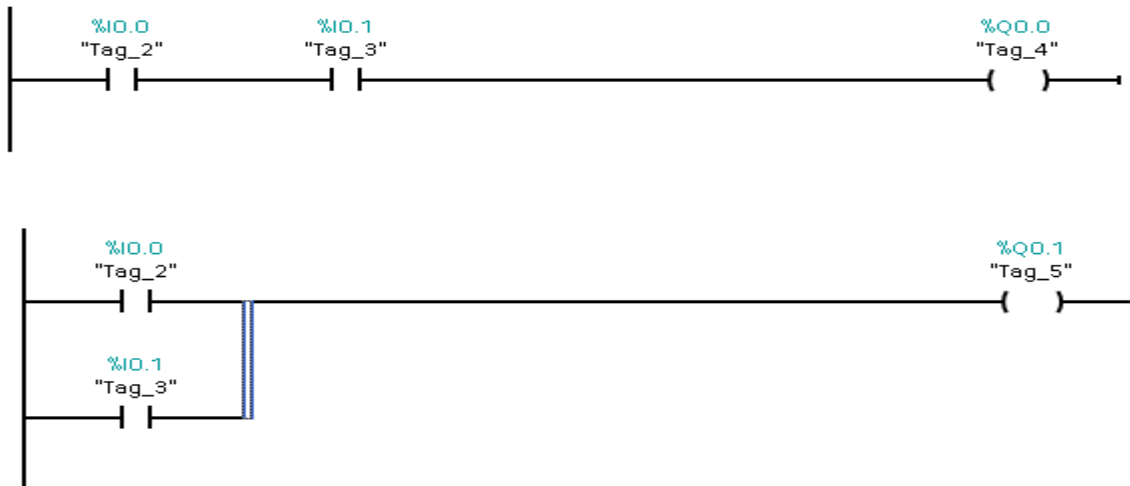


Fig 4.7.6 – Operația SI logic și operația SAU logic

Codul LADDER scris în cadrul funcției bloc este prezentat mai jos în cadrul figurii 4.7.7. În prima rețea LADDER din figura 4.7.7 este implementată funcția SI logic, sunt evaluate valorile variabilelor de intrare IN1 și IN2, iar rezultatul evaluării este returnat în afara funcției bloc utilizând variabila de ieșire OUT1. În cea de a doua rețea din figura 4.7.7 este implementată funcția SAU logic, sunt evaluate valorile variabilelor de intrare IN1 și IN2, iar rezultatul evaluării este returnat în afara funcției bloc utilizând variabila de ieșire OUT2. Folosind doar două rețele LADDER, s-a reușit implementarea funcțiilor logice SI respectiv SAU. Acest cod va fi folosit și va rula ori de câte ori se va folosi un obiect de tipul acestei funcții bloc

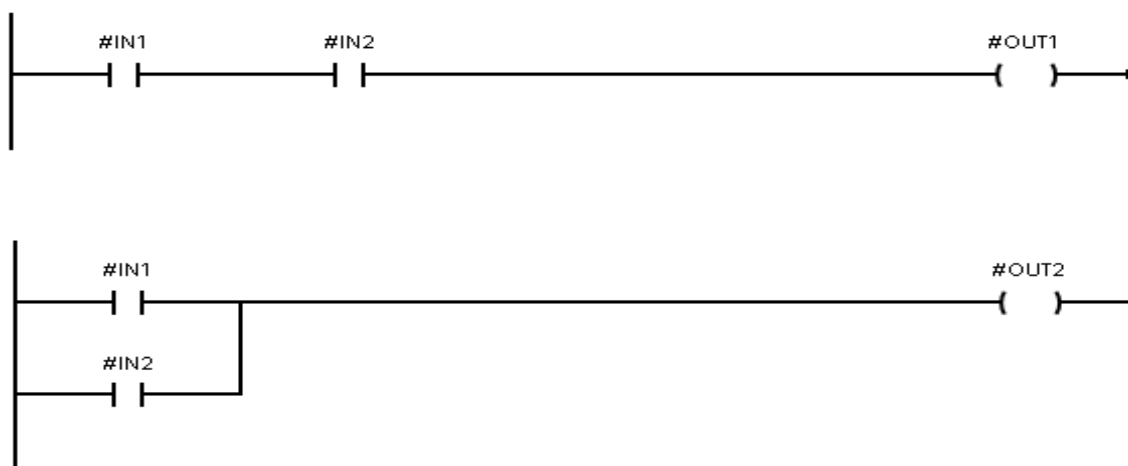


Fig 4.7.7 – Codul care implementează operațiile logice SI și SAU în cadrul funcției bloc

Funcția bloc implementată, va apărea în cadrul unei aplicații sub forma blocului prezent în cadrul figurii 4.7.8. Se pot observa în cadrul acestei figuri existența celor două intrări IN1 respectiv IN2, precum și existența celor două ieșiri OUT1 și OUT2. Aceste intrări/ieșiri ale funcției bloc sunt practic variabilele de intrare și variabilele de ieșire care au fost declarate în cadrul funcției bloc. Pe lângă aceste intrări și ieșiri care au fost amintite, se poate observa în cadrul figurii 4.7.8 existența și a altor două intrări/ieșiri EN respectiv ENO.

EN este folosit pentru a specifica momentul în care funcția bloc este activă, adică momentul în care intrările IN1 și IN2 sunt evaluate, iar rezultatul evaluării acestor mărimi este returnat prin intermediul ieșirilor OUT1 și OUT2. În momentul în care semnalul EN are valoarea 1 logic, funcția bloc este activă, iar în momentul în care EN are valoarea 0 logic funcția bloc este inactivă.

ENO este folosit pentru a preciza la ieșirea funcției bloc faptul că funcția bloc este activă sau nu la un anumit moment de timp. Dacă ENO este 1 logic, acest lucru înseamnă faptul că funcția bloc este activă, iar dacă ENO este zero logic, funcția bloc este inactivă la acel moment de timp.

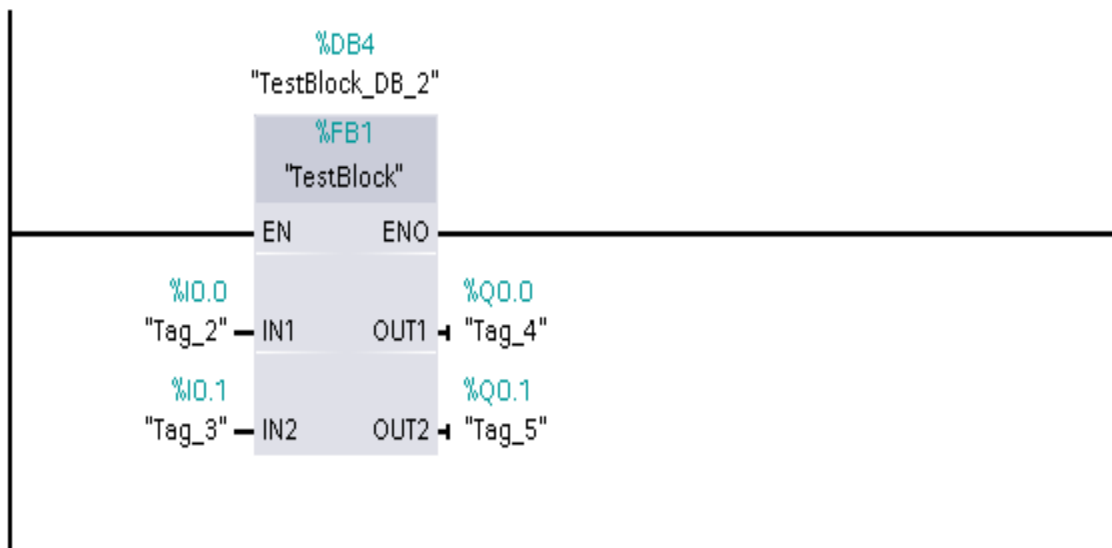


Fig 4.7.8 – Funcție bloc

5. Aplicații

În cadrul acestui capitol se vor prezenta câteva aplicații scrise utilizând limbajul de programare bazat pe diagrame LADDER. Aceste aplicații sunt destinate rezolvării unei cerințe din domeniul tehnic și sunt destinate unor domenii diverse. Fiecare din aplicațiile din cadrul acestui capitol folosesc cunoștințele prezentate în cadrul acestei lucrări și sunt destinate pentru a realiza o vedere de ansamblu asupra domeniilor în care se pot utiliza automate programabile pentru controlul unui sistem.

5.1 Sistem de trecere la nivel cu calea ferată

Se consideră că se dorește să se implementeze folosind un automat un sistem de trecere la nivel cu calea ferată. Sistemul trebuie să poată fi pornit și oprit cu ajutorul a doi senzori care detectează apropierea trenului (I0.0) respectiv momentul în care trenul a trecut de trecerea la nivel (I0.1). Sistemul conține două lumini de semnalizare care trebuie să se aprindă intermitent (Q0.0 respectiv Q0.1) la intervale de 2 secunde. Se consideră ca și ipoteză simplificatoare faptul că trenul trece într-o singură direcție, logica curentă putând fi extinsă cu ușurință pentru a acoperi și cazul când trenul poate să sosească din oricare direcție.

Rezolvarea problemei enunțată mai sus se poate observa mai jos, în cadrul figurii 5.1.1. Se poate observa faptul că s-au folosit două timere de tipul TON. Cu ajutorul acestor două timere care se repornesc unul pe celălalt se realizează temporizarea dorită de 2 secunde necesară pentru aprinderea becurilor sistemului de trecere la nivel cu calea ferată. Ieșirile Q0.0 și Q0.1 sunt setate la valoarea corespunzătoare folosind variabilele de memorie care sunt utilizate și pentru pornirea în mod ciclic a ansamblului format din cele două timere. Pornirea și oprirea sistemului se face prin verificarea stării celor doi senzori care precizează prezența trenului în apropierea trecerii la nivel cu calea ferată (pentru pornirea ansamblului s-a adăugat o condiție suplimentară și anume verificarea simultană a celor doi senzori de prezență chiar dacă nu era nevoie pentru a sugera modul în care trebuie extins sistemul pentru a se detecta sosirea trenului din ambele direcții). În practică sistemul prezentat poate să fie extins prin utilizarea de senzori suplimentari care au rolul

de a oferi redundanță sistemului în cazul în care unu senzor din acest ansamblu cedează la un moment dat.

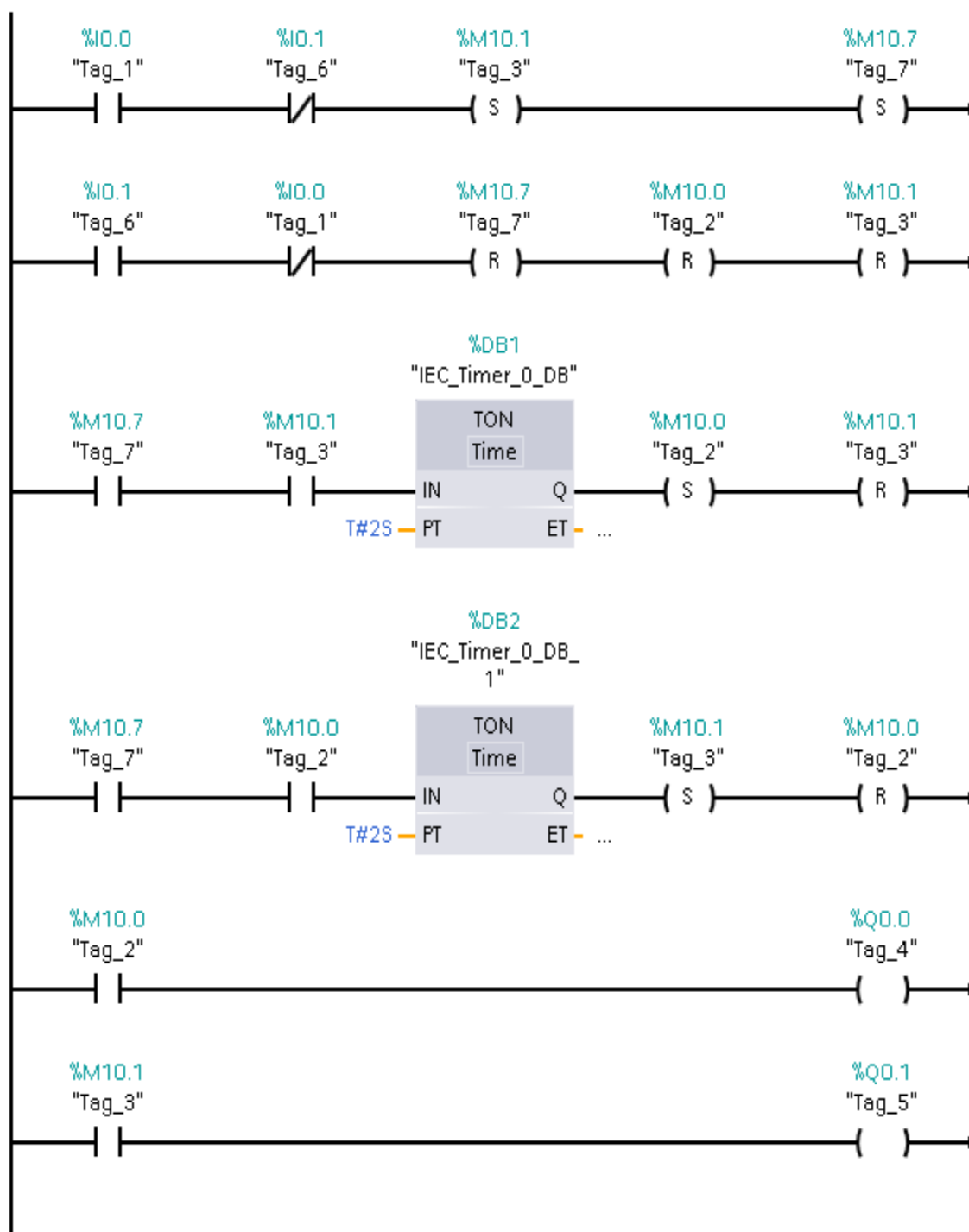


Fig 5.1.1 – Sistem de trecere la nivel cu calea ferată

5.2 Parcare de mașini

Se consideră o parcare cu o capacitate de 10 locuri. Parcare este prevăzută la intrare cu doi senzori care detectează prezența unei mașini (I0.0 și I0.1). Cei doi senzori trebuie să fie folosiți pentru a detecta sensul de deplasare a mașinilor care intră sau ies din parcare (se consideră faptul că la un moment dat de timp doar o singură mașină poate să intre sau să iasă din parcare). Parcare este prevăzută la intrare cu un semafor cu două becuri (Q0.0 și Q0.1) care indică dacă mai sunt locuri disponibile în parcare. În momentul în care parcare este plină și nu mai sunt locuri disponibile trebuie să semnalizeze aceasta cu ajutorul semaforului. Pentru contorizarea numărului de mașini existente în parcare se folosește un counter de tipul CTUD.

În cadrul figurii 5.2.1 este prezentată soluția de implementare aleasă pentru rezolvarea acestei cerințe. Pentru contorizarea numărului de mașini existente în parcare se utilizează un counter de tip CTUD care poate să numere evenimente. Acest tip de counter se pretează pentru foarte bine pentru situația în cauză deoarece trebuie să numărăm atât mașinile care intră în parcare cât și mașinile care ies din parcare.

Se poate observa în figura 5.2.1 faptul că se memorează într-o zonă de memorie momentul în care cei doi senzori reprezentați de intrările I0.0 și I0.1 sunt activi. După aceasta se verifică care din cei doi senzori reprezentați de intrările I0.0 și I0.1 se dezactivează primul pentru a se putea detecta sensul de deplasare a mașinii (mașina poate să intre sau poate să iasă din parcare). După ce se detectează direcția de deplasare a mașinii se folosesc două zone de memorie pentru a transmite counterului folosit faptul că trebuie să numere în sens crescător sau în sens descrescător. De asemenea în momentul în care se detectează direcția de deplasare a autovehicolului trebuie să se reseteze și zona de memorie care a fost folosită pentru a reține faptul că cei doi senzori I0.0 și I0.1 au detectat prezența unei mașini. În acest fel, întregul program va fi reinițializat pentru un nou ciclu de contorizare a mașinilor.

Cele două becuri ale semaforului se aprind în funcție de valoare ieșiri QU a counter-ului de tip CTUD folosit, counter-ul setând starea unui bec, iar starea celui alt bec va fi complementară stării becului setat de counter.

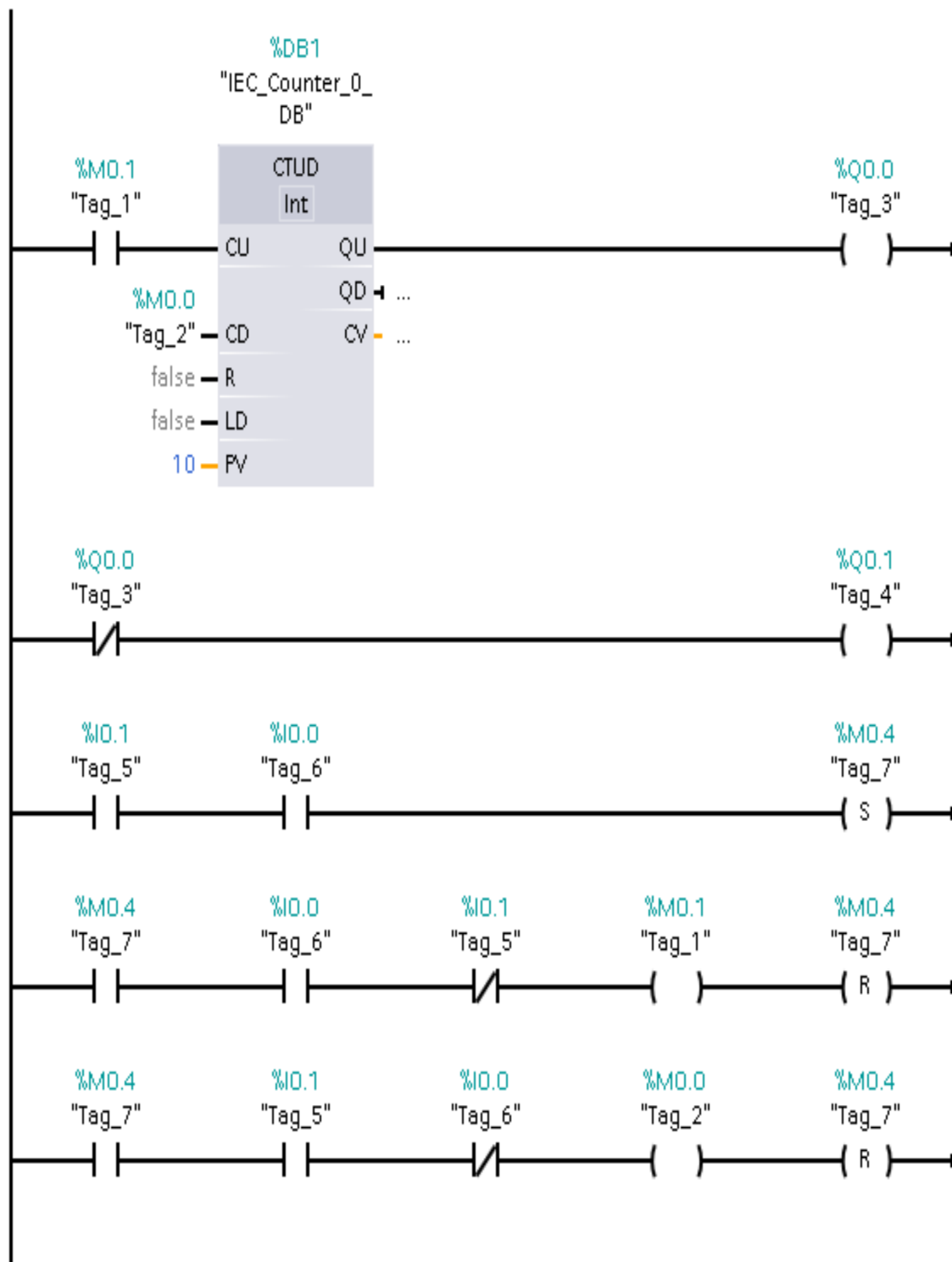


Fig 5.2.1 – Parcare de mașini

5.3 Afișaj șapte segmente

Se consideră că se dorește implementarea funcționării unui afișaj cu șapte segmente utilizând un automat programabil. Un afișaj cu șapte segmente este prezentat în figura 5.3.1. Acesta este utilizat pentru afișarea de valori numerice. El funcționează pe baza următorului principiu: la intrarea sa primește un cod binar, care este decodificat și este afișat la ieșirea sa. Codul binar primit la intrarea afișajului este codificat pe patru biți, deoarece pentru reprezentarea cifrelor de la 0 la 9 în sistemul binar este necesară utilizarea a patru biți. În funcție de valoarea acestui cod binar, trebuie ca anumite segmente din cadrul afișajului cu șapte segmente să fie aprinse pentru a crea imaginea numărului în sistemul zecimal reprezentat de codul în sistemul binar primit la intrarea afișajului.

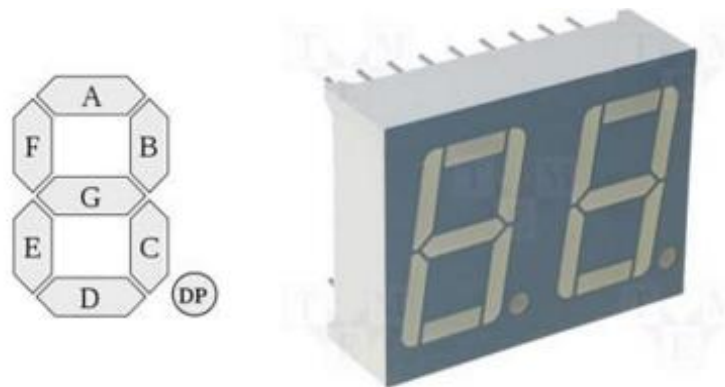


Fig 5.3.1 – Afișaj șapte segmente

Se dorește să se implementeze funcționarea unui afișaj cu șapte segmente utilizând un automat programabil. Ținând cont de modul în care un afișaj cu șapte segmente (modul de funcționare a unui afișaj cu șapte segmente a fost descris mai sus) se dorește să se implementeze funcționarea acestuia utilizând un automat programabil. Pentru implementarea funcționării afișajului cu șapte segmente trebuie să se determine starea fiecărui segment (aprins / stins) în funcție de codul binar primit la intrare. Pentru aceasta este necesară efectuarea unei operații de minimizare în urma căreia va rezulta o funcție logică de patru variabile pentru fiecare din cele

șapte segmente. Fiecare din cele șapte funcții logice va returna un rezultat de tip boolean (0 sau 1) care reprezintă defapt starea unui segment din afișajul cu șapte segmente (aprins sau stins).

Pentru determinarea funcțiilor logice pentru fiecare din cele șapte segmente, sunt necesare efectuarea a două operații. Prima operație este reprezentată de scrierea sub formă tabelară a stărilor celor șapte segmente în funcție de codul binar primit la intrarea afișajului. Această reprezentare sub formă tabelară este prezentată mai jos în cadrul figurii 5.3.2. În acest tabel, cu A,B,C,D s-au notat variabilele de intrare (cele care specifică valoarea în binar a numărului care trebuie afișat).

ABCD	a	b	c	d	e	f	g
0000	1	1	1	1	1	1	0
0001	0	1	1	0	0	0	0
0010	1	1	0	1	1	0	1
0011	1	1	1	1	0	0	1
0100	0	1	1	0	0	1	1
0101	1	0	1	1	0	1	1
0110	1	0	1	1	1	1	1
0111	1	1	1	0	0	0	0
1000	1	1	1	1	1	1	1
1001	1	1	1	1	0	1	1

Fig 5.3.2 – Tabel funcționare afișaj șapte segmente

Pe baza tabelului de funcționare din cadrul figurii 5.3.2 se construiesc pe rând celelalte șapte tabele folosite pentru minimizarea funcțiilor logice corespunzătoare fiecărui segment utilizând metoda de minimizare Veitch-Karnaugh. Această metodă de minimizare folosește pentru minimizare tabele care au un număr de celule egal cu 2 la puterea n, unde n este numărul variabilelor de intrare ale funcției. În cadrul acestor tabele variabilele de intrare sunt dispuse pe

linii și pe coloane în așa fel încât fiecare celulă să difere de o celulă vecină printr-o singură modificare a unei variabile de intrare (variabilele de intrare sunt adiacente). După dispunerea pe linii și coloane a variabilelor de intrare, tabelul de minimizare se completează folosind informațiile din tabelul 5.3.2.

O astfel de diagramă Veitch-Karnaugh care este folosită pentru minimizarea funcției logice folosită pentru stabilirea stării segmentului a al afișajului cu șapte segmente este prezentată mai jos în cadrul figurii 5.3.3. În cadrul acestei figuri, cu * s-a notat valorile variabilelor de intrare care nu prezintă interes pentru determinarea valorii funcției logice.

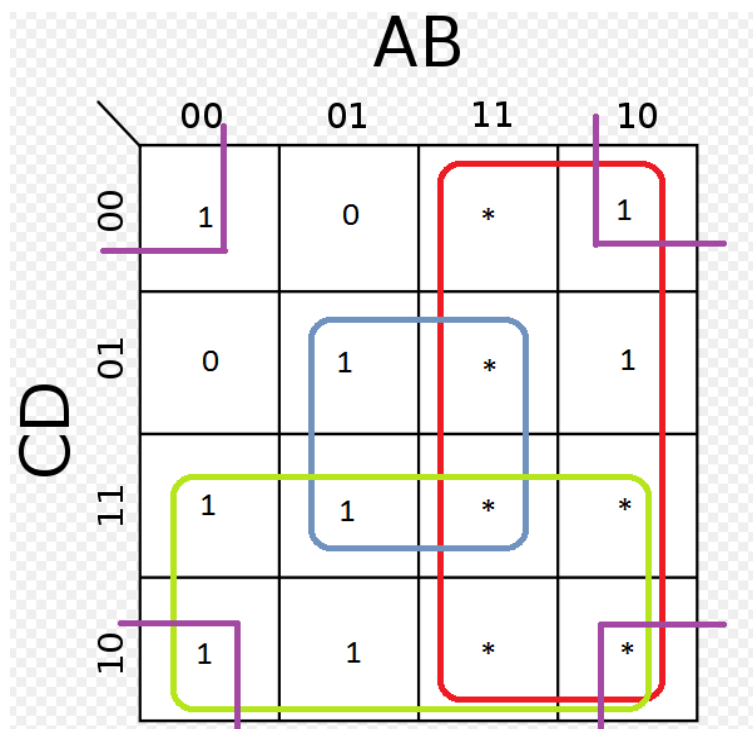


Fig 5.3.3 – Diagrama Veitch-Karnaugh pentru minimizarea funcției logice care determină starea segmentului a

Pentru determinarea funcției care determină starea segmentului a, se grupează celulele care conțin valori de 1 din tabelul din figura 5.3.3 în grupări care sunt multiplu de puteri de ale lui 2 (2,4,8). Aceste grupări pot să includă valori marcate cu * dar nu trebuie să includă valori de 0. Valoarea logică a funcției este dată de variabilele care nu își schimbă starea în gruparea respectivă. Pentru cazul de față valoarea funcției care determină starea segmentului a este:

$$f_a = C + A + BD + \bar{B}\bar{D}$$

Dacă se realizează o corespondență între variabilele de intrare A,B,C și D și intrările automatului (I0.0,I0.1,I0.2,I0.3) se poate implementa funcția logică care determină starea segmentului a (se realizează de asemenea corespondența între segmentul a și ieșirea Q0.0 a automatului). Implementarea lui f_a este prezentată mai jos în cadrul figurii 5.3.4.

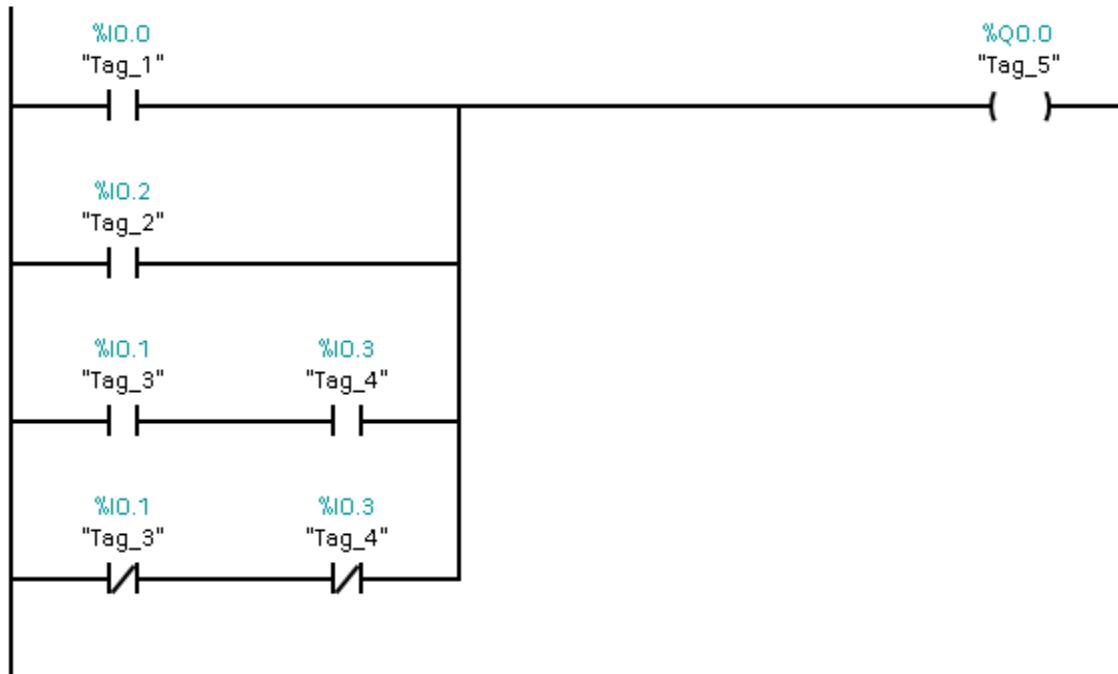


Fig 5.3.4 – Implementare f_a